Computer Vision · Week 4: GANs

Beginner study guide: plain-language notes + rebuilt course notebooks with outputs

Part 1: notes (01 GAN basics · 02 training dynamics · 03 DCGAN & conditional GAN)
Part 2: the three course notebooks, rebuilt from the course guides and run

Plus the cheat sheet. Runnable notebooks are in the **notebooks/** folder next to this PDF.

01 · GAN basics: generator, discriminator and the minimax game

Covers: GAN Class 1 slides ("The GAN framework") and the guide to the course notebook `CV-Week4-Lecture1.ipynb`

Notebook: `notebooks/01_gan_basics_mnist.ipynb` (the course's MNIST GAN rebuilt and run, plus the in-class Tasks 1–4)

---

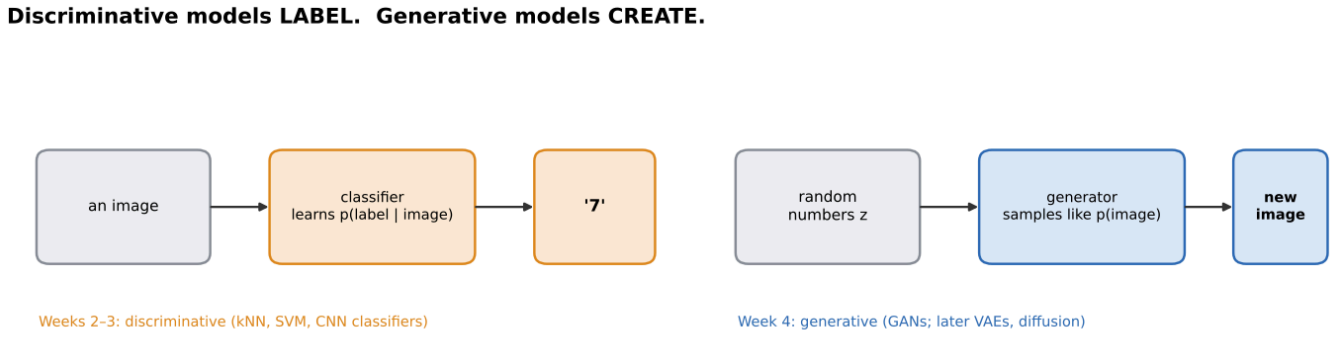
1. What does GAN stand for?

GAN = Generative Adversarial Network. Take the three words one at a time:

Letter	Word	Meaning
G	**Generative**	it **creates new data**: a new image, not a label. It answers "what could a new sample look like?"
A	**Adversarial**	it learns through a **contest between two opponents** with opposite goals
N	**Network(s)**	both players are ordinary **neural networks** trained with backpropagation. There are **two** networks, not one

Origin: Ian **Goodfellow** et al., "Generative Adversarial Nets", **NeurIPS 2014**.

Generative vs discriminative

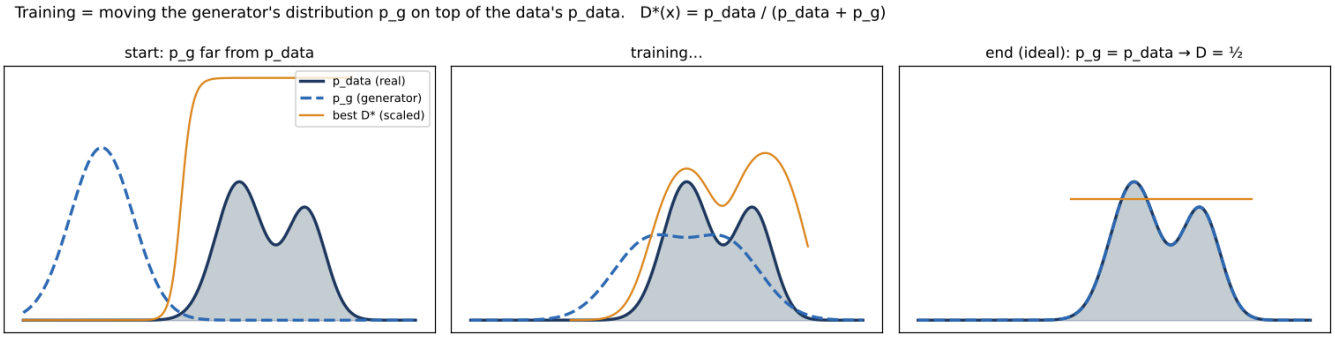


	Discriminative (what you know)	Generative (now)
Example	a classifier	a generator
Input → output	image → **label**	random numbers → **new image**
Learns	p(label | image)	to **sample like p(image)**

What "generate" really means

Every real photo is **one sample** from an unknown distribution **p_data**. We never see its formula, only examples. The generator defines its own distribution **p_g**: whatever comes out

when we feed it random noise. **Training = moving p_g until it sits on top of p_data.** Then every new sample looks like it came from the real collection.



What GANs are used for

- **New images:** realistic faces, objects and scenes that never existed (the StyleGAN family).
- **Image-to-image:** sketch → photo, day → night, horse → zebra (pix2pix, CycleGAN).
- **Super-resolution:** low-resolution → sharper (SRGAN).
- **Data augmentation:** extra synthetic training images when real ones are rare or private (e.g. medical).
- **Video and art:** predicting the next frame, generative art.
- ⚠️ **Caution:** the same power makes **deepfakes** possible. Detecting fakes is its own research area.

---

2. What does "adversarial" mean?

An **adversary** is an opponent (Latin *adversus*, "turned against"). Think of chess players, prosecution vs defence, or a sparring partner who fights you and so makes you better.

The counterfeiter and the police (from the original paper)

Story	GAN
Counterfeiter prints fake banknotes and tries to spend them	**Generator (G)**
Police inspect notes and try to detect the fakes	**Discriminator (D)**

> "Competition in this game drives both teams to improve their methods until the counterfeits are indistinguishable from the genuine articles." (Goodfellow et al., 2014)

The course's running example is the same story with a **flower forger** (G) and an **inspector** (D).

Round by round: - **Round 1:** G paints a random scribble and D catches it easily. - **Round 2:** G learns "flowers are colourful blobs", so D learns to check the shape. - **Round 3:** G adds petals and a centre, so D checks petal detail. - **Round N:** G paints convincing flowers, and D can only guess, about 50/50.

Each side's improvement forces the other to improve.

Zero-sum game: one side's gain is the other's loss. "Winner and loser" is the intuition. In the code, **both** networks take one gradient step on **every** batch: first D, then G.

What adversarial does NOT mean: hacking or attacking. It is also **not** "adversarial examples/attacks", which is a different topic where tiny input changes fool a trained classifier. The two networks are opponents in training but **partners in the result**: D's job is to push G to improve. After training we keep G and usually throw D away.

---

3. The generator (the forger)

Job: turn a **random vector z** (not a photo, not a label) into a **new image G(z)**.

```
class Generator(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc = nn.Sequential(
            nn.Linear(100, 256),     # 100 noise numbers → 256 hidden values
            nn.ReLU(),
            nn.Linear(256, 784),     # → 784 values = one per pixel (28 × 28)
            nn.Tanh())               # squash to −1…1, the same range as the
normalised real images
    def forward(self, x):
        return self.fc(x).view(-1, 1, 28, 28)   # 784 numbers → a 1-channel 28×28
image
```

- **Why random input?** G is **deterministic**: the same z always gives the same image. **All the variety comes from z.** Different z gives a different digit. `torch.randn(64, 100)` is an endless, easy supply. The space z lives in is the **latent space**.
- **Why Tanh?** Real images are normalised to −1…+1 (`Normalize((0.5,), (0.5,))`). If fake pixels could be 5.0, D could spot fakes from their range alone, and G would learn nothing useful.
- **The surprising part: G never sees a real image.** There is no target picture and no pixel-by-pixel comparison. It learns **only from D's reaction**: the gradient flows **backwards through D into G**. **D is G's only teacher.**
- **Best way to watch it learn:** keep one `z_fixed` and plot `G(z_fixed)` after every epoch. The input is fixed, so any change comes from G's weights.

4. The discriminator (the inspector)

Job: look at an image and output **one number, P(real)**, between 0 and 1.

```
class Discriminator(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc = nn.Sequential(
            nn.Linear(784, 256),
            nn.LeakyReLU(0.2),       # lets a little gradient through for negatives →
G keeps getting a signal
            nn.Linear(256, 1),
            nn.Sigmoid())            # a score → a probability 0…1
```

```
    def forward(self, x):
        return self.fc(x.view(-1, 784))   # flatten the image first
```

- **Its labels come for free:** an image picked from the dataset gets label **1**, and an image just painted by G gets label **0**. No human labelling. The label means "which pile it came from", not "is it pretty": a beautiful fake is still label 0 when training D.
- **Why LeakyReLU, not ReLU?** G learns only through gradients passing back through D. A plain ReLU passes **zero** gradient for negative inputs and can block that signal. LeakyReLU always lets a little through (×0.2).
- G and D are **mirror images**: G is 100 → 256 → 784, D is 784 → 256 → 1.

A tug-of-war over one number

For a fake image, p = D(G(z)). **D wants p → 0. G wants p → 1.** At the start p is tiny (e.g. 0.001). If G became perfect, the best D could do is **p = 0.5 for everything**: a coin flip.

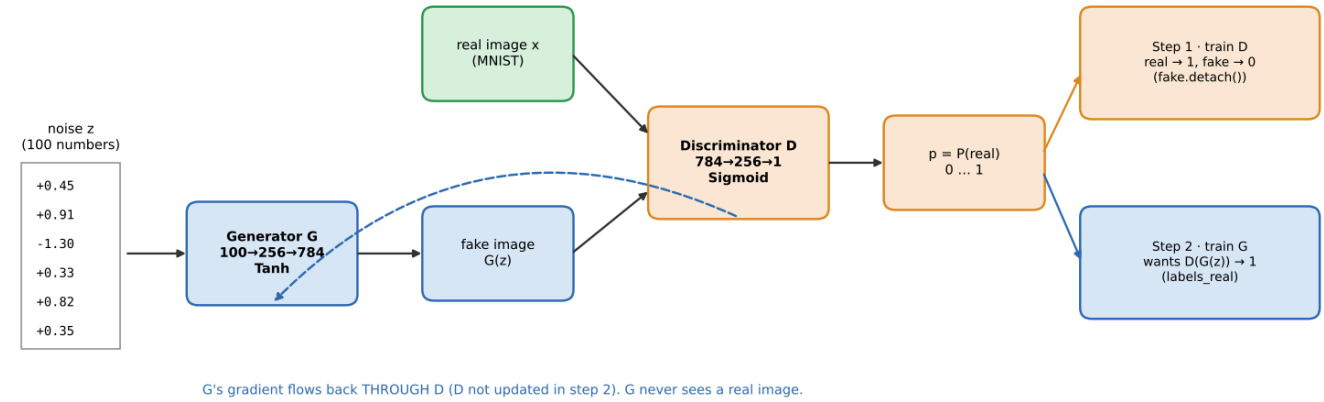
G vs D at a glance (cover the columns and test yourself)

	Generator G (forger)	Discriminator D (inspector)
Input	random noise z (100 numbers)	an image (real or fake)
Output	an image G(z)	one probability P(real)
Sees real data?	**never**	yes, half of what it sees
Wants on fakes	D(G(z)) → 1	D(G(z)) → 0
Wants on reals	(not involved)	D(x) → 1
Kept after training?	**yes**: it makes the images	usually discarded

---

5. Training: the rules of the game

The whole GAN in one picture (blue = forger G, orange = inspector D)



Every mini-batch plays one round in **two steps**.

Step 1: train D (G frozen)

```
optimizer_d.zero_grad()
outputs_real = discriminator(real)
loss_real = criterion(outputs_real, labels_real)        # real → target 1
noise = torch.randn(batch_size, 100)
fake = generator(noise)
outputs_fake = discriminator(fake.detach())             # .detach(): cut the wire
back to G
loss_fake = criterion(outputs_fake, labels_fake)        # fake → target 0
loss_d = loss_real + loss_fake
loss_d.backward()
optimizer_d.step()                                      # only D's weights change
```

Step 2: train G (D not updated)

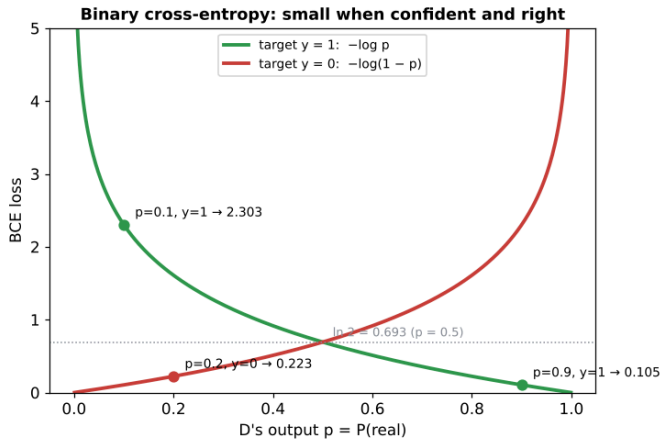
```
optimizer_g.zero_grad()
outputs_fake = discriminator(fake)                      # NO detach: gradients must
reach G
loss_g = criterion(outputs_fake, labels_real)           # target 1 for FAKES: "my
loss is low when D is fooled"
loss_g.backward()
optimizer_g.step()                                      # only G's weights change
```

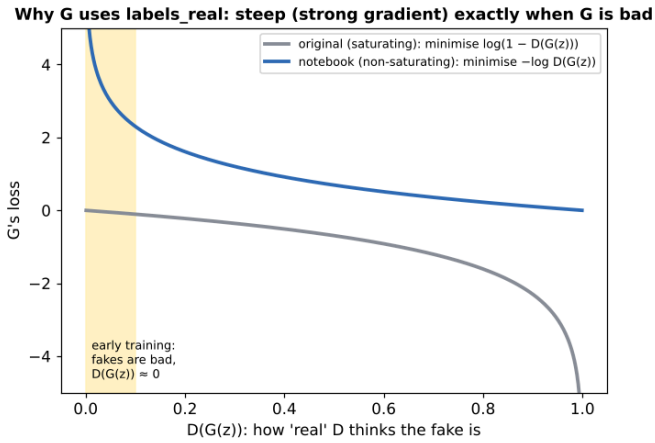
- **Two optimizers:** `optimizer_g` only knows G's weights and `optimizer_d` only knows D's. That is how the code keeps the steps separate.
- **Using `labels_real` for fakes is not cheating.** When D is trained, fakes get target 0 (the truth about where they came from). When G is trained, target 1 is **G's wish**, not a claim about the image.
- **Settings:** `nn.BCELoss()`, Adam with **lr = 0.0002** for both, batch 64 (938 batches per epoch, the last one has 32 images, hence `batch_size = real.size(0)`), `epochs = 2`.

The loss: binary cross-entropy (BCE)

BCE(p, y) = −[y·log p + (1 − y)·log(1 − p)]

So y = 1 gives a loss of −log p, and y = 0 gives −log(1 − p). The loss is small when the prediction is confident and right, and big when it is confident and wrong.





Case	y	p	Loss
D, real image, confident	1	0.9	0.105
D, real image, wrong	1	0.1	2.303
D, fake image, confident	0	0.2	0.223
G, the same fake (wants y = 1)	1	0.2	**1.609**

The same image and the same score 0.2 are good news for D (0.223) and bad news for G (1.609). That's "adversarial" in numbers.

The minimax objective (Goodfellow et al. 2014)

min_G max_D V(D, G) = E_x[log D(x)] + E_z[log(1 − D(G(z)))]

- **Term 1 (real images):** largest (0) when D(x) = 1. D maximises it. G can't touch it.
- **Term 2 (fake images):** D maximises it by pushing D(G(z)) → 0. G minimises it by pushing D(G(z)) → 1.
- **Minimax:** G minimises what the best D can maximise, a zero-sum game. Maximising V is the same as minimising the notebook's `loss_d`.

Where the game settles

- For a fixed G, the best D is **D*(x) = p_data(x) / (p_data(x) + p_g(x))**: "how much of the probability at x comes from real data?"
- The best G makes **p_g = p_data**, so **D*(x) = ½** everywhere: D can only flip a coin.
- This is a **Nash equilibrium** (chapter 02): neither player can improve by changing alone.
- **Reference numbers:** if D outputs 0.5 for everything, `loss_d` = 2 × ln 2 ≈ **1.386** and `loss_g` = ln 2 ≈ **0.693**.

The non-saturating generator loss (why the code differs from the formula)

Early in training the fakes are bad, so D(G(z)) ≈ 0. The original term log(1 − D(G(z))) is almost **flat** there, so G gets tiny gradients and barely learns (the grey curve above). The code instead minimises **−log D(G(z))** (BCE with target 1), which is **steep** near 0 and gives a strong push exactly when G needs it (the blue curve). It has the same goal (D(G(z)) → 1). The original paper suggests it too.

Recap in three languages

Story	Maths	Code
real photos	x ~ p_data	`for real, _ in train_loader`
forger's blank idea	z ~ N(0, I)	`torch.randn(batch_size, 100)`
forger paints	G(z)	`fake = generator(noise)`
inspector's verdict	D(x), D(G(z))	`discriminator(real)`, `discriminator(fake)`
inspector studies	max_D V	`loss_d = loss_real + loss_fake`
forger improves	min_G (non-saturating)	`criterion(outputs_fake, labels_real)`

Story	Maths	Code
fakes pass as real	p_g = p_data, D = ½	`loss_d ≈ 1.386`, `loss_g ≈ 0.693`

---

6. Reading your results (from the course)

- **The first rule: GAN losses alone don't tell you image quality. Always look at the images.** Each loss is measured against an opponent who is also changing.

You see…	It probably means…
blotchy, half-formed digits	normal for 2 epochs of a small MLP GAN
Loss D well below 1.386, Loss G above 0.693	D is still ahead, and G has more to learn
losses jump around between runs	they're last-batch snapshots of a moving target
many samples are the same digit	possible **mode collapse** (chapter 02)
Loss G keeps rising, samples look like noise	D may be too strong
shape-mismatch error	the noise size and `nn.Linear` input size disagree

> **What our rebuilt notebook actually got** (`notebooks/01_gan_basics_mnist.ipynb`, seed 0, CPU): - **The course's exact settings collapsed.** With Adam's default β1 = 0.9, every sample became the same blob, and a digit classifier read all 1,000 samples as "8". The Class 1 guide expected "blotchy digits", but the Class 2–3 guide reports exactly this collapse for the same settings (its "Run A"). GANs are that sensitive to seeds and settings. - **Losses (last batch of each epoch):** epoch 1 D 0.590 / G 1.404, epoch 2 D 1.042 / G 1.058, against the reference values 1.386 / 0.693. They look "reasonable" even though the output had collapsed: **losses don't show quality**. - **Experiment results (Task 3):** - A (5 epochs, watching fixed noise): the collapse slowly breaks. - B (latent size 10): still collapsed. - C (D's learning rate 10×): D wins outright (Loss D 0.048, Loss G 5.1). - **D (betas = (0.5, 0.999)): the fix.** All ten digits appear (42–166 of each per 1,000 samples).

Preview of what goes wrong (chapter 02): - **mode collapse:** one output that fools D, repeated; - **non-convergence:** the players chase each other in circles; - **D too strong:** G's gradient vanishes.

---

7. GANs among other generative models

	GAN	VAE	Diffusion
How it learns	generator vs discriminator game	encode → decode; reconstruction + KL loss	learn to remove noise a little at a time
Making a sample	one forward pass: **fast**	one decoder pass: fast	many denoising steps: **slow**
Typical images	**sharp**	often blurrier	sharp and varied

	GAN	VAE	Diffusion
Main difficulty	unstable training, mode collapse	blur; quality vs latent-structure trade-off	slow sampling, heavy compute

---

Exit ticket / check yourself

1. Write the full form of GAN. Explain "adversarial" in one sentence.
2. In the counterfeiter story, who is G and who is D?
3. D(G(z)) = 0.3 for a fake. Which way does D want it to move? Which way does G?
4. What does `fake.detach()` do in the discriminator step? Why is there no detach in the generator step?
5. Why does the generator's loss use `labels_real`?
6. Why Tanh in G, and why LeakyReLU in D?
7. What are the reference losses when D outputs 0.5 everywhere?
8. If D is thrown away after training, why do we need it at all?

Answers

1. Generative Adversarial Network. Two networks with opposite goals on the same score.
2. Counterfeiter = G (makes fakes). Police = D (detects fakes).
3. D wants it to go to 0. G wants it to go to 1.
4. It stops gradients flowing into G, so only D learns in step 1. In step 2 the gradient must travel through D back into G.
5. G's goal is for D to say "real". It is the non-saturating loss $-\log D(G(z))$, which gives strong gradients early on.
6. Tanh matches the $-1 \ldots 1$ range of the normalised real images. LeakyReLU keeps a small gradient flowing back to G.
7. loss_d = $2 \ln 2 \approx 1.386$ and loss_g = $\ln 2 \approx 0.693$.
8. There is no formula for "all digits", so G can't be scored directly. D learns that score from data, and its gradient is G's only teacher.

02 · GAN training dynamics: Nash equilibrium, instability, mode collapse

Covers: GAN Class 2 & 3 slides ("Training Dynamics") and the guide to the course notebook `GAN-Class-2-3-Training-Dynamics.ipynb`

Notebook: `notebooks/02_gan_training_dynamics.ipynb` (the course notebook rebuilt and run: the x·y game, the 8-cluster ring, five fixes, MNIST diagnostics)

---

1. Why GAN training is different

	Training a classifier	Training a GAN
Target	**fixed**: a 7 is always a 7	**moving**: G's "label" is D's opinion, and D changes every batch
What is optimised	one loss, minimised	one value V: **D maximises, G minimises**
Goal	the lowest point (a **minimum**)	a balance point (an **equilibrium**)
"Done" looks like	loss low and flat	D ≈ 0.5 **and** good, varied samples
Loss going up means	something got worse	maybe the **other player** got better
Analogy	hiking down a valley that stays still	hiking down a valley while someone reshapes the ground

A GAN is looking for a **saddle point**: lowest in G's direction, highest in D's. Gradient descent is built for valleys, not saddles.

True or false (from the slides): - "If G's loss goes up, G got worse." **False:** D may have improved instead. - "D's labels change during training." **False:** real → 1 and fake → 0 always. But the fakes D sees keep changing. - "A GAN is finished when both losses are as low as possible." **False:** both can't be low at once.

---

2. Games and Nash equilibrium

A game has: - **players** (G and D); - **strategies**: their choices (the values of each network's weights); - **payoffs**: what each gets (D gets V, G gets −V).

A **zero-sum game** is one where the payoffs add to zero, which is what the GAN's minimax objective is.

> **Nash equilibrium:** a choice for every player such that **no player can do better by changing only their own choice** while the others keep theirs. It is named after John Nash (1950).

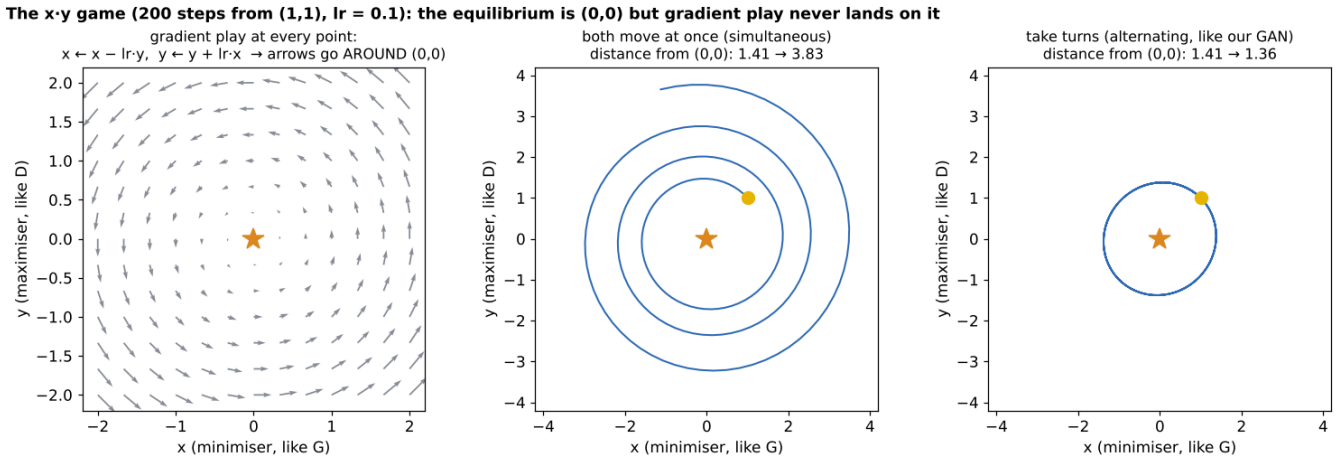
- **"No regrets" test:** ask each player "knowing what the others did, would you change your move?" If everyone says no, it's a Nash equilibrium.
- **Everyday example:** everyone drives on the left. Switching to the right alone only hurts you.
- **Penalty kick:** if the kicker always shoots left, the keeper learns to dive left. The kicker switches, the keeper follows, and they go round in circles. The equilibrium is for **both to randomise 50/50**, so nobody can exploit anybody. The guesser is left with a coin flip, just like **D = 0.5**.

The GAN's equilibrium (Goodfellow et al. 2014)

- **D's no-regret check:** for a fixed G, the best D is D*(x) = p_data / (p_data + p_g).
- **G's no-regret check:** against that D, the best G has **p_g = p_data**.
- Together: **p_g = p_data and D*(x) = ½ everywhere**, with V = −log 4 ≈ −1.386.
- ⚠️ The theory guarantees this point **exists**, not that gradient training **reaches** it.

The smallest GAN-like game: V(x, y) = x · y

- Player **x** wants V small (like G). Player **y** wants V large (like D).
- **Equilibrium: (0, 0).** If y = 0, V = 0 whatever x does, and vice versa. No regrets.
- **Gradient play:** x ← x − lr·y (descent), y ← y + lr·x (ascent).



- **Both move at once (simultaneous):** it spirals **outwards**. The distance grows by √(1 + lr²) each step: 1.41 → **3.83** after 200 steps.
- **Taking turns (alternating, like our GAN loop):** it **orbits forever** (1.41 → 1.36).
- **Why it circles:** each player's best move depends on the other's position. For example, at x > 0, y > 0, x moves left and y moves up. Follow the four quadrants and you get a **rotation**. Nobody's step aims at (0, 0).
- **A smaller learning rate only slows the circling** (lr 0.01 → 1.43, lr 0.1 → 3.83, lr 0.5 → it explodes). Fixing it needs a change to the **game or the optimiser**.

---

3. Instability

Four symptoms

1. **Oscillating losses:** D and G swing in turn and never settle (the circling above).

2. **Quality goes backwards:** good at one epoch, worse at the next. More training is not always better.
3. **One player wins outright:** D's loss drops near 0, G's loss climbs, and the samples stay noise.
4. **Fragile settings:** a small change of learning rate, optimiser setting or seed turns success into failure.

Cause 1: D too good → vanishing gradient (Arjovsky & Bottou 2017)

Early on, fakes and reals look completely different, so D separates them perfectly (D(G(z)) ≈ 0). A perfect D is **flat** around the fakes: nudging a fake still scores ≈ 0. A flat function has ≈ 0 gradient, and **G only learns through D's gradient**, so G stops learning. The non-saturating loss softens this, but **balance matters**.

Cause 2: momentum and step size

Adam's **β1** sets how much of the previous direction it keeps (its momentum). The default is β1 = 0.9 (used in Class 1). - Against a **fixed** landscape (a classifier), momentum helps. - Against a **moving opponent**, momentum makes a player chase where the opponent **used to be**. It overshoots and widens the circles.

That's why **DCGAN recommends Adam lr = 0.0002, β1 = 0.5**.

A better diagnostic than the loss: D's average opinions

mean D(x)	mean D(G(z))	Meaning	Check next
≈ 1	≈ 0	D winning easily	is G stuck? (vanishing gradient)
≈ 0.6–0.8	≈ 0.2–0.4	healthy contest, D slightly ahead	are the samples improving?
≈ 0.5	≈ 0.5	D can't tell, **or** D is broken	look at samples: good and varied?
swinging	swinging	oscillation	try β1 = 0.5 or a smaller lr

These numbers are already in the loop (`outputs_real.mean()`, `outputs_fake.mean()`), so tracking them costs nothing.

Cause	What you see	Typical fix
circling (game structure)	losses and D scores swing in turn	damping: β1 = 0.5, lower lr, better losses
D too strong	D(x) ≈ 1, D(G(z)) ≈ 0, G stuck	non-saturating loss, label smoothing, instance noise, WGAN-GP
momentum overshoot	wide swings, quality goes backwards	Adam β1 = 0.5
fragile settings	works with one seed, fails with another	test several seeds, keep checkpoints

---

4. Mode collapse

- A **mode** is a peak of the data distribution, i.e. one "kind" of sample:

- the notebook's 2-D ring has 8 clusters = **8 modes**;
- MNIST has about **10** (one per digit), plus smaller ones for writing styles;
- faces have many (age, hair, pose, lighting).
- **Mode collapse:** G produces only **one or a few** modes. Each sample may look real, but the **variety is missing**. The original paper called it the "**Helvetica scenario**": G maps too many values of z to the same x.
- **Mode hopping:** the collapsed output **jumps** around (all 1s now, all 8s an epoch later). G sits on a cluster, D learns "points here are fake", G jumps elsewhere, and D follows. It is the penalty-kick chase inside a real GAN.

Why it happens

1. **D judges one sample at a time.** A perfect "1" looks real on its own. D never sees that **all** the samples are 1s.
2. **G optimises against today's D.** The fastest way to lower its loss is to send **every** z to D's current favourite spot.
3. **Nothing rewards spreading out.** The standard loss scores each fake separately.

Kinds and detection

Kind	MNIST example
complete	every sample is the same "1"
partial	only 3, 7 and 1 appear
within a mode	all 7s in the same style
hopping	all 1s now, all 8s an epoch later

How to catch it: 1. Look at a **grid of 64+ samples**, not one image. 2. Plot a **fixed-noise grid** every epoch to spot hopping. 3. **Count classes with a pretrained classifier:** a flat histogram is good, a few tall bars means collapse. Read it together with the grid, because the classifier always answers *some* digit, even for junk. 4. Use **coverage metrics** (the notebook's "modes covered"; FID in research papers).

⚠️ **Calm losses can hide collapse.** In the course's run, the ring GAN covered **at most 1 of 8** clusters the whole time while the losses just wobbled normally.

---

5. The stabilisation toolbox (each fix targets a specific problem)

Fix	How it works	Targets
Non-saturating G loss (2014)	G minimises −log D(G(z))	vanishing gradients early
Adam β1 = 0.5 (DCGAN, 2015)	less momentum	oscillation, overshoot
One-sided label smoothing (2016)	real target 0.9 instead of 1	over-confident D
Instance noise (2016–17)	add noise to D's real and fake inputs	D separates too easily

Fix	How it works	Targets
TTUR (2017)	different learning rates for D and G	D lagging behind G
Minibatch discrimination / minibatch std (2016, 2018)	D sees **statistics of the whole batch**	**mode collapse**
Spectral normalisation (2018)	limits how steep D can be	unstable D gradients
WGAN / WGAN-GP (2017)	a new loss: a **distance**, not a probability	vanishing gradients, instability

The course's experiment on the collapsed ring GAN (seed 0, 4,000 steps):

Baseline	A smoothing	B noise	C TTUR	D minibatch std	E z_dim 2
1/8 modes	0/8, scattered	1/8	1/8, sharper	**8/8**	**8/8** (thin ring)

Our rebuilt notebook (seed 0) reproduced this: baseline 1/8 (hopping), A 0/8, B 1/8, C 2/8, **D 8/8, E 8/8**. Re-run with seed 1, both successful fixes still reached 8/8. Combining minibatch std with TTUR gave only 6/8 and 7/8: two good ideas don't simply add up.

> **Lesson:** match the fix to the cause. Collapse happens because D **can't see variety**. Minibatch std gives D exactly that information (the spread of the whole batch), so it fixed the collapse. Smoothing and noise calm an **over-confident** D, which is a different problem, so they didn't help. Better quality (TTUR) is not better variety. Our rebuilt notebook reruns this, and your numbers may differ.

WGAN (intuition only)

- A standard D outputs a probability. When fakes are far from the real data it just says "0, fake", however far away they are, so G gets a near-zero gradient and doesn't know which way to go.
- A **WGAN critic** outputs an unbounded score estimating the **earth mover's distance**: how much "earth" must move to turn the fake pile into the real pile. Farther away means a bigger score, so there is always a useful slope.
- The critic must be smooth (1-Lipschitz). WGAN uses weight clipping, and **WGAN-GP** adds a gradient penalty, which is more reliable.
- It is **not a cure-all**: in the course's short test it didn't fix the ring collapse.

MNIST: Run A vs Run B (course's run, seed 0, 3 epochs)

- **Run A** (Class 1 settings, β1 = 0.9): **collapsed completely**. Every fixed-noise image was the same noisy blob, and the digit counter said "1,000 eights" even though the grid showed no eights at all.
- **Run B** (β1 = 0.5 + label smoothing 0.9): recognisable digits with all **ten** present.
- **Our rebuilt run matched it:** Run A again gave "1,000 eights" from one collapsed blob, and Run B produced all ten digits (14–197 of each).
- **One optimiser setting changed everything, and the losses alone couldn't tell you which run was healthy.**

---

6. Today in five sentences

1. A GAN chases a **moving target**, so its losses don't show quality.
2. Its goal is a **Nash equilibrium**: p_g = p_data and D = ½, where no player gains by changing alone.
3. **Gradient play circles that point**, so training can oscillate, overshoot or stall (**instability**).
4. G can fool a one-sample-at-a-time D with a few outputs: **mode collapse** and hopping.
5. **Fixes target specific causes.** Diagnose with D scores, fixed-noise grids and histograms, then test.

Exit ticket / check yourself

1. Define Nash equilibrium. What is the GAN's equilibrium?
2. In the x·y game, why doesn't gradient play reach (0, 0)? Does a smaller learning rate fix it?
3. Losses look calm, but 900 of 1,000 samples are the digit 1. Name the problem and one fix.
4. Mean D(x) ≈ 1.0 and D(G(z)) ≈ 0.0 for a long time. What is happening to G?
5. Why can't a one-sample discriminator detect mode collapse?
6. Why does Adam β1 = 0.5 help GANs but momentum help classifiers?
7. Why did label smoothing not fix mode collapse on the ring?
8. Which diagnostic would you check first for mode collapse?

Answers

1. Choices where no player can do better by changing only their own. For a GAN: p_g = p_data and D outputs ½ for every image.
2. Each gradient depends on the other player, so the combined step rotates around (0, 0) instead of pointing at it. A smaller lr only slows the circling.
3. Mode collapse. Fix: minibatch std / minibatch discrimination (also check Adam β1 = 0.5).
4. D is too strong, so G's gradients vanish and it stops learning.
5. It scores each image alone. A single realistic image looks real even if every other sample is identical, so the loss has no penalty for missing variety.
6. With a moving opponent, momentum makes a player chase where the opponent used to be and overshoot. With a fixed landscape the downhill direction stays put, so momentum speeds things up.
7. It targets an over-confident D, not D's inability to see variety. Fixes only work on the problem they're designed for.
8. A grid of many samples (and/or a class histogram), not the losses.

03 · DCGAN and conditional GAN (GAN Class 4)

Covers: GAN Class 4 slides ("DCGAN and Conditional GAN") and the guide to the course notebook `GAN-Class-4-DCGAN-cGAN.ipynb`. It also covers the "later" cells of the Class 1 notebook (the DCGAN skeleton and the conditional-GAN assignment).

Notebook: `notebooks/03_dcgan_and_conditional_gan.ipynb` (the Class 4 notebook rebuilt and run)

> **The plan of Class 4:** the training game does **not** change (same losses, same D step then G step, Adam lr 0.0002, **β1 = 0.5**). We make two upgrades to the Class 1 GAN and measure each with the Class 2 & 3 diagnostics: 1. **DCGAN:** change **how G and D see images** (convolutions instead of Linear layers). 2. **Conditional GAN:** change **what G is told** (the digit label), so you can ask for a 7 and get a 7.

---

0. Revision of Classes 2 & 3 (the quiz the class starts with)

Question	Answer
A GAN's Nash equilibrium?	p_g = p_data. D outputs ½ for everything (loss_d = 2 ln 2 ≈ 1.386, loss_g = ln 2 ≈ 0.693)
Why do gradient steps circle the x·y equilibrium?	each player's step depends on the other's position, so the combined motion rotates around (0, 0)
D(x) ≈ 1, D(G(z)) ≈ 0 for a long time?	D wins easily and is flat around the fakes, so G's gradient vanishes and G stops learning
Every sample looks real but all are the same digit?	**mode collapse**. Targeted fix: **minibatch std** (lets D see the lack of variety)
The three diagnostics? Which one alone can fool you?	fixed-noise grid, D(x) vs D(G(z)), digit histogram. **The histogram** alone fools you, because it labels junk as some digit

Diagnose-it checkpoint: - (a) Losses calm, D(x) ≈ 0.6, D(G(z)) ≈ 0.4, but 64 near-identical 1s → **collapsed**. - (b) D(x) ≈ 0.99, D(G(z)) ≈ 0.01 for 2,000 steps, and the grid is unchanged → **D too strong**. - (c) Both D scores swing between 0.2 and 0.9, and quality goes up and down → **unstable**: lower β1 or the learning rate. - (d) D(x) ≈ 0.85, D(G(z)) ≈ 0.2, varied digits getting sharper, all ten in the histogram → **healthy**: train longer.

A new number used in Class 4: **evenness** (0 to 1) summarises the digit histogram. 1.0 means all ten digits appear equally often.

---

1. Why convolutions? (the baseline's problems)

The Class 1 **fully connected** generator (trained 2 epochs, β1 = 0.5), in the course's run: - **Background:** grey speckle, never clean black. Each of the 784 outputs is learned **on its own**, so no pixel knows its neighbours are black. - **Strokes:** blurry and broken. A stroke is a pattern

of **neighbouring** pixels, and a Linear layer has no idea which pixels are neighbours. - **Variety: no 6s at all** in 1,000 samples, with 8s and 3s dominating. **Evenness 0.79**.

Two assumptions a convolution makes, which fit images (you met them in Week 2–3 CNN classifiers):

Assumption	What it means	Effect in a GAN
Locality	each output pixel is computed from a small neighbourhood (a 4×4 window), not all 784 pixels	neighbours decide together → **smooth backgrounds**
Weight sharing	the same filter slides over the whole image	a "curve" detector learned in one corner works everywhere → a stroke is learned **once**

Size is not the point (parameter counts printed by the notebook):

	Fully connected	DCGAN
Generator	227,344	759,680 (3.3× larger)
Discriminator	201,217	**138,624 (31% smaller)**

The convolutional D is **smaller yet a much better critic**, because each filter is reused across the image. A better critic is a better teacher: G only learns through D's gradient.

---

2. DCGAN: Deep Convolutional GAN (Radford, Metz & Chintala, 2015)

Shrink in D, grow in G: the two size formulas

Layer	Output size	With k = 4, s = 2, p = 1
Conv2d (in D)	**(n + 2p − k) // s + 1**	halves: 28 → 14 → 7
ConvTranspose2d (in G)	**(n − 1)·s − 2p + k**	doubles: 7 → 14 → 28

The first G layer is `ConvTranspose2d(100, 128, 7, 1, 0)` on a 1×1 input: (1 − 1)·1 − 0 + 7 = **7×7**. That's how a noise vector becomes a 7×7 feature map.

What a transposed convolution does: "stamping"

Each input pixel **stamps a copy of the kernel** onto a bigger output, 2 pixels apart (stride 2). Where stamps overlap they **add**. The notebook's demo uses an all-ones 4×4 kernel (stride 2, padding 1) on the 2×2 input [[1, 0], [0, 2]]:

```
[[1, 1, 1, 0],
 [1, 3, 3, 2],
 [1, 3, 3, 2],
 [0, 2, 2, 2]]      ← the centre is 1 + 2 = 3, because the two stamps overlap there
```

In a real G the kernels are **learned**, so each feature stamps a learned patch of strokes. **Checkerboard artifacts:** if the kernel size isn't divisible by the stride, some pixels get more overlaps than others and a faint grid appears (Odena et al., 2016). **Kernel 4 with stride 2 overlaps evenly.**

The course's DCGAN for 28×28 MNIST

Network	Layer	Output shape
G	input z reshaped with `z.view(B, -1, 1, 1)`	[B, 100, 1, 1]
G	`ConvTranspose2d(100, 128, 7, 1, 0)` + BatchNorm + ReLU	[B, 128, 7, 7]
G	`ConvTranspose2d(128, 64, 4, 2, 1)` + BatchNorm + ReLU	[B, 64, 14, 14]
G	`ConvTranspose2d(64, 1, 4, 2, 1)` + **Tanh**	[B, 1, 28, 28]
D	`Conv2d(1, 64, 4, 2, 1)` + LeakyReLU(0.2) (no BN on D's input)	[B, 64, 14, 14]
D	`Conv2d(64, 128, 4, 2, 1)` + BatchNorm + LeakyReLU(0.2)	[B, 128, 7, 7]
D	`Conv2d(128, 1, 7, 1, 0)`, then `.view(-1, 1)`	[B, 1]: one raw score

- In G the image grows while the channel count shrinks, and D is the mirror image.
- `bias=False` before BatchNorm, because BN has its own shift.
- D ends with a **raw score** (no Sigmoid), because training uses `BCEWithLogitsLoss` (Sigmoid + BCE in one numerically stable step).
- `init_weights`: conv weights from N(0, 0.02), BatchNorm scales from N(1, 0.02), as in the paper.

The DCGAN recipe (seven rules, and why)

Rule	Why it helps
strided convolutions, **no pooling**	the network learns how to shrink and grow
BatchNorm in G and D	stable activations, helps stop G collapsing to one output
no BN on G's output or D's input	BN on every layer made samples oscillate
ReLU in G, **Tanh** output	output in [−1, 1], the range of the normalised images
LeakyReLU(0.2) in D	gradients still flow for negative inputs
Adam lr 0.0002, **β1 = 0.5**	less momentum in a moving game (Class 2 & 3)
weights from N(0, 0.02)	a small, standard start
(also) no fully connected hidden layers	

Results in the course's run (2 epochs, about 2.5 min on CPU)

	Fully connected	DCGAN
Look	grey speckle, blurry	**black backgrounds, solid connected strokes** (some shapes still malformed: 2 epochs is short, real runs train 25+)
Evenness	0.79	**0.93**
Rarest digit	6: none at all	1: 12 of 1,000

	Fully connected	DCGAN
D(x) / D(G(z))	≈ 0.7 / 0.4	≈ 0.9 / 0.15: D is clearly ahead, but D(G(z)) isn't stuck at 0 and the samples improve → **healthy** (case d)

Our rebuilt notebook vs the course guide:

	Our run	Course guide
CNN digit counter accuracy	98.1%	98.1%
MLP GAN: evenness, rarest digit	0.816, 6 (4 of 1,000)	0.786, 6 (0)
DCGAN: evenness, rarest digit	0.922, 1 (10)	0.929, 1 (12)
conditional accuracy	**84.4%**	84.4% (same total by coincidence; the per-digit split differs)
mismatch test (7s scored with label 7 vs label 1)	0.248 vs 0.063	0.315 vs 0.141
assignment C: `nn.Embedding(10, 50)` instead of one-hot	78.7% conditional accuracy (one seed, 2 epochs, so not proof that embeddings are worse)	—

Lesson: the architecture changed **what kind of mistakes** G makes. It is undertrained, not broken. Our rebuilt notebook reruns all of this, and your numbers will differ slightly.

The Class 1 skeleton (CelebA faces, 32×32 colour)

`ConvTransposе2d(100, 512, 4, 1, 0)` → 4×4 → 8×8 → 16×16 → **[B, 3, 32, 32]**. The calculation: $(1-1)\cdot 1 - 0 + 4 = 4$, then $(4-1)\cdot 2 - 2 + 4 = 8$, and so on. One more `ConvTranspose2d(…, 4, 2, 1)` gives **64×64**, the standard DCGAN face size.

---

3. Conditional GAN: choose the digit (Mirza & Osindero, 2014)

The problem

```
for real, _ in train_loader:     # Classes 1, 2 and 3: the _ is the digit label, thrown away!
```

An unconditional G(z) gives **some digit chosen by chance**. To get a 7 you'd have to sample, classify and throw away nine out of ten, and if G has partly collapsed, a rare digit may **never** come out. **Goal:** G(z, y) = "draw digit **y**, in the style **z**".

The idea: give the label y to BOTH players

min_G max_D E[log D(x, y)] + E[log(1 − D(G(z, y), y))]

- **G(z, y):** "draw a digit y, in style z".
- **D(x, y):** "is x a **real image of digit y**?" A fake can now fail in **two ways**: it looks fake, **or** it is the wrong digit.

- **Why D must see the label too:** if only G had it, D would accept a perfect 3 labelled "7". G would get no signal to obey y and would learn to **ignore** it.

How a network "reads" a label: one-hot, concatenated

- **One-hot:** 7 → [0,0,0,0,0,0,0,1,0,0].
- **Into G:** glue it to z, so G's first layer gets **110 input channels** (100 noise + 10 label): `torch.cat([z, one_hot(y, 10)], dim=1)`.
- **Into D:** spread the one-hot over **10 image-sized maps** (28×28 each, the true label's map all ones, the rest all zeros) and stack them on the image, giving **11 input channels**: `torch.cat([x, y_maps], dim=1)`.
- Everything else is the DCGAN unchanged (`CondDCGenerator`, `CondDCDiscriminator`).
- In training, G draws the **real batch's labels** and D scores each fake **with the same label G was given**. Scoring with a different random label would let D learn the shortcut "mismatch = fake", and G's gradient would push it towards the wrong label.

Results in the course's run (2 epochs, about 3.3 min on CPU)

- A 10×10 grid where **row r asks for digit r** shows the requested digit in almost every cell, with **varied handwriting within each row** (that variety comes from z). Shapes are cleaner than the unconditional DCGAN's, because knowing the digit makes G's job easier.
- **Conditional accuracy:** generate 100 samples per digit and let the notebook's CNN digit counter (98.1% test accuracy) read them. **84.4%** were the digit asked for.

Digit	0	1	2	3	4	5	6	7	8	9
Accuracy (%)	97	92	84	82	73	88	82	93	84	69

- The largest confusions are between digits that share strokes: asked for 9 and saw a 7 (13×), 2 → 3 (12×), 3 → 5 (11×), 4 → 9 (10×). Part of the 15.6% error is the classifier's own mistakes on unusual handwriting.
- **Mismatch test:** D scored 200 generated 7s at **0.315 with label 7** but **0.141 with label 1**. D has learned to check the image–label **match**, which is the signal that teaches G to obey y.

Content vs style

- **The label picks the digit (content). The noise picks the handwriting (style).**
- **Style vs content grid:** one fixed z per row and labels 0–9 across the columns. Thickness, slant and stroke style stay similar across the row while the digit changes.
- **Interpolation:** fix the label and walk z from z_a to z_b in small steps. The handwriting changes **smoothly**, so G has learned a continuous **style space** (a theme of the next topic, VAEs).

True or false (slides)

- "Only the generator needs the label." **False:** D needs it to punish the wrong digit.
- "A cGAN can't suffer mode collapse." **False:** it can collapse **within** a class (every 7 identical). A grid row shows it, while conditional accuracy does not.
- "100% conditional accuracy proves the cGAN is perfect." **False:** it measures **control**, not variety or quality.

The same idea with bigger conditions

Condition y	Example	Model
class label	"draw a 7"	cGAN, AC-GAN
an image	sketch → photo, map → satellite	pix2pix (paired images)
another domain	horse ↔ zebra, no pairs	CycleGAN
text	"a small yellow bird"	text-to-image GANs, later diffusion (Stable Diffusion's prompt is a condition too)

The pattern is always the same: **encode y as numbers and give it to both G and D**, so the critic can check the match.

---

4. Assignment (choose one, from the course)

Option	Task
A · Fashion-MNIST	train the DCGAN on `datasets.FashionMNIST` (same 28×28) and submit a fixed-noise grid per epoch with a note
B · DCGAN faces	extend the Class 1 skeleton to 64×64, write a mirror D, train on CelebA (GPU recommended)
C · Embedding	replace the one-hot in `CondDCGenerator` with `nn.Embedding(10, 50)` and compare conditional accuracy

Next topic: autoencoders and VAEs (encoder, bottleneck, decoder, reconstruction loss).

Exit ticket / check yourself

1. Why does a fully connected generator give speckled backgrounds while a DCGAN gives clean ones?
2. ConvTranspose2d(k = 4, s = 2, p = 1) on 16×16? Conv2d(k = 4, s = 2, p = 1) applied three times to 64×64?
3. What does `ConvTranspose2d(100, 128, 7, 1, 0)` do to a 1×1 input?
4. Why no BatchNorm on G's last layer? Which DCGAN rule did we already use in Class 2 & 3?
5. The DCGAN's D has fewer parameters than the MLP's D. Why is it a better critic?
6. Why must D see the label in a cGAN? How is the label given to D in the course's notebook?
7. What does conditional accuracy measure, and what does it miss?
8. In a cGAN, what does the label control and what does the noise control?

Answers

1. A fully connected G learns each of the 784 outputs separately, so neighbouring pixels never agree. Convolutions compute each output from a small neighbourhood with shared filters, so backgrounds come out smooth and strokes connected.

2. $(16 - 1)\cdot 2 - 2 + 4 = 32\times 32$. 64 → 32 → 16 → 8.
3. It makes it 7×7 with 128 channels: $(1 - 1)\cdot 1 - 0 + 7 = 7$.
4. G's output must match the real pixel values, and BN there caused instability. $\beta 1 = 0.5$ (less momentum in a moving game).
5. Each small filter is reused across the whole image (weight sharing and locality), so it gets more useful features from fewer weights, and G learns from a better teacher.
6. Otherwise D accepts realistic images of the wrong digit and G learns to ignore the label. The one-hot is spread over 10 image-sized maps stacked on the image (11 input channels).
7. Whether G draws the requested digit (control). It misses variety and quality, e.g. collapse within a class.
8. The label controls the content (which digit). The noise controls the style (thickness, slant, loops).

01 · GAN basics: a digit forger on MNIST

The Class 1 notebook (CV-Week4-Lecture1.ipynb), rebuilt, explained line by line, and all four in-class tasks done for real

Covers (GAN Class 1 slides): what GAN stands for · Generator and Discriminator · binary cross-entropy · the training loop (D step, then G step) · the minimax objective and the 1.386 / 0.693 reference numbers · the non-saturating generator loss · Tasks 1–4 and the exit ticket.

Where the code comes from. The course notebook itself was not handed out, but the course guide *"GAN Class 1 Notebook Explained"* (in `course-files/`) quotes every line of its GAN cell. The code below is that code, with the **same class names, layer sizes and variable names** (`optimizer_g`, `optimizer_d`, `labels_real`, `labels_fake`, `loss_d`, `loss_g`, `fake.detach()` ...), so you will recognise it in the exam. Lines marked `# (added)` are extras for this notebook (seeds, timing, record keeping); they do not change what the GAN does.

Run each cell with **Shift + Enter** and read the text around it. The whole notebook runs on a laptop CPU in a few minutes.

> **The story (from the slides).** A **forger** (the Generator, G) paints fake pictures but is never allowed to look at real ones. An **inspector** (the Discriminator, D) is shown real pictures and the forger's fakes and must say how likely each one is to be real. The forger learns *only* from the inspector's reactions. Round after round both improve, until the inspector can only guess: it says **0.5** for everything.

0 · Setup

Install once from a terminal: `pip install torch torchvision matplotlib`. On Google Colab everything is already installed. No GPU is needed.

```
%matplotlib inline
import copy
import time
import math
import numpy as np
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torchvision.utils import make_grid
from torch.utils.data import DataLoader

torch.manual_seed(0)          # (added) same random numbers every run, so your output matches the text
np.random.seed(0)
plt.rcParams["figure.dpi"] = 70
print("torch", torch.__version__, "| GPU available:", torch.cuda.is_available())
```

```
torch 2.14.1+cu130 | GPU available: False
```

Words you need first

Term	Plain meaning	In the code
Generator (G)	network that turns random numbers into an image (the forger)	`Generator` class
Discriminator (D)	network that takes an image and outputs P(real), a number from 0 to 1 (the inspector)	`Discriminator` class
Noise vector z	a list of 100 random numbers fed to G; a different z gives a different image	`torch.randn(batch_size, 100)`
Real image x	an image from the dataset	`real`
Fake image G(z)	an image made by the generator	`fake`
Label / target	the answer we want D to give: 1 = real, 0 = fake	`labels_real` , `labels_fake`
BCE	binary cross-entropy: loss for yes/no predictions	`nn.BCELoss()`
Batch / epoch	64 images at a time / one pass over all 60,000 images	`batch_size=64` , `epochs = 2`
Optimizer	updates a network's weights using gradients	`optim.Adam(...)`

If you know how a classifier is trained (forward pass → loss → `backward()` → `optimizer.step()`), you already know every building block. **A GAN simply trains two networks in turns, against each other.**

In the course notebook the whole GAN is **one code cell** (cell 5). Here it is split into small cells so each part can be explained, in exactly the same order: data → Generator → Discriminator → loss and optimizers → training loop → plot 16 samples.

1 · The data: MNIST

```
# Dataset  (course code)
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])
train_data = datasets.MNIST(root='./data', train=True, transform=transform, download=True)
train_loader = DataLoader(train_data, batch_size=64, shuffle=True)

print("training images:", len(train_data))
print("batches per epoch:", len(train_loader), "(937 full batches of 64 + one of",
len(train_data) - 937 * 64, ")")
```

```
training images: 60000
batches per epoch: 938 (937 full batches of 64 + one of 32 )
```

- **MNIST** = 60,000 training images of handwritten digits 0–9, each **28 × 28 = 784** grey pixels. `download=True` fetches it into `./data` the first time (about 10 MB).
- `ToTensor()` turns each image into a tensor of shape **(1, 28, 28)** with values 0 (black) to 1 (white).

- `Normalize((0.5,), (0.5,))` computes (pixel − 0.5) / 0.5, so **0 → −1 and 1 → +1**. Remember this range: the Generator must produce the same range.
- `batch_size=64`, `shuffle=True`: the loop receives 64 images at a time, in a new random order every epoch.
- The dataset also has the digit labels, but **this GAN never uses them**. It learns what digits look like in general, not which one is which.

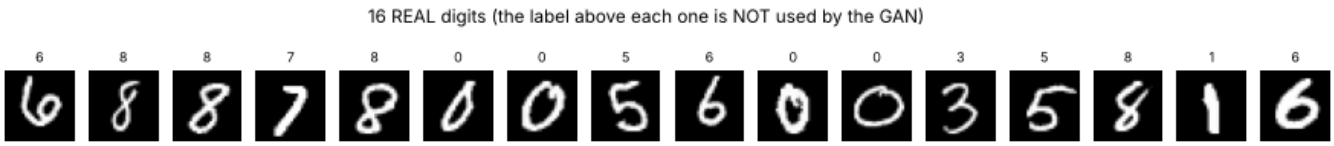
Let's check the slide's example: *pixel 204 → 0.8 → +0.6, pixel 0 → 0.0 → −1.0*, and look at one batch.

```
for raw in [0, 204, 255]:
    v = raw / 255                       # what ToTensor does
    print(f"stored pixel {raw:3d} -> ToTensor {v:.2f} -> Normalize {(v - 0.5) / 0.5:+.2f}")

real, labels = next(iter(train_loader))
print("\none batch:", tuple(real.shape), "| min", real.min().item(), "| max", real.max().item())
fig, axes = plt.subplots(1, 16, figsize=(16, 1.6))
for i in range(16):
    axes[i].imshow(real[i][0], cmap='gray'); axes[i].set_title(int(labels[i]), fontsize=9);
axes[i].axis('off')
plt.suptitle("16 REAL digits (the label above each one is NOT used by the GAN)", y=1.08)
plt.show()
```

```
stored pixel   0 -> ToTensor 0.00 -> Normalize -1.00
stored pixel 204 -> ToTensor 0.80 -> Normalize +0.60
stored pixel 255 -> ToTensor 1.00 -> Normalize +1.00

one batch: (64, 1, 28, 28) | min -1.0 | max 1.0
```



2 · The Generator: the forger

```
class Generator(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc = nn.Sequential(
            nn.Linear(100, 256),      # 100 noise numbers -> 256 hidden values
            nn.ReLU(),                # keep positives, set negatives to 0
            nn.Linear(256, 784),      # 256 -> 784 = one value per pixel (28 x 28)
            nn.Tanh()                 # squash every value into -1 ... +1, like the real images
        )

    def forward(self, x):
        return self.fc(x).view(-1, 1, 28, 28)   # 784 numbers -> an image (batch, 1 channel, 28,
28)
```

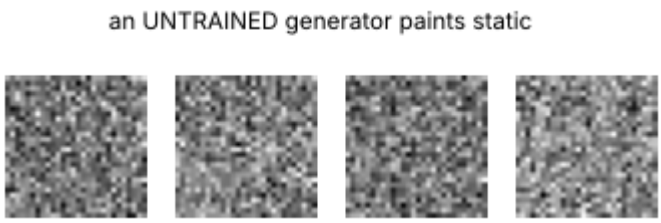
Line	What it does	Why
`nn.Linear(100, 256)`	mixes the 100 noise values into 256 hidden values	100 must match `torch.randn(..., 100)` in the loop
`nn.ReLU()`	keeps positives, sets negatives to 0	non-linearity, so G can draw curved shapes
`nn.Linear(256, 784)`	produces 784 values	one per pixel: 28 × 28 = 784

Line	What it does	Why
`nn.Tanh()`	squashes each value into −1 … +1	**same range as the real images** after `Normalize`. Otherwise D could catch fakes just by their range, which teaches G nothing
`.view(-1, 1, 28, 28)`	reshapes 784 numbers into an image	1 grey channel, 28 × 28; `-1` = "whatever the batch size is"

Where does variety come from? G is a fixed function: the same 100 numbers always give the same image. New images come from **new noise**. Before training, the weights are random, so the output is static. Let's see that.

```
g_untrained = Generator()
z = torch.randn(4, 100)                          # 4 noise vectors of 100 numbers each
out = g_untrained(z)
print("input z:", tuple(z.shape), "-> output:", tuple(out.shape), "| values between",
round(out.min().item(), 2), "and", round(out.max().item(), 2))
print("parameters in G:", sum(p.numel() for p in g_untrained.parameters()))
fig, axes = plt.subplots(1, 4, figsize=(6, 1.8))
for i in range(4):
    axes[i].imshow(out[i][0].detach(), cmap='gray'); axes[i].axis('off')
plt.suptitle("an UNTRAINED generator paints static", y=1.05); plt.show()
```

```
input z: (4, 100) -> output: (4, 1, 28, 28) | values between -0.66 and 0.72
parameters in G: 227344
```



3 · The Discriminator: the inspector

```
class Discriminator(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc = nn.Sequential(
            nn.Linear(784, 256),     # 784 pixels -> 256 hidden values (strokes, loops ...)
            nn.LeakyReLU(0.2),       # negatives are multiplied by 0.2 instead of zeroed
            nn.Linear(256, 1),       # 256 -> one score
            nn.Sigmoid()             # score -> probability 0 ... 1 = P(real)
        )

    def forward(self, x):
        return self.fc(x.view(-1, 784))   # flatten each 1x28x28 image into 784 numbers first
```

Line	What it does	Why
`x.view(-1, 784)`	flattens each image into 784 numbers	`Linear` layers need a flat list
`nn.Linear(784, 256)`	784 pixels → 256 hidden values	learns patterns such as strokes and loops

Line	What it does	Why
`nn.LeakyReLU(0.2)`	keeps positives; multiplies negatives by 0.2	a small gradient **always** flows back. G learns *only* through gradients passing back through D, so D must not block them (plain ReLU gives zero gradient for negatives)
`nn.Linear(256, 1)`	one score	one decision: real or fake
`nn.Sigmoid()`	squashes the score into 0 … 1	read it as a probability; needed for `nn.BCELoss`

The two networks are **mirror images**: G is 100 → 256 → 784, D is 784 → 256 → 1.

Where do D's labels come from? For free: we know which pile each image came from. Images from MNIST get target 1, images just made by G get target 0. The target describes the **source**, not the quality; a perfect-looking fake is still target 0 when D is trained.

```
d_untrained = Discriminator()
p = d_untrained(real[:4])                        # 4 real digits from the batch above
print("input:", tuple(real[:4].shape), "-> output:", tuple(p.shape))
print("untrained D's P(real) for 4 real digits:", p.detach().view(-1).numpy().round(3), " (about
0.5: it has no opinion yet)")
```

```
input: (4, 1, 28, 28) -> output: (4, 1)
untrained D's P(real) for 4 real digits: [0.445 0.444 0.442 0.382]  (about 0.5: it has no opinion
yet)
```

4 · One loss, two optimizers

Binary cross-entropy, with the slide's numbers computed in code

BCE(p, y) = −[y·log p + (1 − y)·log(1 − p)] (natural log)

Only one half is active at a time: target **y = 1 → loss = −log p**; target **y = 0 → loss = −log(1 − p)**. Small when confident and right, large when confident and wrong. Let's reproduce the table from the "THE LOSS: NN.BCELOSS" slide with `nn.BCELoss` itself.

```
criterion = nn.BCELoss()
cases = [("D, real digit, confident", 1, 0.9),
         ("D, real digit, wrong",     1, 0.1),
         ("D, fake digit, confident", 0, 0.2),
         ("G, same fake (wants y=1)", 1, 0.2)]
print(f"{'case':28s} {'y':>2s} {'p':>5s} {'BCE':>7s}")
for name, y, p in cases:
    loss = criterion(torch.tensor([p]), torch.tensor([float(y)]))
    print(f"{name:28s} {y:2d} {p:5.1f} {loss.item():7.3f}")

print("\nD's loss for one real (0.9) + one fake (0.2):", round(-math.log(0.9) - math.log(1 -
0.2), 3), "(low: D is doing well)")
print("G's loss on that same fake:  -ln 0.2 =", round(-math.log(0.2), 3), "(high: G must
improve)")
print("\nReference numbers when D says 0.5 for everything:")
print("  loss_d = -ln 0.5 - ln 0.5 =", round(2 * math.log(2), 3), "   loss_g = -ln 0.5 =",
round(math.log(2), 3))
```

```
case                          y     p      BCE
D, real digit, confident      1   0.9   0.105
D, real digit, wrong          1   0.1   2.303
D, fake digit, confident      0   0.2   0.223
G, same fake (wants y=1)      1   0.2   1.609

D's loss for one real (0.9) + one fake (0.2): 0.329 (low: D is doing well)
G's loss on that same fake:  -ln 0.2 = 1.609 (high: G must improve)

Reference numbers when D says 0.5 for everything:
  loss_d = -ln 0.5 - ln 0.5 = 1.386    loss_g = -ln 0.5 = 0.693
```

The last two rows of the table are **"adversarial" in numbers**: the same fake and the same score 0.2 are **good news for D (0.223)** and **bad news for G (1.609)**.

The slides also walk through **one round of the game** with illustrative scores. Here are those losses computed:

```python
def bce_pair(dx, dgz):
    loss_d = -math.log(dx) - math.log(1 - dgz)     # real -> 1, fake -> 0
    loss_g = -math.log(dgz)                        # fake -> 1 (G's wish)
    return loss_d, loss_g

print(f"{'moment':16s} {'D(x)':>5s} {'D(G(z))':>8s} {'D loss':>7s} {'G loss':>7s}")
for moment, dx, dgz in [("start of round", 0.60, 0.40), ("after D step", 0.70, 0.30), ("after G step", 0.70, 0.45)]:
    ld, lg = bce_pair(dx, dgz)
    print(f"{moment:16s} {dx:5.2f} {dgz:8.2f} {ld:7.3f} {lg:7.3f}")
```

```
moment           D(x)  D(G(z))  D loss  G loss
start of round   0.60     0.40   1.022   0.916
after D step     0.70     0.30   0.713   1.204
after G step     0.70     0.45   0.955   0.799
```

The D step lowers D's loss (1.022 → 0.713) and, as a side effect, raises G's (0.916 → 1.204). The G step lowers G's loss (→ 0.799) and raises D's again. **Neither wins for good**, which is why GAN losses wobble instead of falling steadily.

The course code: create the players, the loss and two optimizers

```python
torch.manual_seed(0)                       # (added) so this run is repeatable
generator = Generator()
discriminator = Discriminator()
criterion = nn.BCELoss()
optimizer_g = optim.Adam(generator.parameters(), lr=0.0002)
optimizer_d = optim.Adam(discriminator.parameters(), lr=0.0002)
```

Two optimizers are the trick that keeps the two steps separate: `optimizer_g` holds **only G's weights**, `optimizer_d` holds **only D's**. So `optimizer_d.step()` can change only D and `optimizer_g.step()` only G. Both use Adam with learning rate 0.0002 (a common GAN setting; Adam's other settings are left at the PyTorch default `betas=(0.9, 0.999)`, which matters in Task 3 D).

5 · The training loop: every batch = one round of the game

Step 1 trains the inspector, step 2 trains the forger. This is the course loop line for line. The only additions (marked) keep a record of the losses and of D's average opinions so we can compare

runs later.

```
epochs = 2
history = []                                                    # (added) one row per epoch
t0 = time.time()                                                # (added)
for epoch in range(epochs):
    sum_d = sum_g = sum_dx = sum_dgz = 0.0; n = 0               # (added) running totals for epoch
averages
    for real, _ in train_loader:
        real = real                                             # (in the course notebook; does
nothing)
        batch_size = real.size(0)
        labels_real = torch.ones(batch_size, 1)
        labels_fake = torch.zeros(batch_size, 1)

        # Step 1: Train Discriminator
        optimizer_d.zero_grad()
        outputs_real = discriminator(real)
        loss_real = criterion(outputs_real, labels_real)
        noise = torch.randn(batch_size, 100)
        fake = generator(noise)
        outputs_fake = discriminator(fake.detach())
        loss_fake = criterion(outputs_fake, labels_fake)
        loss_d = loss_real + loss_fake
        loss_d.backward()
        optimizer_d.step()

        # Step 2: Train Generator
        optimizer_g.zero_grad()
        outputs_fake = discriminator(fake)
        loss_g = criterion(outputs_fake, labels_real)
        loss_g.backward()
        optimizer_g.step()

        sum_d += loss_d.item(); sum_g += loss_g.item(); n += 1                      # (added)
        sum_dx += outputs_real.mean().item(); sum_dgz += outputs_fake.mean().item()  # (added)
    print(f"Epoch {epoch+1}/{epochs}, Loss D: {loss_d.item()}, Loss G: {loss_g.item()}")
    history.append((loss_d.item(), loss_g.item(), sum_d / n, sum_g / n, sum_dx / n, sum_dgz / n))
# (added)
print(f"(added) training time: {time.time() - t0:.0f} s for {epochs} epochs x {len(train_loader)}
batches")
```

```
Epoch 1/2, Loss D: 0.5900115966796875, Loss G: 1.403838038444519
Epoch 2/2, Loss D: 1.041710376739502, Loss G: 1.057669997215271
(added) training time: 112 s for 2 epochs x 938 batches
```

What each part of the loop does

Setting up each batch

- `for real, _ in train_loader` takes 64 real digits; `_` throws the digit labels away.
- `batch_size = real.size(0)` reads the *actual* batch size: 64, except the last batch of each epoch, which has 32. (That is why the code does not just write 64.)
- `labels_real` is a column of 1s, `labels_fake` a column of 0s, one per image.

Step 1: train the Discriminator (G stays fixed)

- `optimizer_d.zero_grad()` clears old gradients.
- *Real exam:* D judges the real digits; `loss_real` = BCE against target 1, so D is punished for calling real digits fake.
- *Fake exam:* `torch.randn(batch_size, 100)` draws fresh noise, G turns it into fakes, D judges them; `loss_fake` = BCE against target 0.

- `fake.detach()` cuts the wire from these images back to G, so D's loss sends no gradient into G.
- `loss_d = loss_real + loss_fake` is D's total mistake; `backward()` + `optimizer_d.step()` update **only D**.

Step 2: train the Generator (D is not updated)

- The **same** fakes go through the *just-updated* D, this time **without** `.detach()`, so gradients flow back **through D into G**.
- `loss_g = criterion(outputs_fake, labels_real)`: target **1** for fake images. Not a lie about the data: it states G's goal, "my loss is low only when D believes my images are real".
- `optimizer_g.step()` changes **only G**. D's leftover gradients are cleared at the next `optimizer_d.zero_grad()`.

The print line sits outside the batch loop, so it shows the losses of the **last batch** of each epoch: noisy snapshots, not averages. One epoch = 938 batches = 938 rounds of the game; 2 epochs = 1,876 rounds.

6 · Making 16 new digits (course code)

```
noise = torch.randn(16, 100)
fake_images = generator(noise).detach()
fig, axes = plt.subplots(1, 16, figsize=(16,2))
for i in range(16):
    axes[i].imshow(fake_images[i][0], cmap='gray')
    axes[i].axis('off')
plt.show()
```



This uses **only the Generator**: 16 new noise vectors in, 16 new digits out. `.detach()` says "no gradients needed, we are only looking". `fake_images[i][0]` picks image *i* and its only channel.

In-class tasks (slides "TASK 1 OF 4" ... "TASK 4 OF 4")

Task 1 · Run the GAN and observe

> 1. Predict what the 16 digits will look like. 2. Write down Loss D and Loss G for each epoch and compare with **1.386 / 0.693**. 3. How many look like digits? How many different digits? 4. Re-run just the plotting lines: why do you get different digits?

```
print(f"{'epoch':>5s} | {'last-batch D':>12s} {'last-batch G':>12s} | {'epoch-avg D':>11s}
{'epoch-avg G':>11s} | {'mean D(x)':>9s} {'mean D(G(z))':>12s}")
for e, (ld, lg, ad, ag, dx, dgz) in enumerate(history, 1):
```

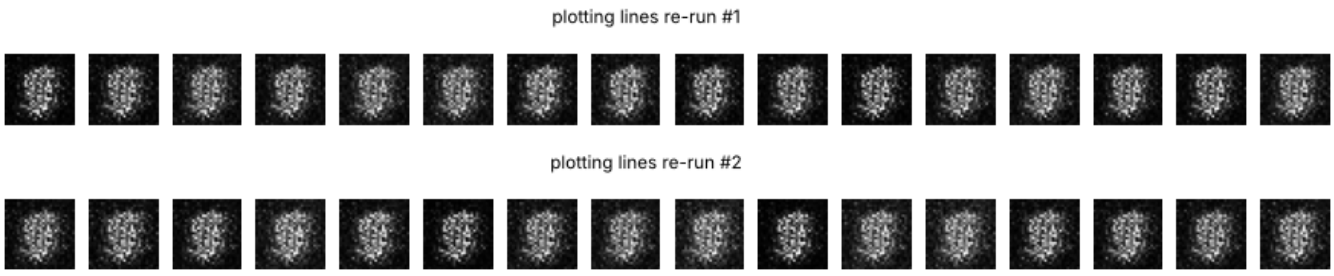
```
    print(f"{e:5d} | {ld:12.3f} {lg:12.3f} | {ad:11.3f} {ag:11.3f} | {dx:9.3f} {dgz:12.3f}")
print(f"{'ref':>5s} | {1.386:12.3f} {0.693:12.3f} |  <- the values if D said 0.5 for everything")
```

```
epoch | last-batch D last-batch G | epoch-avg D epoch-avg G | mean D(x) mean D(G(z))
    1 |        0.590        1.404 |       0.700       1.369 |     0.811        0.298
    2 |        1.042        1.058 |       0.829       1.261 |     0.714        0.302
  ref |        1.386        0.693 |  <- the values if D said 0.5 for everything
```

Answers for this run (seed 0, CPU).

1. *Prediction:* most people expect blurry digits. What we actually got is below.
2. *Losses vs the reference.* Epoch 1 ended at **Loss D 0.590, Loss G 1.404**, epoch 2 at **Loss D 1.042, Loss G 1.058**. Both times D's loss is **below 1.386** and G's **above 0.693**: D is still ahead. The epoch averages tell the same story more reliably (D 0.70 → 0.83, G 1.37 → 1.26), and D's average opinion of the fakes, mean D(G(z)) ≈ 0.30, is well under 0.5. Remember the last-batch numbers are single noisy snapshots: epoch 2's 1.042 looks "close to 1.386", but the average over the epoch is 0.83.
3. *How many look like digits?* **None.** All 16 samples are the **same** speckled blob on a dark background, roughly digit-sized and vaguely like an "8" or "0". There is **no variety**: the generator has collapsed to one output (**mode collapse**, Class 2's topic), and the 4.3 count below confirms it on 1,000 samples. The course guide's own Class 2 & 3 test run reports exactly this for these settings ("Run A collapsed completely … the digit counter labelled all 1,000 samples as 8"), so this is a genuine weakness of the default settings, not a bug. The course guide for Class 1 expected "blotchy, half-formed digits after 2 epochs"; GANs are sensitive to the random seed and machine, and on this run the result is worse than that. Task 3 D shows the one-line change that fixes it.
4. See the next cell.

```
# 4. Run just the plotting lines again: new noise, same generator, no retraining
for run in range(2):
    noise = torch.randn(16, 100)
    fake_images = generator(noise).detach()
    fig, axes = plt.subplots(1, 16, figsize=(16, 1.4))
    for i in range(16):
        axes[i].imshow(fake_images[i][0], cmap='gray'); axes[i].axis('off')
    plt.suptitle(f"plotting lines re-run #{run + 1}", y=1.05); plt.show()
```



Why different digits without retraining? `torch.randn(16, 100)` draws **new noise** every time. G's weights did not change, but its input did, and G is a function: new input, new output. That is the Generator doing its job.

Task 2 · Read the code like a GAN detective

Question	Answer
a. Which line gives D its "fake" targets? What value?	`labels_fake = torch.zeros(batch_size, 1)` : all **0**
b. Why `.detach()` in the D step but not in the G step?	D step: we train only D, so we **stop** gradients flowing into G. G step: the gradient **must** travel through D back into G, so the path stays connected
c. Why does the G loss use `labels_real` ?	G wants D(G(z)) → 1. Minimising BCE with target 1 = minimising −log D(G(z)): the **non-saturating** generator loss (next section shows why it is used)
d. Shape of `fake` in a full batch?	**(64, 1, 28, 28)**: 64 images, 1 channel, 28 × 28

Two of these can be checked in code:

```
noise = torch.randn(64, 100)
fake = generator(noise)
print("d. shape of fake in a full batch:", tuple(fake.shape))

# b. what .detach() does: does a loss on D's output reach G's weights?
generator.zero_grad()
criterion(discriminator(fake.detach()), torch.zeros(64, 1)).backward()
print("b. with    .detach(): G's first-layer gradient is", generator.fc[0].weight.grad)
criterion(discriminator(fake), torch.ones(64, 1)).backward()
print("b. without .detach(): G's first-layer gradient size =",
round(generator.fc[0].weight.grad.abs().sum().item(), 3))
generator.zero_grad(); discriminator.zero_grad()      # tidy up: we are not training here
```

```
d. shape of fake in a full batch: (64, 1, 28, 28)
b. with    .detach(): G's first-layer gradient is None
b. without .detach(): G's first-layer gradient size = 97.59
```

With `.detach()` no gradient reaches G at all (`None`); without it, G receives a gradient. That is exactly how one loop trains two networks separately.

Why `labels_real` for G? Saturating vs non-saturating loss

The minimax objective (Goodfellow et al., 2014) is

min over G, max over D of E[log D(x)] + E[log(1 − D(G(z)))]

so, read literally, G should **minimise log(1 − D(G(z)))** (the *saturating* loss). The code instead minimises **−log D(G(z))** (BCE with target 1, the *non-saturating* loss). Both push D(G(z)) towards 1. The difference is the **size of the push** early in training, when fakes are bad and D(G(z)) ≈ 0.

D's last layer is a Sigmoid: p = sigmoid(a), where *a* is D's raw score. The gradient that reaches G passes through *a*. A little calculus gives:

- saturating loss log(1 − p): gradient w.r.t. *a* is **−p**, which is ≈ 0 when p ≈ 0 (flat, "saturated");
- non-saturating loss −log p: gradient w.r.t. *a* is **−(1 − p)**, which is ≈ −1 when p ≈ 0 (steep).

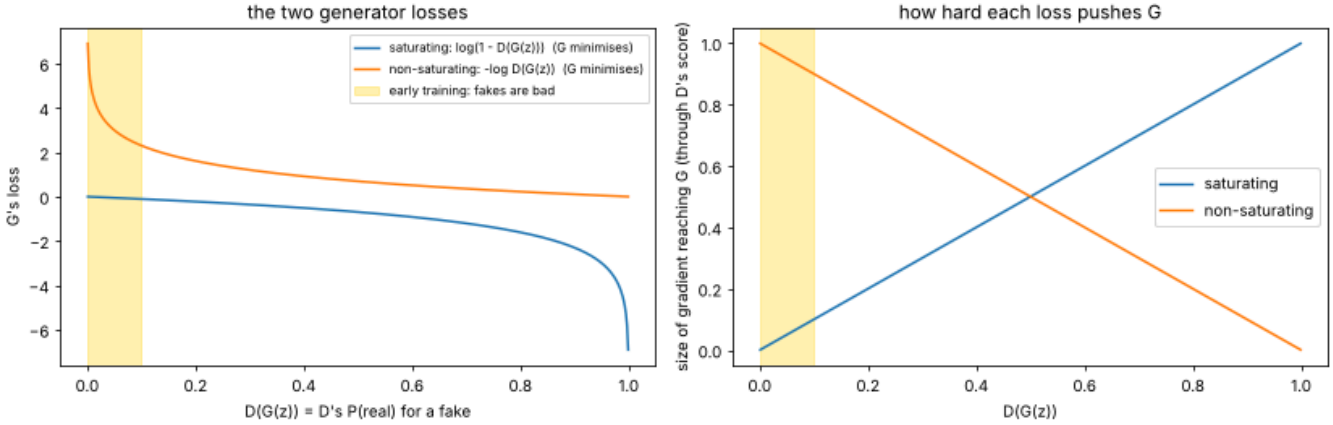
We check this with autograd rather than trusting the algebra:

```
p = torch.linspace(0.001, 0.999, 500)
fig, ax = plt.subplots(1, 2, figsize=(12, 4))
ax[0].plot(p, torch.log(1 - p), label="saturating: log(1 - D(G(z)))  (G minimises)")
```

```
ax[0].plot(p, -torch.log(p), label="non-saturating: -log D(G(z))  (G minimises)")
ax[0].axvspan(0, 0.1, color="gold", alpha=0.3, label="early training: fakes are bad")
ax[0].set_xlabel("D(G(z)) = D's P(real) for a fake"); ax[0].set_ylabel("G's loss");
ax[0].legend(fontsize=8); ax[0].set_title("the two generator losses")

a = torch.logit(p).clone().requires_grad_(True)            # D's raw score before the Sigmoid
torch.log(1 - torch.sigmoid(a)).sum().backward(); g_sat = a.grad.clone(); a.grad = None
(-torch.log(torch.sigmoid(a))).sum().backward(); g_ns = a.grad.clone()
ax[1].plot(p, g_sat.abs(), label="saturating")
ax[1].plot(p, g_ns.abs(), label="non-saturating")
ax[1].axvspan(0, 0.1, color="gold", alpha=0.3)
ax[1].set_xlabel("D(G(z))"); ax[1].set_ylabel("size of gradient reaching G (through D's score)");
ax[1].legend(); ax[1].set_title("how hard each loss pushes G")
plt.tight_layout(); plt.show()

for pv in [0.001, 0.01, 0.1, 0.5, 0.9]:
    i = int(torch.argmin((p - pv).abs()))
    print(f"D(G(z)) = {p[i]:.3f}:  saturating push {g_sat[i].abs():.3f}   non-saturating push
{g_ns[i].abs():.3f}")
```




```
D(G(z)) = 0.001:  saturating push 0.001   non-saturating push 0.999
D(G(z)) = 0.009:  saturating push 0.009   non-saturating push 0.991
D(G(z)) = 0.099:  saturating push 0.099   non-saturating push 0.901
D(G(z)) = 0.499:  saturating push 0.499   non-saturating push 0.501
D(G(z)) = 0.899:  saturating push 0.899   non-saturating push 0.101
```

When D(G(z)) is 0.001 (a terrible fake, the usual start), the saturating loss pushes G with strength about 0.001, the non-saturating loss with about 1: **roughly a thousand times stronger, exactly when G needs it**. Same goal, better gradients. The 2014 GAN paper itself suggests this trick, and `criterion(outputs_fake, labels_real)` is how the course code implements it.

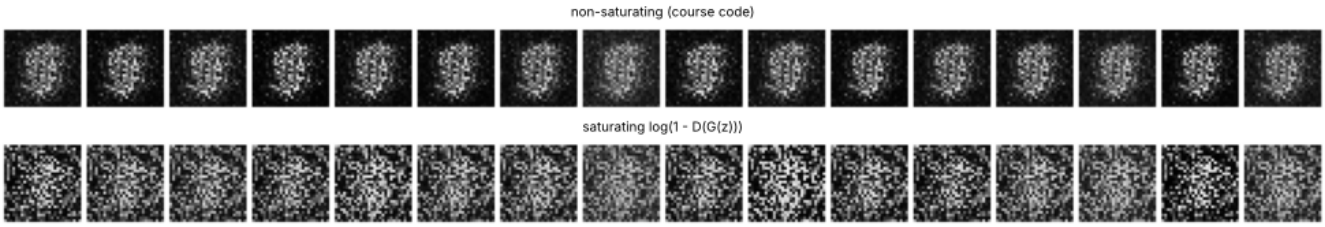
Does it matter in practice? Let's train the same GAN for the same 2 epochs with the **saturating** loss (the only change: G minimises log(1 − D(G(z))) instead of BCE with `labels_real`), from the same seed.

```
def train_saturating(epochs=2, seed=0):
    torch.manual_seed(seed)
    G, D = Generator(), Discriminator()
    opt_g = optim.Adam(G.parameters(), lr=0.0002); opt_d = optim.Adam(D.parameters(), lr=0.0002)
    for epoch in range(epochs):
        dgz_sum, n = 0.0, 0
        for real, _ in train_loader:
            b = real.size(0)
            opt_d.zero_grad()
            fake = G(torch.randn(b, 100))
            loss_d = criterion(D(real), torch.ones(b, 1)) + criterion(D(fake.detach()),
torch.zeros(b, 1))
            loss_d.backward(); opt_d.step()
            opt_g.zero_grad()
            out = D(fake)
            loss_g = torch.log(1 - out + 1e-8).mean()     # <- the ONLY change: the saturating
```

```
minimax loss
            loss_g.backward(); opt_g.step()
            dgz_sum += out.mean().item(); n += 1
        print(f"saturating loss, epoch {epoch+1}: mean D(G(z)) = {dgz_sum / n:.3f}")
    return G

t0 = time.time()
G_sat = train_saturating()
print(f"({time.time() - t0:.0f} s)")
print("course (non-saturating) run, mean D(G(z)) per epoch:", [round(h[5], 3) for h in history])
torch.manual_seed(123)
z16 = torch.randn(16, 100)
fig, ax = plt.subplots(2, 1, figsize=(16, 2.6))
for a, G, name in [(ax[0], generator, "non-saturating (course code)"), (ax[1], G_sat, "saturating
log(1 - D(G(z)))")]:
    a.imshow(make_grid(G(z16).detach(), nrow=16, normalize=True, value_range=(-1, 1),
pad_value=1).permute(1, 2, 0)); a.axis("off"); a.set_title(name, fontsize=10)
plt.tight_layout(); plt.show()
```

```
saturating loss, epoch 1: mean D(G(z)) = 0.085
saturating loss, epoch 2: mean D(G(z)) = 0.018
(119 s)
course (non-saturating) run, mean D(G(z)) per epoch: [0.298, 0.302]
```



What happened (this run). With the saturating loss, D's average belief that fakes are real fell to **0.085 after epoch 1 and 0.018 after epoch 2**: D wins more and more easily and G's gradient (≈ D(G(z)), from the plot above) shrinks towards zero, so G barely moves. Its samples are still near-random static. The course loss kept mean D(G(z)) around **0.30**, so G kept receiving a useful signal. (The course-loss samples are the collapsed blob from Task 1, so neither run is *good* yet; the point here is the size of G's learning signal, which the D(G(z)) numbers show directly.)

Task 3 · One experiment: change one thing, predict, run

The slide asks you to pick **one** of A–D. Here we run **all four** and compare them. To make that easy, the course loop is wrapped in a function `train_gan(...)` whose arguments are the four things the experiments change. With the default arguments it is **exactly** the loop above.

```
class GeneratorZ(nn.Module):
    """The course Generator with the noise size as a setting (Experiment B changes 100 -> 10)."""
    def __init__(self, latent_dim=100):
        super().__init__()
        self.fc = nn.Sequential(nn.Linear(latent_dim, 256), nn.ReLU(), nn.Linear(256, 784),
nn.Tanh())
    def forward(self, x):
        return self.fc(x).view(-1, 1, 28, 28)


def show_row(images, title, ax=None):
    """Draw a batch of images (values -1..1) side by side in one row."""
    grid = make_grid(images, nrow=len(images), normalize=True, value_range=(-1, 1), pad_value=1)
    ax = ax or plt.figure(figsize=(16, 1.4)).gca()
    ax.imshow(grid.permute(1, 2, 0)); ax.axis('off'); ax.set_title(title, fontsize=10,
loc='left')
```

```
def train_gan(epochs=2, latent_dim=100, lr_d=0.0002, lr_g=0.0002, betas=(0.9, 0.999), seed=0,
fixed_noise=None):
    torch.manual_seed(seed)
    generator = GeneratorZ(latent_dim)
    discriminator = Discriminator()
    criterion = nn.BCELoss()
    optimizer_g = optim.Adam(generator.parameters(), lr=lr_g, betas=betas)
    optimizer_d = optim.Adam(discriminator.parameters(), lr=lr_d, betas=betas)
    log = {"loss_d": [], "loss_g": [], "dx": [], "dgz": [], "epoch_rows": [], "snapshots": [],
"copies": []}
    t0 = time.time()
    for epoch in range(epochs):
        for real, _ in train_loader:
            batch_size = real.size(0)
            labels_real = torch.ones(batch_size, 1)
            labels_fake = torch.zeros(batch_size, 1)
            # Step 1: Train Discriminator
            optimizer_d.zero_grad()
            outputs_real = discriminator(real)
            loss_real = criterion(outputs_real, labels_real)
            noise = torch.randn(batch_size, latent_dim)
            fake = generator(noise)
            outputs_fake = discriminator(fake.detach())
            loss_fake = criterion(outputs_fake, labels_fake)
            loss_d = loss_real + loss_fake
            loss_d.backward()
            optimizer_d.step()
            # Step 2: Train Generator
            optimizer_g.zero_grad()
            outputs_fake = discriminator(fake)
            loss_g = criterion(outputs_fake, labels_real)
            loss_g.backward()
            optimizer_g.step()
            # record keeping
            log["loss_d"].append(loss_d.item()); log["loss_g"].append(loss_g.item())
            log["dx"].append(outputs_real.mean().item());
log["dgz"].append(outputs_fake.mean().item())
        k = len(train_loader)
        row = (loss_d.item(), loss_g.item(), np.mean(log["loss_d"][-k:]), np.mean(log["loss_g"][-
k:]),
               np.mean(log["dx"][-k:]), np.mean(log["dgz"][-k:]))
        log["epoch_rows"].append(row)
        print(f"Epoch {epoch+1}/{epochs}, Loss D: {row[0]:.3f}, Loss G: {row[1]:.3f}   "
              f"(epoch avg D {row[2]:.3f}, G {row[3]:.3f}; mean D(x) {row[4]:.3f}, D(G(z))
{row[5]:.3f})")
        if fixed_noise is not None:
            log["snapshots"].append(generator(fixed_noise).detach())
        log["copies"].append(copy.deepcopy(generator))          # a frozen copy of G after every
epoch
    log["seconds"] = time.time() - t0
    return generator, log

runs = {}   # every trained generator goes in here, so Task 4 can compare them all
```

Experiment A · Watch G learn (fixed noise, 5 epochs)

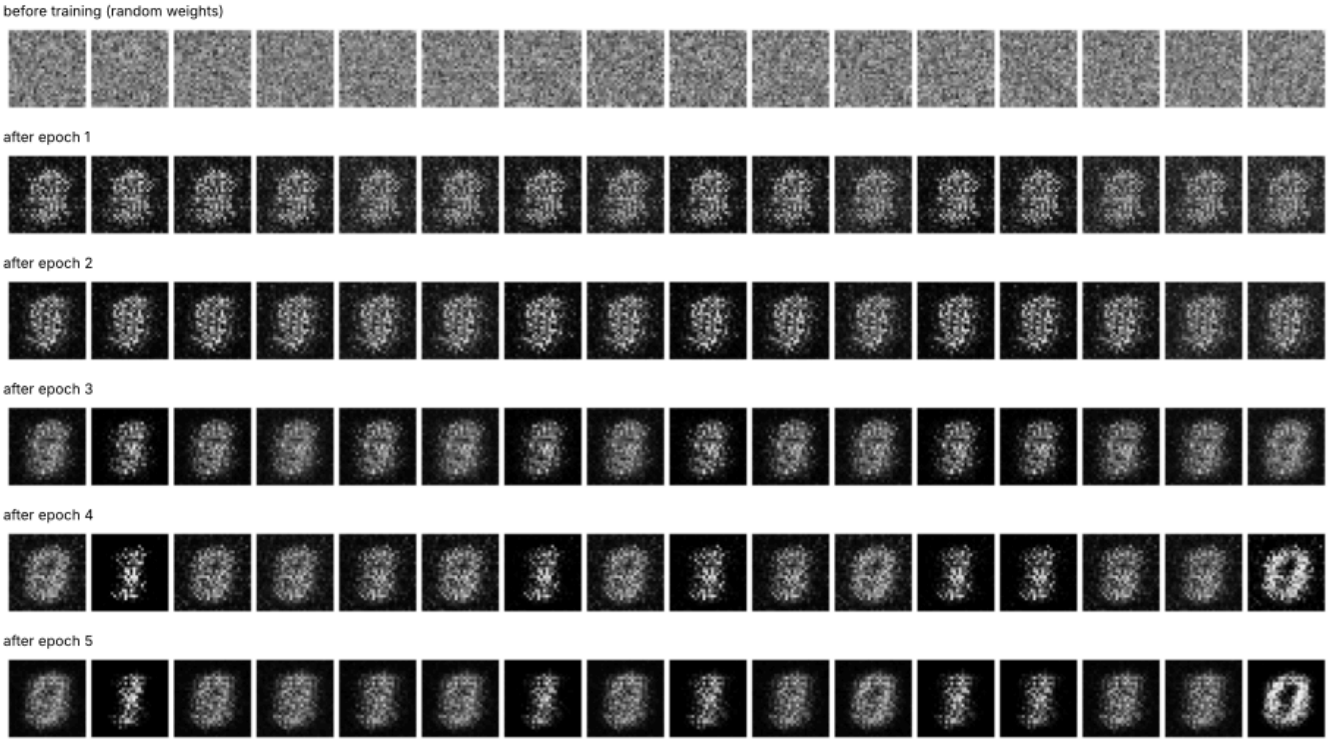
Before the loop, draw `fixed_noise = torch.randn(16, 100)` **once**. After each epoch, plot `generator(fixed_noise)`. Because the input never changes, **any change you see comes only from G's weights changing**. This is the "fixed z" view the slides call *the best way to watch a GAN learn*.

> ✏️ **Predict:** after epoch 1, will you see digits? Will the 16 images change *which* digit they show from epoch to epoch, or only get sharper?

```
torch.manual_seed(42)
fixed_noise = torch.randn(16, 100)
G_A, log_A = train_gan(epochs=5, fixed_noise=fixed_noise)
runs["A: 5 epochs"] = (G_A, log_A, 100)
# Same seed and settings as the course cell, so A's first 2 epochs ARE the course run (compare
the numbers
# with Task 1). Its copy after epoch 2 is our baseline for Experiments B-D: no need to train it
again.
k2 = 2 * len(train_loader)
log_base = {key: log_A[key][:k2] for key in ["loss_d", "loss_g", "dx", "dgz"]}
log_base["epoch_rows"] = log_A["epoch_rows"][:2]
runs["baseline (course)"] = (log_A["copies"][1], log_base, 100)
print(f"time: {log_A['seconds']:.0f} s")

fig, axes = plt.subplots(6, 1, figsize=(16, 7.5))
torch.manual_seed(0)
show_row(GeneratorZ()(fixed_noise).detach(), "before training (random weights)", axes[0])
for e, snap in enumerate(log_A["snapshots"]):
    show_row(snap, f"after epoch {e + 1}", axes[e + 1])
plt.tight_layout(); plt.show()
```

```
Epoch 1/5, Loss D: 0.590, Loss G: 1.404   (epoch avg D 0.700, G 1.369; mean D(x) 0.811, D(G(z))
0.298)
Epoch 2/5, Loss D: 1.042, Loss G: 1.058   (epoch avg D 0.829, G 1.261; mean D(x) 0.714, D(G(z))
0.302)
Epoch 3/5, Loss D: 1.200, Loss G: 0.963   (epoch avg D 0.721, G 1.475; mean D(x) 0.739, D(G(z))
0.255)
Epoch 4/5, Loss D: 0.930, Loss G: 1.171   (epoch avg D 0.899, G 1.306; mean D(x) 0.687, D(G(z))
0.295)
Epoch 5/5, Loss D: 1.367, Loss G: 0.812   (epoch avg D 0.948, G 1.178; mean D(x) 0.658, D(G(z))
0.328)
time: 154 s
```



Answer revealed (this run). Row 1 is pure static. After epoch 1 every image is the **same** blob: G has already collapsed. Epochs 2–3 change the blob's texture but not its variety. By **epoch 4–5** a few columns break away (a thin vertical stroke in columns 2, 7, 9 and 12–13, and a clear "0"-shaped ring in the last column): G is slowly starting to separate a few modes, i.e. variety is returning, but most of the 16 are still the same blob after 5 epochs. Because the input is fixed, all of this change comes from G's weights. Note that epochs 1–2 of this run print exactly the same losses as the Task 1 run: same seed, same code, same result.

Experiment B · Smaller latent space (100 → 10)

Change 100 → 10 in all three places (`nn.Linear(100, 256)` and both `torch.randn(..., 100)`). Here that is `latent_dim=10` .

> ✏️ **Predict:** with only 10 random numbers per image, does variety drop? Does quality change?

Experiment C · Unbalanced players (D learns 10× faster)

D gets `lr = 0.002` , G keeps 0.0002.

> ✏️ **Predict:** who wins? What happens to Loss G and to the samples?

Experiment D · A common GAN setting: `betas=(0.5, 0.999)`

The DCGAN paper recommends Adam with lr 0.0002 and **β1 = 0.5** (less momentum than the default 0.9).

> ✏️ **Predict:** better or worse samples after 2 epochs?

All three run for the course's 2 epochs from the same seed as the baseline, so the **only** difference is the one setting. The baseline itself is the copy of Experiment A's generator after epoch 2 (identical to the course run above).

```
G_B, log_B = train_gan(latent_dim=10)
runs["B: latent 10"] = (G_B, log_B, 10)
G_C, log_C = train_gan(lr_d=0.002)
runs["C: D lr x10"] = (G_C, log_C, 100)
G_D, log_D = train_gan(betas=(0.5, 0.999))
runs["D: betas 0.5"] = (G_D, log_D, 100)
print("seconds per 2-epoch run:", [round(l['seconds']) for l in (log_B, log_C, log_D)])
```

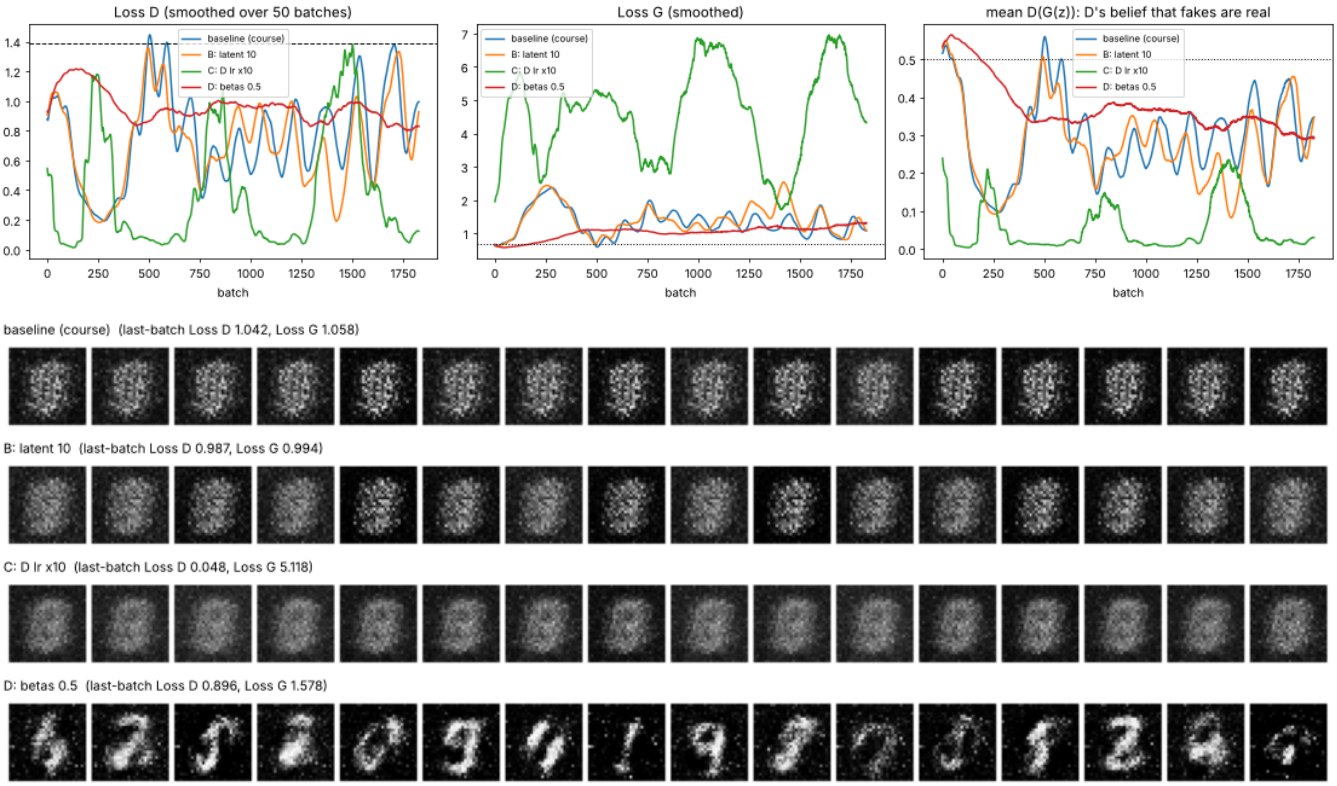
```
Epoch 1/2, Loss D: 0.933, Loss G: 1.044   (epoch avg D 0.694, G 1.374; mean D(x) 0.805, D(G(z))
0.291)
Epoch 2/2, Loss D: 0.987, Loss G: 0.994   (epoch avg D 0.751, G 1.420; mean D(x) 0.738, D(G(z))
0.269)
Epoch 1/2, Loss D: 0.180, Loss G: 5.775   (epoch avg D 0.358, G 4.136; mean D(x) 0.926, D(G(z))
0.062)
Epoch 2/2, Loss D: 0.048, Loss G: 5.118   (epoch avg D 0.386, G 4.954; mean D(x) 0.910, D(G(z))
0.056)
Epoch 1/2, Loss D: 1.052, Loss G: 0.953   (epoch avg D 0.996, G 0.916; mean D(x) 0.720, D(G(z))
0.412)
Epoch 2/2, Loss D: 0.896, Loss G: 1.578   (epoch avg D 0.913, G 1.138; mean D(x) 0.681, D(G(z))
0.337)
seconds per 2-epoch run: [29, 29, 30]
```

```
def smooth(x, k=50):
    return np.convolve(x, np.ones(k) / k, mode="valid")

names = ["baseline (course)", "B: latent 10", "C: D lr x10", "D: betas 0.5"]
fig, ax = plt.subplots(1, 3, figsize=(16, 3.8))
for name in names:
    log = runs[name][1]
    ax[0].plot(smooth(log["loss_d"]), label=name); ax[1].plot(smooth(log["loss_g"]), label=name);
ax[2].plot(smooth(log["dgz"]), label=name)
ax[0].axhline(1.386, ls="--", c="k", lw=1); ax[1].axhline(0.693, ls=":", c="k", lw=1);
ax[2].axhline(0.5, ls=":", c="k", lw=1)
```

```
ax[0].set_title("Loss D (smoothed over 50 batches)"); ax[1].set_title("Loss G (smoothed)");
ax[2].set_title("mean D(G(z)): D's belief that fakes are real")
for a in ax: a.set_xlabel("batch"); a.legend(fontsize=8)
plt.tight_layout(); plt.show()

torch.manual_seed(7)
z100 = torch.randn(16, 100); z10 = z100[:, :10]
fig, axes = plt.subplots(4, 1, figsize=(16, 5.2))
for a, name in zip(axes, names):
    G, log, zd = runs[name]
    show_row(G(z100 if zd == 100 else z10).detach(), f"{name}  (last-batch Loss D
{log['epoch_rows'][-1][0]:.3f}, Loss G {log['epoch_rows'][-1][1]:.3f})", a)
plt.tight_layout(); plt.show()
```



Answers revealed (this run, all 2 epochs, seed 0):

Run	last-batch Loss D / Loss G	epoch-2 average D / G	mean D(x) / D(G(z)) in epoch 2	samples
baseline (course)	1.042 / 1.058	0.829 / 1.261	0.714 / 0.302	one collapsed blob
B · latent 10	0.987 / 0.994	0.751 / 1.420	0.738 / 0.269	the same kind of blob: **no worse, no better**
C · D lr × 10	**0.048 / 5.118**	0.386 / 4.954	**0.910 / 0.056**	a faint grey smudge, identical in every column
D · betas (0.5, 0.999)	0.896 / 1.578	0.913 / 1.138	0.681 / 0.337	**real, varied digits** (6, 2, 5, 0, 9, 1 …)

- **B (smaller latent space):** prediction "variety drops" cannot be tested here because the baseline already has no variety; both collapsed. With only 10 noise numbers G *could* still make all ten digits (10 numbers are plenty to choose a digit and a style), so latent size is not the bottleneck in this run.
- **C (D 10× faster): D wins outright.** Loss D falls towards 0, Loss G climbs to about 5, mean D(G(z)) sits near 0.05, and the samples stay grey mush. This is the slides' "D too strong"

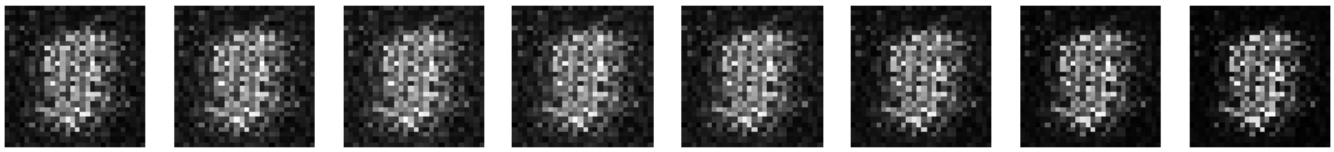
failure: D rejects every fake with total confidence and G's learning signal becomes weak. Notice in the loss plot that it is not a steady state: G occasionally catches up (the dips), then D pulls away again.

- **D (β1 = 0.5):** the **only** change that fixes this run. Smooth, calm curves (red), D(G(z)) settles around 0.3–0.35 instead of swinging, and the samples are recognisable, different digits. Lower β1 means less momentum: in a game where the opponent keeps moving, a player with a lot of momentum keeps chasing where the opponent *used to be* and overshoots (the swinging blue and orange curves). This is why the DCGAN paper recommends β1 = 0.5, and why Classes 2–4 use it.
- **Could you have picked the winner from the losses alone?** Not really: the baseline's last-batch numbers (1.042 / 1.058) look *closer* to the 1.386 / 0.693 "balance" than D's (0.896 / 1.578), yet the baseline is the broken one. The steadiness of the curves is a hint; **the pictures are the proof.**

Task 4 · Explore the latent space

4.1 Walk from one digit to another (course code, on the 2-epoch course generator)

```
# new cell, after training  (course code)
torch.manual_seed(1)                                  # (added) repeatable pair of endpoints
z1 = torch.randn(1, 100)
z2 = torch.randn(1, 100)
steps = 8
fig, axes = plt.subplots(1, steps, figsize=(16, 2))
for i in range(steps):
    t = i / (steps - 1)
    z = (1 - t) * z1 + t * z2
    img = generator(z).detach()
    axes[i].imshow(img[0][0], cmap='gray')
    axes[i].axis('off')
plt.show()
```



`t` goes 0 → 1 in 8 steps; `z = (1 − t)·z1 + t·z2` is a straight line in latent space from z1 to z2. If G has learnt a **smooth** map, nearby z give similar images, so the picture should **morph** rather than jump.

To answer the questions properly we need to know *which digit* each image shows. A person can count, but a quick classifier is faster and fairer. We use the same tiny digit classifier as the Class 2–3 notebook: `Flatten → Linear(784, 256) → ReLU → Linear(256, 10)`, trained for one epoch on the real labelled digits. It is a **measuring tool**, not part of the GAN.

```
torch.manual_seed(0)
classifier = nn.Sequential(nn.Flatten(), nn.Linear(784, 256), nn.ReLU(), nn.Linear(256, 10))
opt_c = optim.Adam(classifier.parameters(), lr=1e-3)
for x, y in train_loader:                          # one epoch
    opt_c.zero_grad(); nn.functional.cross_entropy(classifier(x), y).backward(); opt_c.step()
test_data = datasets.MNIST(root='./data', train=False, transform=transform, download=True)
with torch.no_grad():
```

```
    xt = torch.stack([test_data[i][0] for i in range(len(test_data))]); yt = test_data.targets
    acc = (classifier(xt).argmax(1) == yt).float().mean().item()
print(f"digit classifier accuracy on the 10,000 real test digits: {acc:.1%}")

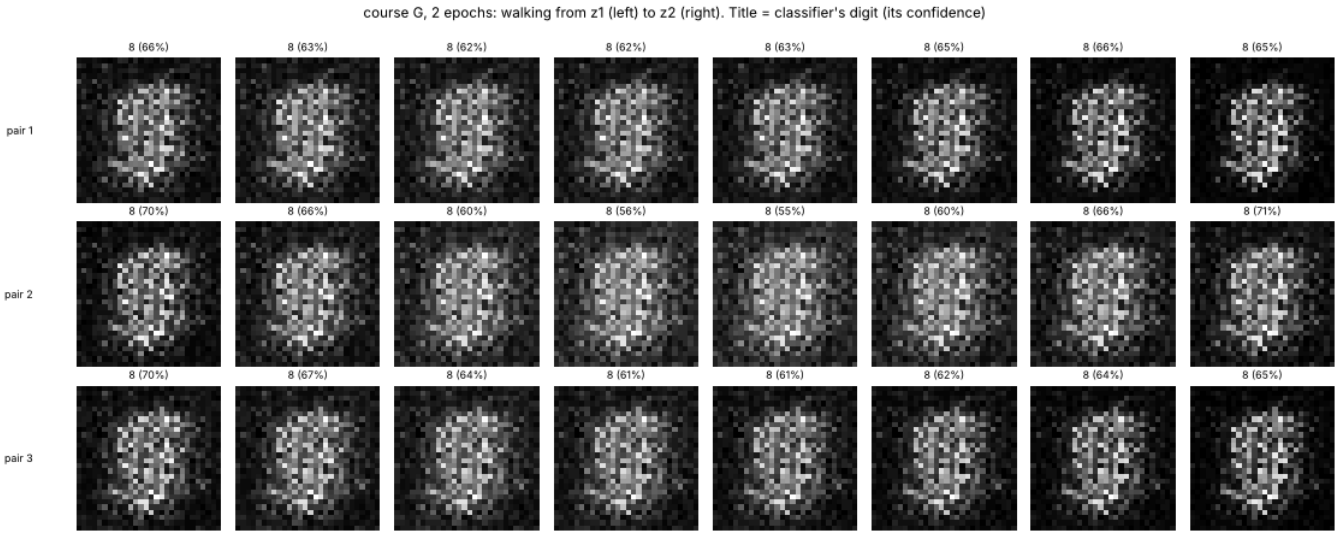
def classify(images):
    with torch.no_grad():
        probs = torch.softmax(classifier(images), 1)
    return probs.argmax(1), probs.max(1).values     # predicted digit and how sure it is
```

```
digit classifier accuracy on the 10,000 real test digits: 93.8%
```

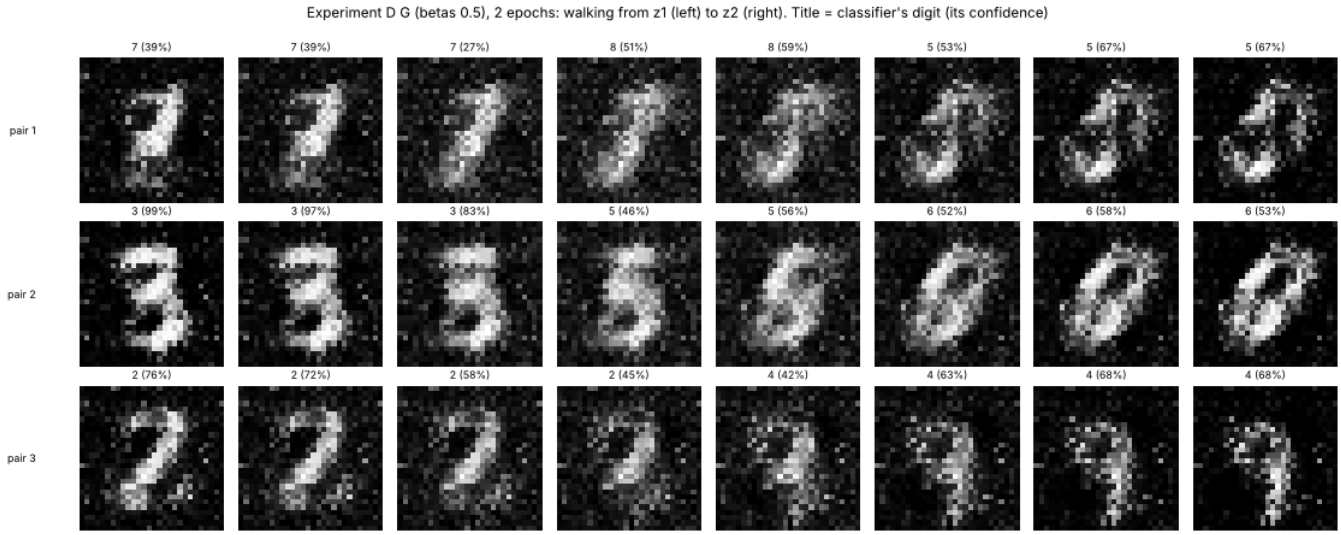
```
def interpolate(G, z1, z2, steps=8, ax_row=None, title=""):
    ts = torch.linspace(0, 1, steps).view(-1, 1)
    imgs = G((1 - ts) * z1 + ts * z2).detach()
    digit, conf = classify(imgs)
    for i in range(steps):
        ax_row[i].imshow(imgs[i][0], cmap='gray'); ax_row[i].axis('off')
        ax_row[i].set_title(f"{digit[i].item()} ({conf[i]:.0%})", fontsize=9)
    ax_row[0].text(-0.3, 0.5, title, transform=ax_row[0].transAxes, ha="right", va="center",
fontsize=9)
    return digit.tolist()

pairs = []
for s in [1, 2, 3]:                          # "Run it 3 times"
    torch.manual_seed(s); pairs.append((torch.randn(1, 100), torch.randn(1, 100)))
for G, gname in [(generator, "course G, 2 epochs"), (G_D, "Experiment D G (betas 0.5), 2
epochs")]:
    fig, axes = plt.subplots(3, 8, figsize=(16, 6.4))
    for r, (za, zb) in enumerate(pairs):
        d = interpolate(G, za, zb, ax_row=axes[r], title=f"pair {r + 1}")
        print(f"{gname}, pair {r + 1}: classifier reads", d)
    plt.suptitle(f"{gname}: walking from z1 (left) to z2 (right). Title = classifier's digit (its
confidence)", y=1.0)
    plt.tight_layout(); plt.show()
```

```
course G, 2 epochs, pair 1: classifier reads [8, 8, 8, 8, 8, 8, 8, 8]
course G, 2 epochs, pair 2: classifier reads [8, 8, 8, 8, 8, 8, 8, 8]
course G, 2 epochs, pair 3: classifier reads [8, 8, 8, 8, 8, 8, 8, 8]
```




```
Experiment D G (betas 0.5), 2 epochs, pair 1: classifier reads [7, 7, 7, 8, 8, 5, 5, 5]
Experiment D G (betas 0.5), 2 epochs, pair 2: classifier reads [3, 3, 3, 5, 5, 6, 6, 6]
Experiment D G (betas 0.5), 2 epochs, pair 3: classifier reads [2, 2, 2, 2, 4, 4, 4, 4]
```

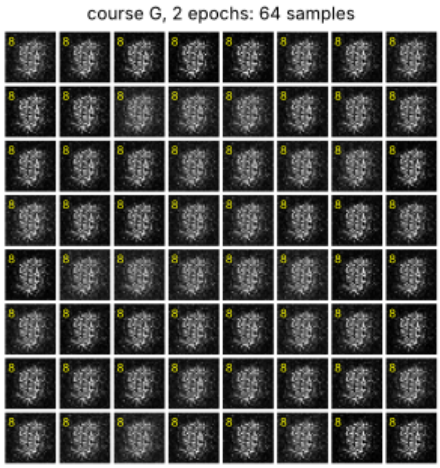


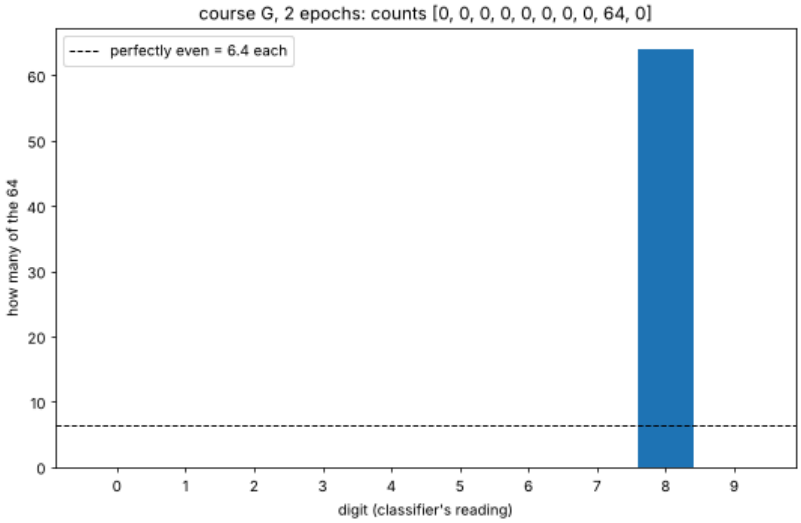
Answers (this run).

1. *Smooth or jumps?* With the **course generator** (first figure of the course cell and the top block) there is nothing to see: every z gives the same collapsed blob, so the walk shows the same image 8 times and the classifier reads "8" every time. That is another way to *detect* mode collapse: moving through latent space changes nothing.
2. With the **β1 = 0.5 generator** (Experiment D) the walk is **smooth**: pair 1 goes 7 → (an in-between, read as 8) → 5, pair 2 goes 3 → 5 → 6, pair 3 goes 2 → 4. *What do the middle images look like?* Blends: strokes of the start digit fade while strokes of the end digit appear, and the classifier is least sure in the middle (27–46% confidence) and most sure near the ends. The digit changes gradually, not by jumping, because G is a smooth function of z: nearby z give similar images. (When the endpoints are different digits, the middle often passes through a third digit that shares strokes with both, like 5 between 3 and 6.)

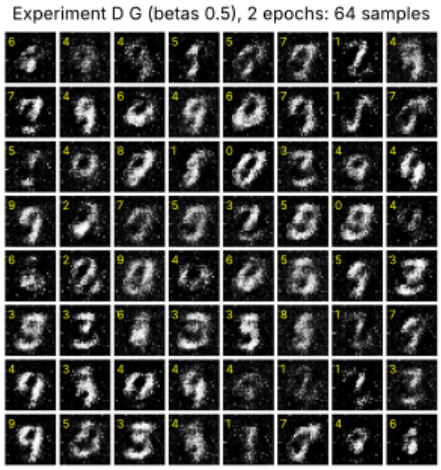
4.2 Generate 64 samples and count the digits: any mode collapse?

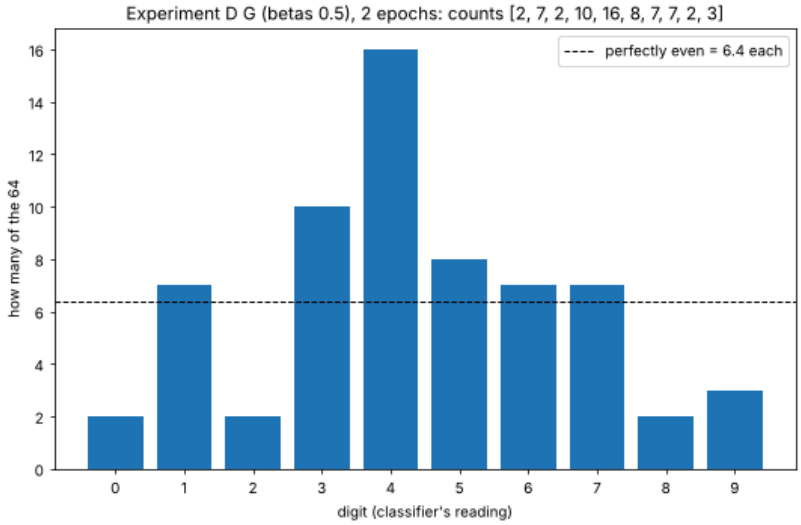
```
torch.manual_seed(2024)
z64 = torch.randn(64, 100)
for G, gname in [(generator, "course G, 2 epochs"), (G_D, "Experiment D G (betas 0.5), 2
epochs")]:
    imgs = G(z64).detach()
    digit, conf = classify(imgs)
    fig, ax = plt.subplots(1, 2, figsize=(14, 5.2), gridspec_kw={"width_ratios": [1, 1.1]})
    ax[0].imshow(make_grid(imgs, nrow=8, normalize=True, value_range=(-1, 1),
pad_value=1).permute(1, 2, 0)); ax[0].axis("off")
    for k in range(64):                       # write the classifier's answer in the corner of
each cell
        ax[0].text((k % 8) * 30 + 3, (k // 8) * 30 + 9, str(digit[k].item()), color="yellow",
fontsize=8)
    counts = torch.bincount(digit, minlength=10)
    ax[1].bar(range(10), counts); ax[1].set_xticks(range(10)); ax[1].set_xlabel("digit
(classifier's reading)"); ax[1].set_ylabel("how many of the 64")
    ax[1].axhline(6.4, ls="--", c="k", lw=1, label="perfectly even = 6.4 each"); ax[1].legend()
    ax[0].set_title(f"{gname}: 64 samples"); ax[1].set_title(f"{gname}: counts
{counts.tolist()}")
    plt.tight_layout(); plt.show()
    print(f"{gname}: digits present = {(counts > 0).sum().item()} of 10, most common =
{counts.argmax().item()} ({counts.max().item()} of 64), "
          f"samples the classifier is >90% sure about: {(conf > 0.9).sum().item()} of 64")
```






```
course G, 2 epochs: digits present = 1 of 10, most common = 8 (64 of 64), samples the classifier
is >90% sure about: 0 of 64
```






```
Experiment D G (betas 0.5), 2 epochs: digits present = 10 of 10, most common = 4 (16 of 64), samp
les the classifier is >90% sure about: 8 of 64
```

Answer (this run). The **course generator** gives **64 of 64 the same output**, read as "8" (with low confidence: it is a blob, not a real 8): **complete mode collapse**. The **β1 = 0.5 generator** shows **all 10 digits** in 64 samples: counts {0: 2, 1: 7, 2: 2, 3: 10, 4: 16, 5: 8, 6: 7, 7: 7, 8: 2, 9: 3}. That is uneven (4s and 3s are common, 0s, 2s and 8s rare) but it is variety, not collapse. Only 8 of the 64 are classified with more than 90% confidence: after 2 epochs many digits are still blotchy, which matches the pictures. With only 64 samples, counts of 2 vs 7 are partly luck; the next cell uses 1,000.

4.3 Bonus: variety and "digit-likeness" of every run, measured on 1,000 samples

Sixty-four samples is a small count. Here every generator from Tasks 1 and 3 makes **1,000** digits; we count how many of each digit the classifier sees, and how many samples it is more than 90% sure about (a rough proxy for "looks like a clear digit"; a classifier can be confidently wrong on junk, so always check the pictures too).

```
runs_all = dict(runs)        # (the Task 1 generator is identical to "baseline (course)")
torch.manual_seed(99)
z1000 = torch.randn(1000, 100)
print(f"{'run':22s} {'digit counts 0..9':48s} {'present':>7s} {'largest':>7s} {'>90% sure':>9s}")
```

```
hist = {}
for name, (G, log, zd) in runs_all.items():
    digit, conf = classify(G(z1000[:, :zd]).detach())
    counts = torch.bincount(digit, minlength=10)
    hist[name] = counts
    print(f"{name:22s} {str(counts.tolist()):48s} {(counts >= 10).sum().item():7d}
{counts.max().item():7d} {(conf > 0.9).float().mean().item():9.1%}")
print("(present = digits with at least 10 of the 1,000 samples; a perfectly even generator gives
100 each)")
```

```
run                    digit counts 0..9                                  present largest >90% sure
A: 5 epochs            [173, 118, 0, 0, 0, 0, 0, 0, 709, 0]                   3     709      0.1%
baseline (course)      [0, 0, 0, 0, 0, 0, 0, 0, 1000, 0]                      1    1000      0.0%
B: latent 10           [0, 0, 0, 0, 0, 0, 0, 0, 1000, 0]                      1    1000      0.0%
C: D lr x10            [0, 0, 0, 0, 0, 0, 0, 0, 1000, 0]                      1    1000      0.0%
D: betas 0.5           [42, 123, 90, 166, 132, 76, 136, 59, 87, 89]          10     166     19.4%
(present = digits with at least 10 of the 1,000 samples; a perfectly even generator gives 100 eac
h)
```

What the 1,000-sample counts say (this run).

- **Baseline, B and C:** all 1,000 samples read as "8": complete collapse, and the classifier is never more than 90% sure (0.0%), because the "8" is really a blob. This is the trap from Class 2 & 3: *a histogram alone would say "lots of eights"*; the grids show there are no real eights at all.
- **A (5 epochs):** 709 "8"s, 173 "0"s, 118 "1"s: three more training epochs have started to break the collapse (the ring "0" and the thin strokes seen in Experiment A's grid), but most samples are still the blob.
- **D (β1 = 0.5): all ten digits present**, between 42 (digit 0) and 166 (digit 3) of each, and 19.4% classified with high confidence. Not perfectly even after 2 epochs, but healthy.

The lesson of this notebook in one line: with the course's original settings (Adam's default β1 = 0.9) this run collapsed; one optimiser setting (β1 = 0.5) changed the result completely. That is exactly why Classes 2–4 switch to β1 = 0.5, and why GAN results must be judged by looking at many samples, not by the loss numbers.

Exit ticket (from the last slide) with answers

1. **Write the full form of GAN. Explain "adversarial" in one sentence.** Generative Adversarial Network(s). Two networks with **opposite goals on the same score**: D wants fakes scored "fake", G wants them scored "real".
2. **For a generated image, D(G(z)) = 0.3. Which way does D want it to move? Which way does G?** D wants it to go **down to 0**; G wants it to go **up to 1**.
3. **In the notebook, what does `fake.detach()` do in the discriminator step?** It stops gradients flowing back into G, so only D learns in that step (we proved it above: G's gradient was `None`).
4. **Muddiest point:** yours to write. A common one is "why `labels_real` for fakes?": see Task 2c and the saturating vs non-saturating plot.

Viva practice: 14 questions with short answers

1. **Generative vs discriminative model?** A discriminative model learns p(label | image) and outputs a label; a generative model learns to sample like p(image) and outputs new data.
2. **What goes in and out of G and D here?** G: 100 noise numbers → a 1×28×28 image (100 → 256 → 784, Tanh). D: an image → one probability P(real) (784 → 256 → 1, Sigmoid).
3. **Why Tanh at the end of G?** Real pixels were normalised to −1…+1; Tanh gives fakes the same range, so D cannot spot fakes by range alone.
4. **Why LeakyReLU(0.2) in D?** G learns only through gradients flowing back through D; LeakyReLU never fully blocks them for negative inputs.
5. **Write the minimax objective. Who pushes which way?** min_G max_D E[log D(x)] + E[log(1 − D(G(z)))]. D pushes V up, G pushes it down; G cannot affect the first term.
6. **How does `loss_d` relate to V?** BCE is the negative log, so `loss_d = −[log D(x) + log(1 − D(G(z)))]`; minimising `loss_d` = maximising V.
7. **What is the optimal D for a fixed G, and what happens when G is perfect?** D*(x) = p_data(x) / (p_data(x) + p_g(x)); if p_g = p_data, D* = ½ everywhere: a Nash equilibrium.
8. **Where do 1.386 and 0.693 come from?** If D outputs 0.5 everywhere, each BCE term is −ln 0.5 = 0.693; `loss_d` has two terms (1.386), `loss_g` one (0.693).
9. **Why does G's loss use target 1 (non-saturating loss)?** log(1 − D(G(z))) is flat when D(G(z)) ≈ 0, so G gets almost no gradient early; −log D(G(z)) is steep there (about 1000× stronger at 0.001 in our check). Same goal, better gradients.
10. **Why two optimizers?** Each holds only its own network's weights, so each `step()` updates one player while the other stays fixed.
11. **Why `real.size(0)` instead of 64?** The last batch of each epoch has only 32 images (60,000 = 937 × 64 + 32).
12. **Do GAN losses tell you image quality?** No. Each loss is measured against a moving opponent; always look at samples (and, better, count variety).
13. **What is mode collapse and how would you detect it here?** G produces only one or a few kinds of output (e.g. mostly one digit). Look at a grid of 64+ samples and count digits with a classifier, as in Task 4.
14. **Which network is kept after training, and why do we need the other?** G is kept (it makes images). D is the coach: it learns from real data what digits look like and passes that knowledge to G through its gradients; there is no formula to score "looks like a digit" otherwise.

Next: notebook `02_gan_training_dynamics.ipynb` (Classes 2 & 3: why this game is hard to win) and `03_dcgan_and_conditional_gan.ipynb` (the DCGAN mini-project and the conditional-GAN assignment from this notebook's last cells).

Source: code reconstructed from the course guide "GAN Class 1 Notebook Explained" (CV-Week4-Lecture1.ipynb) and the GAN Class 1 slides; reference: Goodfellow et al., Generative Adversarial Nets (2014).

02 · GAN training dynamics: equilibrium, instability, mode collapse

The Class 2 & 3 notebook (GAN-Class-2-3-Training-Dynamics.ipynb), rebuilt and run

Covers (GAN Class 2 & 3 slides): Nash equilibrium and why gradient play circles it · instability · mode collapse and mode hopping · five popular fixes, tested · diagnosing a real MNIST GAN with D(x) / D(G(z)) plots, fixed-noise grids and a digit histogram.

Where the code comes from. The course guide *"GAN Class 2 & 3 Notebook Explained"* describes this notebook part by part, with its function names (`play_game` , `sample_real` , `Generator2D` , `Discriminator2D` , `mode_coverage` , `train_ring_gan` , `show_snapshots` , `digit_histogram` , `train_mnist_gan`), defaults and experiments. This notebook follows it exactly. The guide quotes the results of the course's own test run; **your numbers here come from this run** and are compared with the guide's honestly where they differ (GANs are sensitive to seeds and hardware; compare patterns, not exact numbers).

> **The key fact:** a GAN is a **game**, not a single optimisation. A classifier chases a target that stays still. In a GAN, G's target is D's opinion, and D is learning too. Like hiking down a valley while someone keeps reshaping the ground.

Part	What you do	Links to theory
0	imports and seeds	
1	play the x·y game; watch the players circle the equilibrium	Nash equilibrium
2	8-cluster ring data, the two networks, a coverage measure	what a mode is
3	train a GAN on the ring and watch it collapse and hop	mode collapse
4	test five fixes and fill in a results table	the toolbox
5	the Class 1 MNIST GAN twice, with diagnostic plots	instability
6	reflection questions with model answers	all

Cells marked ✏️ **Predict** ask for a guess *before* you run the next cell. Write the guess first; the answer is revealed after the run.

Words you need

Term	Plain meaning
Game / zero-sum game	each player's best choice depends on the other's; in zero-sum, one's gain is exactly the other's loss (a GAN's minimax objective is one)
Nash equilibrium	choices where no player can do better by changing **only its own** choice
Gradient descent / ascent	moving a number in the direction that makes a value smaller / larger

Term	Plain meaning
Mode	a peak of the data: one "kind" of sample (one cluster, one digit)
Mode collapse / hopping	G produces only one or a few modes / the collapsed output jumps between modes during training
Logit	a raw score before the sigmoid squashes it into 0–1
Label smoothing	training D with target 0.9 for real images instead of 1.0
Instance noise	adding a little random noise to the images D sees
TTUR	two time-scale update rule: D and G get different learning rates
Minibatch std	giving D one extra number: how spread out the whole batch is
Adam β1	how much of its previous direction Adam keeps (its momentum)
Seed	the starting number for random choices; same seed → same run

Part 0 · Setup

```
%matplotlib inline
import math
import time
import numpy as np
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torchvision.utils import make_grid
from torch.utils.data import DataLoader

torch.manual_seed(0)
np.random.seed(0)
plt.rcParams["figure.dpi"] = 80
print("torch", torch.__version__, "| GPU available:", torch.cuda.is_available())
```

```
torch 2.14.1+cu130 | GPU available: False
```

Part 1 · The smallest possible game: V(x, y) = x · y

A real GAN has hundreds of thousands of weights. To *see* the behaviour, shrink the game to two numbers:

- **Player x** wants V = x·y **small** (it plays G's role);
- **Player y** wants V = x·y **large** (it plays D's role).

Where is the equilibrium? At **(0, 0)**. If y = 0, V = 0 whatever x does, so x has nothing to gain by moving; if x = 0, the same holds for y. Neither player regrets its choice: a Nash equilibrium.

Gradient play. The slope of x·y with respect to x is y, and with respect to y is x. So each step is

```
x = x - lr * y   # x walks downhill on V (gradient descent)
y = y + lr * x   # y walks uphill on V (gradient ascent)
```

`play_game` runs 200 of these steps from **(1, 1)** with **lr = 0.1**, in two ways:

- **simultaneous:** both players use the *old* values: `x, y = x - lr * y, y + lr * x`;
- **alternating:** x moves first, then y reacts to the *new* x. This matches our GAN code (D steps, then G steps).

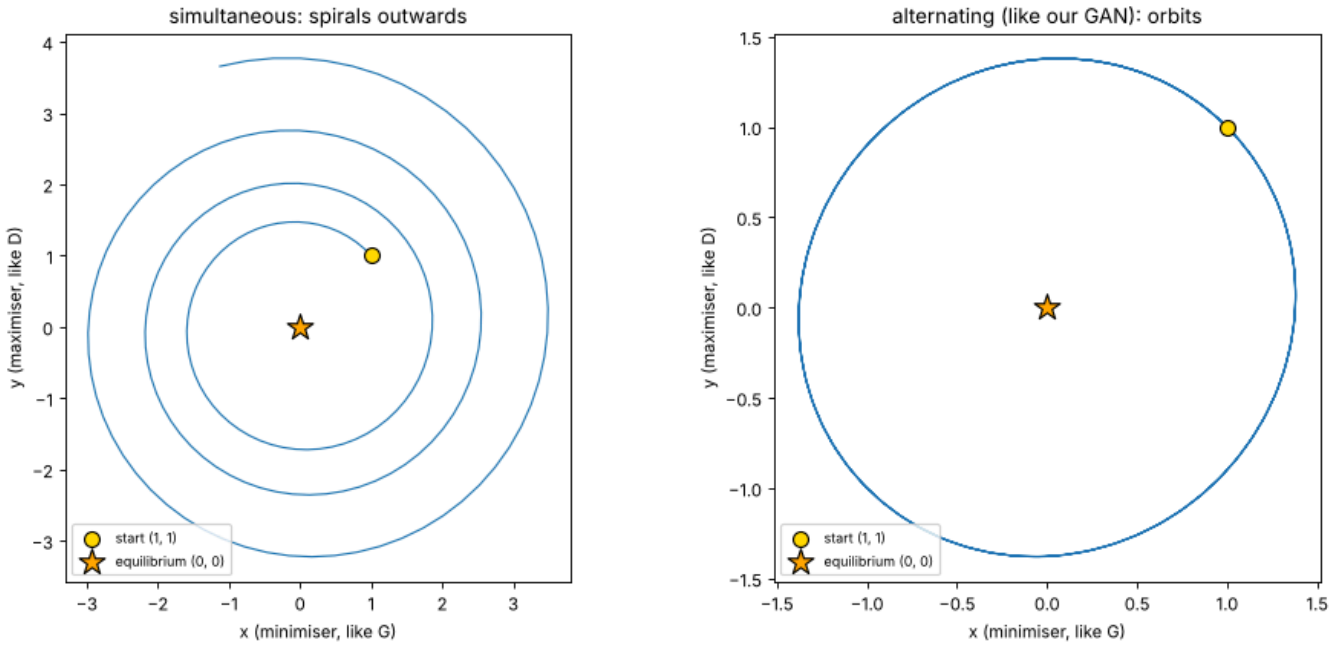
```
def play_game(steps=200, lr=0.1, simultaneous=True, start=(1.0, 1.0)):
    x, y = start
    path = [(x, y)]
    for _ in range(steps):
        if simultaneous:
            x, y = x - lr * y, y + lr * x      # both use the old values
        else:
            x = x - lr * y                     # x moves first...
            y = y + lr * x                     # ...then y reacts to the NEW x
        path.append((x, y))
    return np.array(path)
```

> ✏️ **Predict:** after 200 steps, which version ends closer to the equilibrium (0, 0)? Will either of them reach it? (Start distance = √2 ≈ 1.41.)

```
sim = play_game(simultaneous=True)
alt = play_game(simultaneous=False)
print(f"start distance from (0, 0): {math.hypot(1, 1):.2f}")
print(f"simultaneous: distance after 200 steps = {np.hypot(*sim[-1]):.2f}")
print(f"alternating : distance after 200 steps = {np.hypot(*alt[-1]):.2f}")

fig, ax = plt.subplots(1, 2, figsize=(12, 5.5))
for a, path, title in [(ax[0], sim, "simultaneous: spirals outwards"), (ax[1], alt, "alternating
(like our GAN): orbits")]:
    a.plot(path[:, 0], path[:, 1], lw=1)
    a.scatter(*path[0], c="gold", s=80, zorder=3, edgecolor="k", label="start (1, 1)")
    a.scatter(0, 0, marker="*", c="orange", s=250, zorder=3, edgecolor="k", label="equilibrium
(0, 0)")
    a.set_aspect("equal"); a.set_title(title); a.set_xlabel("x (minimiser, like G)");
a.set_ylabel("y (maximiser, like D)"); a.legend(loc="lower left", fontsize=8)
plt.tight_layout(); plt.show()
```

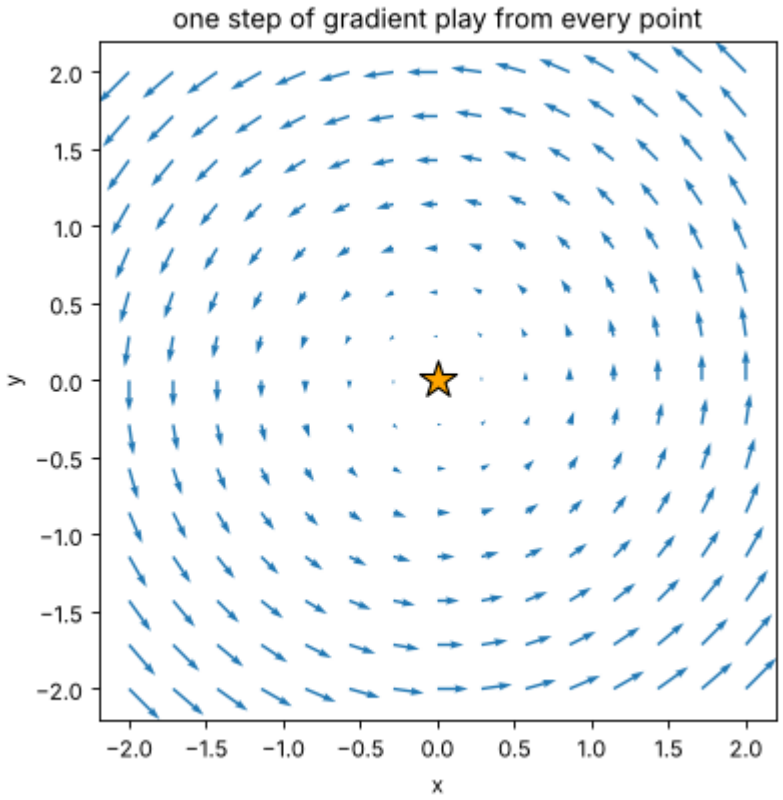
```
start distance from (0, 0): 1.41
simultaneous: distance after 200 steps = 3.83
alternating : distance after 200 steps = 1.36
```



Answer revealed. Neither reaches the equilibrium. **Simultaneous** updates spiral **outwards**: the distance grows from 1.41 to **3.83**. **Alternating** updates go round and round on a closed loop: the distance after 200 steps is **1.36**, about where it started. These are exactly the guide's numbers (about 3.83 and about 1.36): this part involves no randomness, so every machine gets the same result.

Why does it circle? The arrow plot below draws, at every point, the direction one step of gradient play pushes (x, y). Each player's step depends on the **other** player's position:

```
xs, ys = np.meshgrid(np.linspace(-2, 2, 15), np.linspace(-2, 2, 15))
u, v = -ys, xs                           # x moves by -y (descent), y moves by +x (ascent)
plt.figure(figsize=(5.5, 5.5))
plt.quiver(xs, ys, u, v, color="tab:blue")
plt.scatter(0, 0, marker="*", c="orange", s=300, edgecolor="k", zorder=3)
plt.gca().set_aspect("equal"); plt.title("one step of gradient play from every point");
plt.xlabel("x"); plt.ylabel("y")
plt.show()
```



The arrows go **around** (0, 0), never towards it. Follow the quadrants: at x > 0, y > 0, x moves left and y moves up; at x < 0, y > 0, x moves left and y down; … a rotation. Nobody's step aims at the equilibrium.

With simultaneous updates the distance is multiplied by $\sqrt{(1 + lr^2)}$ at every step (for lr = 0.1, 200 steps: √2 · 1.01^100 ≈ 3.83), so it grows for ever. Alternating updates cancel that growth exactly, so the path stays on a closed loop.

Exercise 1 · Does a smaller learning rate fix it?

> ✏️ **Predict:** with lr = 0.01, will the players reach (0, 0)?

```
print("simultaneous updates, 200 steps:")
for lr in [0.01, 0.1, 0.5]:
    d = np.hypot(*play_game(lr=lr)[-1])
    print(f"  lr = {lr:<4}: distance = {d:,.3f}   (formula sqrt(2) * (1 + lr^2)^100 =
{math.sqrt(2) * (1 + lr**2) ** 100:,.3f})")
print("alternating updates, 2,000 steps (min / max distance along the path):")
for lr in [0.01, 0.1, 0.5]:
    d = np.hypot(*play_game(steps=2000, lr=lr, simultaneous=False).T)
    print(f"  lr = {lr:<4}: final {d[-1]:.3f}, min {d.min():.3f}, max {d.max():.3f}")
```

```
simultaneous updates, 200 steps:
  lr = 0.01: distance = 1.428   (formula sqrt(2) * (1 + lr^2)^100 = 1.428)
  lr = 0.1 : distance = 3.825   (formula sqrt(2) * (1 + lr^2)^100 = 3.825)
  lr = 0.5 : distance = 6,942,506,557.581   (formula sqrt(2) * (1 + lr^2)^100 = 6,942,506,557.58
1)
alternating updates, 2,000 steps (min / max distance along the path):
  lr = 0.01: final 1.408, min 1.407, max 1.414
  lr = 0.1 : final 1.367, min 1.345, max 1.414
  lr = 0.5 : final 1.237, min 1.095, max 1.414
```

Answer revealed. No. With simultaneous updates the distance after 200 steps is **1.428** for lr = 0.01 (spirals out very slowly), **3.825** for lr = 0.1 and about **6.9 billion** for lr = 0.5 (explodes), matching the guide's 1.428 / 3.825 / ≈ 6.9 billion and the $\sqrt{(1 + lr^2)}$ formula exactly. With alternating updates and 2,000 steps the distance stays between about 1.1 and 1.41 for every learning rate: the players keep orbiting and **never** approach (0, 0). A smaller learning rate only slows the spiral; it does not point the players at the equilibrium.

One sentence: two players each descending **their own** loss is not the same as minimising **one** loss, because each player's gradient depends on the other player's position; the combined motion rotates. Real GAN fixes therefore change the game or the optimiser, not just the step size.

Part 2 · A GAN you can see: 8 clusters on a ring

MNIST images have 784 numbers each, so you can't see the whole data distribution. Here each sample is just a **point (x, y)**.

```
MODES = torch.tensor([[2 * math.cos(2 * math.pi * i / 8), 2 * math.sin(2 * math.pi * i / 8)] for
i in range(8)])

def sample_real(n, std=0.02):
    idx = torch.randint(0, 8, (n,))
    return MODES[idx] + std * torch.randn(n, 2)
```

- `MODES` holds **8 centre points** evenly spaced on a circle of radius 2.
- `sample_real(n)` picks a random centre for each of the n points and adds a tiny bit of noise (`std=0.02`): **8 tight clusters**. Each cluster is **one mode**; a good generator must put points in all 8.

The two networks

```
class Generator2D(nn.Module):
    def __init__(self, z_dim=256):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(z_dim, 128), nn.ReLU(),
            nn.Linear(128, 128), nn.ReLU(),
```

```
            nn.Linear(128, 2))                  # a point (x, y)
    def forward(self, z):
        return self.net(z)


class Discriminator2D(nn.Module):
    def __init__(self, mbstd=False):
        super().__init__()
        self.mbstd = mbstd
        self.net = nn.Sequential(
            nn.Linear(2 + int(mbstd), 128), nn.LeakyReLU(0.2),
            nn.Linear(128, 128), nn.LeakyReLU(0.2),
            nn.Linear(128, 1))                  # a raw score (logit): no Sigmoid
    def forward(self, x):
        if self.mbstd:
            # minibatch standard deviation: how spread out is the WHOLE batch?
            spread = x.std(dim=0).mean()                          # one number for the batch
            x = torch.cat([x, spread.expand(x.size(0), 1)], 1)  # copied to every point as a 3rd
input
        return self.net(x)

bce = nn.BCEWithLogitsLoss()
```

- **Generator2D:** a noise vector (256 numbers by default) → two hidden layers of 128 with ReLU → 2 numbers, a point.
- **Discriminator2D:** a point → LeakyReLU hidden layers → **one raw score (a logit)**. There is **no Sigmoid** at the end because we use `nn.BCEWithLogitsLoss`, which applies the sigmoid *inside* the loss. Same maths as Class 1's Sigmoid + `BCELoss`, just more numerically stable.
- `mbstd=True` adds **minibatch standard deviation**: D gets one extra input, the average spread of the whole batch, copied to every point. A batch where all points sit in one place has a tiny spread, so D can notice **missing variety**. Used in Part 4.

Measuring variety: `mode_coverage`

Returns two numbers for a set of points:

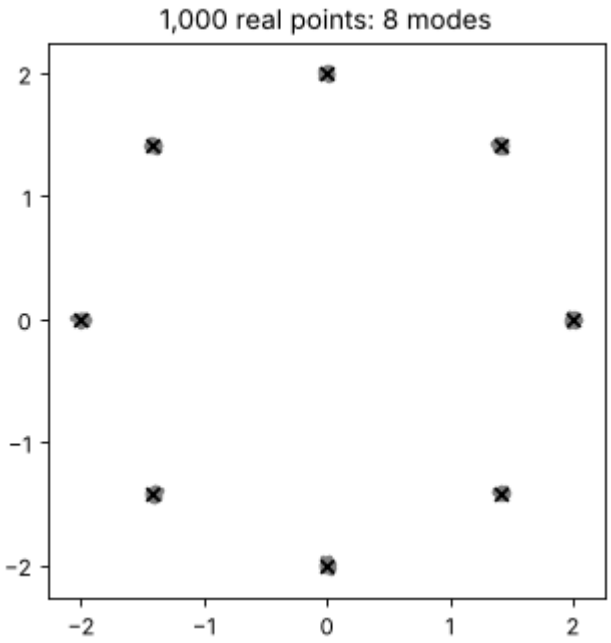
- **modes covered (0–8):** how many clusters receive **at least 2%** of the points, where a point counts for a cluster when it is **within 0.3** of that cluster's centre;
- **quality (0–1):** the fraction of points that land near **any** cluster.

```
def mode_coverage(points, radius=0.3, min_share=0.02):
    dist = torch.cdist(points, MODES)            # distance from every point to every centre
    nearest_d, nearest = dist.min(dim=1)
    near = nearest_d < radius
    share = torch.bincount(nearest[near], minlength=8).float() / len(points)
    modes = int((share >= min_share).sum())
    quality = near.float().mean().item()
    return modes, quality

real_pts = sample_real(1000)
print("real data: modes covered = %d / 8, quality = %.2f" % mode_coverage(real_pts))
one_blob = MODES[0] + 0.02 * torch.randn(1000, 2)
print("a 'collapsed' set, all near cluster 0: modes covered = %d / 8, quality = %.2f" %
mode_coverage(one_blob))

plt.figure(figsize=(4.5, 4.5))
plt.scatter(*real_pts.T, s=4, c="grey"); plt.scatter(*MODES.T, marker="x", c="k")
plt.gca().set_aspect("equal"); plt.title("1,000 real points: 8 modes"); plt.show()
```

```
real data: modes covered = 8 / 8, quality = 1.00
a 'collapsed' set, all near cluster 0: modes covered = 1 / 8, quality = 1.00
```



Real data scores **(8, 1.00)**. A collapsed generator can score **high quality but only 1 mode**: every point looks real, but they are all in the same place. That is why we need *both* numbers.

Part 3 · Train the baseline and watch it collapse

`train_ring_gan(...)` is the Class 1 loop with switches added for the experiments. Its defaults are the **baseline**: `z_dim=256`, `lr_g=lr_d=1e-4`, Adam `betas=(0.5, 0.999)`, batches of 256 points, **4,000 steps**, `seed=0`.

```python
def train_ring_gan(z_dim=256, lr_g=1e-4, lr_d=1e-4, betas=(0.5, 0.999), batch=256, steps=4000,
seed=0,
                   real_label=1.0, instance_noise=0.0, mbstd=False, snapshot_every=500):
    torch.manual_seed(seed)
    G, D = Generator2D(z_dim), Discriminator2D(mbstd)
    opt_g = optim.Adam(G.parameters(), lr=lr_g, betas=betas)
    opt_d = optim.Adam(D.parameters(), lr=lr_d, betas=betas)
    fixed_z = torch.randn(1000, z_dim)           # reused for every snapshot
    losses, snapshots = [], []
    for step in range(1, steps + 1):
        # Step 1: train D (G frozen by .detach())
        real = sample_real(batch)
        fake = G(torch.randn(batch, z_dim)).detach()
        if instance_noise > 0:                   # blur both piles before D sees them
            real = real + instance_noise * torch.randn_like(real)
            fake = fake + instance_noise * torch.randn_like(fake)
        loss_d = bce(D(real), torch.full((batch, 1), real_label)) + bce(D(fake),
torch.zeros(batch, 1))
        opt_d.zero_grad(); loss_d.backward(); opt_d.step()
        # Step 2: train G (fresh fakes, no detach)
        fake = G(torch.randn(batch, z_dim))
        loss_g = bce(D(fake), torch.ones(batch, 1))  # non-saturating loss from Class 1
        opt_g.zero_grad(); loss_g.backward(); opt_g.step()
        losses.append((loss_d.item(), loss_g.item()))
        if step % snapshot_every == 0:
            with torch.no_grad():
                pts = G(fixed_z)
            snapshots.append((step, pts, *mode_coverage(pts)))
    return G, np.array(losses), snapshots
```

```
def show_snapshots(snapshots, title=""):
    fig, axes = plt.subplots(1, len(snapshots), figsize=(2.1 * len(snapshots), 2.4))
    ref = sample_real(1000)
    for a, (step, pts, modes, quality) in zip(axes, snapshots):
        a.scatter(*ref.T, s=1, c="lightgrey"); a.scatter(*pts.T, s=1, c="tab:blue")
        a.set_xlim(-3, 3); a.set_ylim(-3, 3); a.set_aspect("equal"); a.set_xticks([]);
a.set_yticks([])
        a.set_title(f"step {step}\n{modes}/8 modes", fontsize=9)
    plt.suptitle(title, y=1.04); plt.tight_layout(); plt.show()

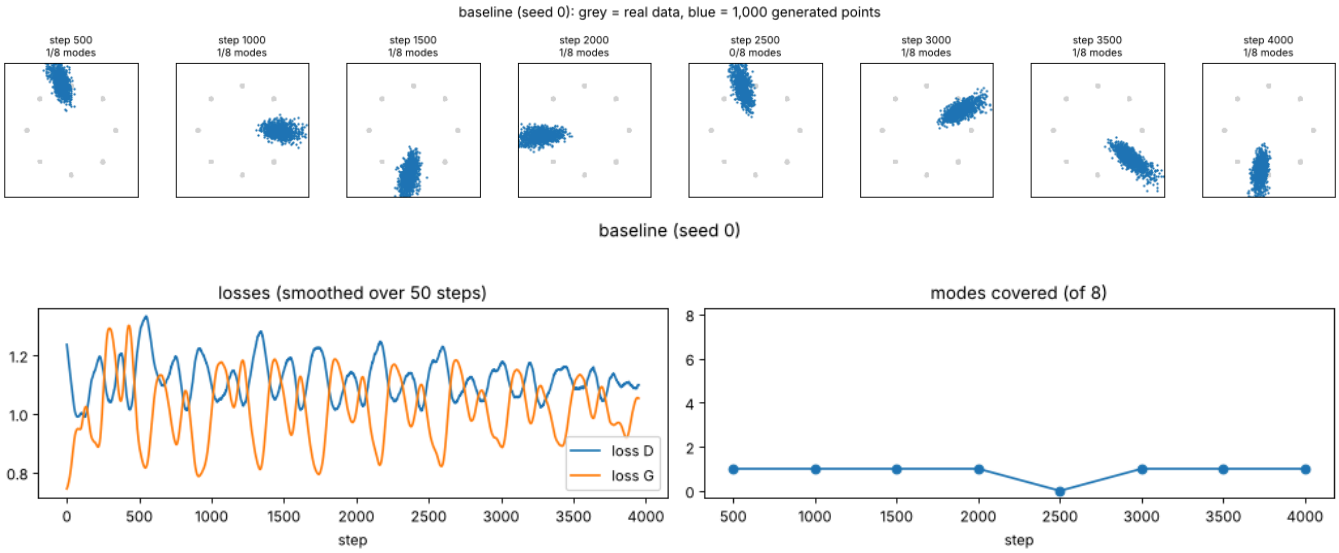

def show_curves(losses, snapshots, title=""):
    fig, ax = plt.subplots(1, 2, figsize=(13, 3.2))
    k = 50
    for i, name in enumerate(["loss D", "loss G"]):
        ax[0].plot(np.convolve(losses[:, i], np.ones(k) / k, mode="valid"), label=name)
    ax[0].set_xlabel("step"); ax[0].set_title("losses (smoothed over 50 steps)"); ax[0].legend()
    ax[1].plot([s[0] for s in snapshots], [s[2] for s in snapshots], "o-")
    ax[1].set_ylim(-0.3, 8.3); ax[1].set_xlabel("step"); ax[1].set_title("modes covered (of 8)")
    plt.suptitle(title, y=1.03); plt.tight_layout(); plt.show()
```

Code	What it does
`torch.manual_seed(seed)`	makes the run repeatable
`fixed_z = torch.randn(1000, z_dim)`	1,000 noise vectors reused for every snapshot, so pictures are comparable over time
Step 1: `real = sample_real(batch)`, `fake = G(...).detach()`	one pile of real points, one of fakes; `.detach()` keeps G frozen
`if instance_noise > 0: ...`	optional: blur both piles with random noise before D sees them
`loss_d = bce(D(real), real_label) + bce(D(fake), 0)`	D's loss; `real_label` is 1.0 normally, 0.9 with label smoothing
Step 2: `fake = G(torch.randn(...))`	fresh fakes, **without** detach, so gradients flow into G
`loss_g = bce(D(fake), 1)`	the non-saturating generator loss from Class 1
`if step % snapshot_every == 0`	every 500 steps, draw the 1,000 fixed-noise points and record their coverage

> ✏️ **Predict:** after 4,000 steps, how many of the 8 clusters will the generator cover? Will the loss curves warn you if something is wrong?

```
t0 = time.time()
G_base, losses_base, snaps_base = train_ring_gan()
print(f"baseline trained in {time.time() - t0:.0f} s")
print("modes covered at each snapshot:", [s[2] for s in snaps_base])
print("quality at each snapshot     :", [round(s[3], 2) for s in snaps_base])
print("blob centre at each snapshot  :", [tuple(round(float(v), 1) for v in s[1].mean(0)) for s
in snaps_base])
show_snapshots(snaps_base, "baseline (seed 0): grey = real data, blue = 1,000 generated points")
show_curves(losses_base, snaps_base, "baseline (seed 0)")
```

```
baseline trained in 25 s
modes covered at each snapshot: [1, 1, 1, 1, 0, 1, 1, 1]
quality at each snapshot     : [0.07, 0.25, 0.23, 0.18, 0.01, 0.03, 0.27, 0.07]
blob centre at each snapshot  : [(-0.5, 2.2), (1.7, -0.0), (-0.2, -2.1), (-2.0, -0.3), (-0.6, 1.
9), (1.5, 0.9), (1.5, -1.3), (-0.4, -1.9)]
```



Answer revealed. At **every** snapshot the 1,000 blue points form **one blob**, covering at most **1 of 8** clusters (0 of 8 at step 2,500, when the blob sat between clusters). A thousand different noise vectors are mapped to almost the same place: **mode collapse**. And the blob **moves** between snapshots: top, right, bottom, left, top again, then right, bottom-right, bottom (see the printed blob centres): **mode hopping**. Meanwhile the loss plot wobbles in a regular rhythm between about 0.8 and 1.3 and says nothing about the 7 missing clusters. **Losses do not measure variety.**

This matches the guide's description of its own seed-0 run (one blob, at most 1 of 8, hopping, losses between about 0.8 and 1.4). One small difference: in our run the blob is fairly wide and often sits just beside a cluster rather than on it, so its quality score is low (0.01–0.27); the guide describes a tight blob.

Why does this happen?

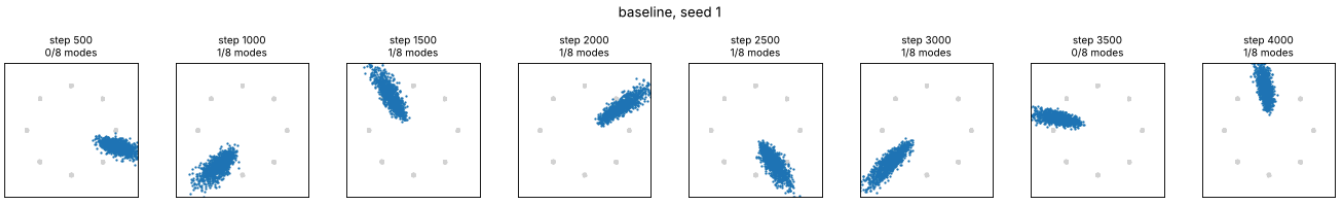
- **D judges one point at a time.** A point in (or near) a real cluster looks real on its own. D never sees that *all* the points are in the same place, so it cannot punish missing variety.
- **G only follows today's D.** Its gradient says "move towards what D likes right now". The quickest way to lower its loss is to send every noise vector to D's current favourite spot.
- **Hopping is the chase from Part 1.** G settles on a spot; D learns that points there are fake; G's gradient pushes it somewhere D still likes; D follows. The rhythmic wobble in the loss plot is this chase.

The original GAN paper warned about exactly this, calling it the **"Helvetica scenario"**: G collapses too many values of z to the same output.

Exercise 2 · Same settings, different seed

```
G_s1, losses_s1, snaps_s1 = train_ring_gan(seed=1)
print("seed 1, modes covered at each snapshot:", [s[2] for s in snaps_s1])
show_snapshots(snaps_s1, "baseline, seed 1")
```

```
seed 1, modes covered at each snapshot: [0, 1, 1, 1, 1, 1, 0, 1]
```



Exercise 2 answers (from this run).

1. *How many modes?* **1 of 8** at almost every snapshot (0 at step 2,500), and the covered spot **changes** between snapshots: mode hopping.
2. *With `seed=1` ?* The same kind of collapse: 0 or 1 of 8 at every snapshot (`[0, 1, 1, 1, 1, 1, 0, 1]`), in different places. The failure is not bad luck with one seed.
3. *Why doesn't D punish G for ignoring the other clusters?* Because D only ever scores **single points**, and a single point near a cluster looks real. Nothing in the loss rewards spreading out.

Part 4 · Testing five fixes

Start from the collapsed baseline and change **one thing at a time**.

Experiment	Change	The idea behind it
A · one-sided label smoothing	`real_label=0.9`	stop D becoming over-confident about real data
B · instance noise	`instance_noise=0.3`	blur both piles so D's gradients stay useful
C · TTUR	`lr_d=4e-4` (D learns 4× faster)	keep D close to its best response
D · minibatch std	`mbstd=True`	let D **see** how spread out the batch is
E · smaller noise vector	`z_dim=2`	give G a simpler input to map

> ✏️ **Predict (yes / no / partly) for each:** does it fix the collapse? Hint: ask *which problem* each fix was designed for.

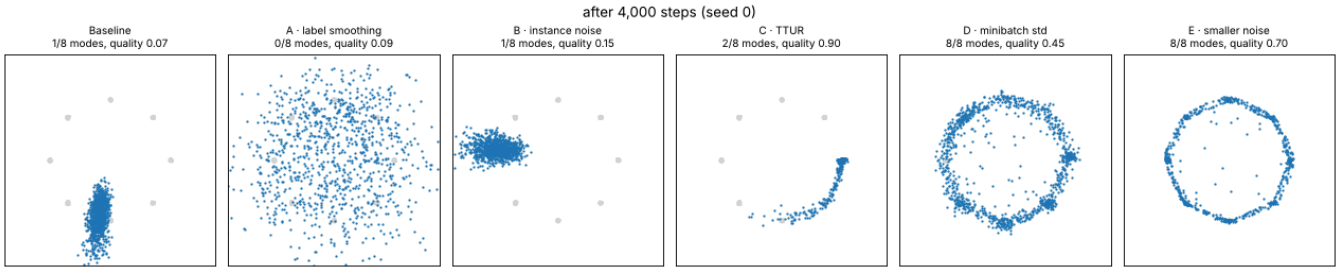
```
experiments = {
    "A · label smoothing (real_label=0.9)": dict(real_label=0.9),
    "B · instance noise (0.3)": dict(instance_noise=0.3),
    "C · TTUR (lr_d=4e-4)": dict(lr_d=4e-4),
    "D · minibatch std (mbstd=True)": dict(mbstd=True),
    "E · smaller noise (z_dim=2)": dict(z_dim=2),
}
results = {"Baseline": (losses_base, snaps_base)}
t0 = time.time()
for name, kwargs in experiments.items():
    _, losses, snaps = train_ring_gan(**kwargs)
    results[name] = (losses, snaps)
    print(f"{name:38s} modes per snapshot {[s[2] for s in snaps]}  final quality {snaps[-1]
[3]:.2f}")
print(f"(5 runs in {time.time() - t0:.0f} s)")
```

```
A · label smoothing (real_label=0.9)   modes per snapshot [1, 1, 1, 2, 0, 0, 0, 0]  final quality
0.09
B · instance noise (0.3)               modes per snapshot [0, 1, 1, 1, 1, 1, 1, 1]  final quality
0.15
```

```
C · TTUR (lr_d=4e-4)                  modes per snapshot [1, 2, 3, 3, 3, 3, 2, 2]  final quality
0.90
D · minibatch std (mbstd=True)        modes per snapshot [1, 0, 0, 0, 3, 8, 8, 8]  final quality
0.45
E · smaller noise (z_dim=2)           modes per snapshot [1, 2, 6, 8, 8, 8, 8, 8]  final quality
0.70
(5 runs in 111 s)
```

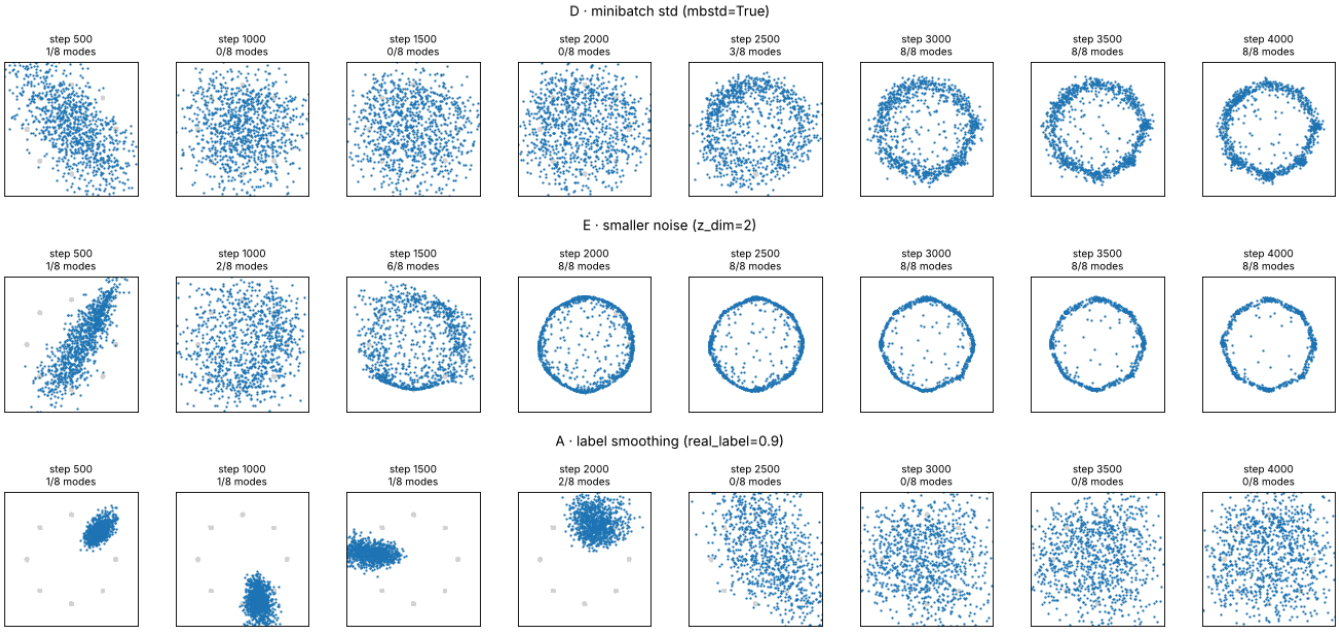
```
fig, axes = plt.subplots(1, 6, figsize=(16, 3.2))
ref = sample_real(1000)
for a, (name, (losses, snaps)) in zip(axes, results.items()):
    step, pts, modes, quality = snaps[-1]
    a.scatter(*ref.T, s=1, c="lightgrey"); a.scatter(*pts.T, s=1, c="tab:blue")
    a.set_xlim(-3.5, 3.5); a.set_ylim(-3.5, 3.5); a.set_aspect("equal"); a.set_xticks([]);
a.set_yticks([])
    a.set_title(f"{name.split(' (')[0]}\n{modes}/8 modes, quality {quality:.2f}", fontsize=9)
plt.suptitle("after 4,000 steps (seed 0)", y=1.02); plt.tight_layout(); plt.show()

print(f"{'experiment':38s} {'final modes':>11s} {'best modes at any snapshot':>27s} {'final
quality':>13s}")
for name, (losses, snaps) in results.items():
    print(f"{name:38s} {snaps[-1][2]:>9d}/8 {max(s[2] for s in snaps):>25d}/8 {snaps[-1]
[3]:>13.2f}")
```



experiment	final modes	best modes at any snapshot	final quality
Baseline	1/8	1/8	0.07
A · label smoothing (real_label=0.9)	0/8	2/8	0.09
B · instance noise (0.3)	1/8	1/8	0.15
C · TTUR (lr_d=4e-4)	2/8	3/8	0.90
D · minibatch std (mbstd=True)	8/8	8/8	0.45
E · smaller noise (z_dim=2)	8/8	8/8	0.70

```
for name in ["D · minibatch std (mbstd=True)", "E · smaller noise (z_dim=2)", "A · label
smoothing (real_label=0.9)"]:
    show_snapshots(results[name][1], name)
```



Results table (filled in from this run, next to the course guide's run)

Experiment	Change	Fixes collapse?	This run (seed 0): final modes, quality	Best modes at any snapshot	Course guide's run (seed 0)
Baseline	none	—	**1/8**, 0.07	1/8	1/8
A · one-sided label smoothing	`real_label=0.9`	**no** (worse)	**0/8**, 0.09: points scattered all over, missing the clusters	2/8	0/8, scattered
B · instance noise	`instance_noise=0.3`	**no**	**1/8**, 0.15: still one blob	1/8	1/8, still one blob
C · TTUR	`lr_d=4e-4`	**no** (partly sharper)	**2/8**, 0.90: a sharp arc reaching one or two clusters	3/8	1–2 of 8, sharper points
D · minibatch std	`mbstd=True`	**yes**	**8/8**, 0.45: a ring with all 8 clusters, many points still between them	8/8	8/8
E · smaller noise vector	`z_dim=2`	**yes**	**8/8**, 0.70: a thin ring	8/8	8/8, as a thin ring

Our run reproduces the guide's pattern **fix for fix**. The only differences are details: TTUR covered 2 (at best 3) clusters instead of 1–2, and minibatch std recovered all 8 clusters but with a lower quality score (0.45: about half the points lie on the "bridges" between clusters) than a perfect generator would get. Minibatch std needed time: it was at 0–3 modes until step 2,500, then jumped to 8/8 by step 3,000. Label smoothing briefly touched 2 clusters at step 2,000 before scattering.

What the results teach. - **Match the fix to the cause.** Collapse happens because D can't see variety. **Minibatch std gives D exactly that information**, and it recovered all 8 clusters.

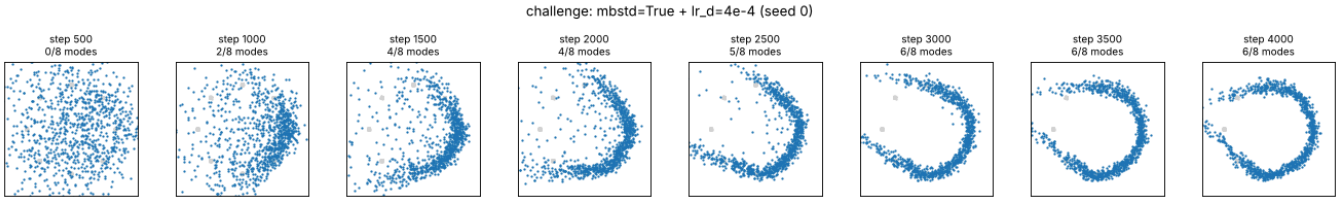
- **A fix aimed at a different problem won't help.** Label smoothing and instance noise are designed to calm an *over-confident* D. They did not fix collapse here, and label smoothing even made the generator worse (0/8).
- **Better quality isn't better variety.** TTUR produced the sharpest points (quality 0.90) but still covered only 2 clusters.
- **Setup matters too.** With a 2-number noise vector, G stretched over the whole ring. It leaves points on the "bridges" between clusters because G is a smooth function of its input: it cannot jump from one cluster to the next without passing through the gap.

Exercise 3 · Re-run the best setting with `seed=1`, then combine two fixes

One success can be luck. And the challenge: combine two fixes, for example `train_ring_gan(mbstd=True, lr_d=4e-4)`.

```
for name, kwargs in [("D · minibatch std, seed 1", dict(mbstd=True, seed=1)),
                     ("E · z_dim=2, seed 1", dict(z_dim=2, seed=1)),
                     ("challenge: mbstd + TTUR (seed 0)", dict(mbstd=True, lr_d=4e-4)),
                     ("challenge: mbstd + TTUR (seed 1)", dict(mbstd=True, lr_d=4e-4, seed=1))]:
    _, losses, snaps = train_ring_gan(**kwargs)
    results[name] = (losses, snaps)
    print(f"{name:34s} modes per snapshot {[s[2] for s in snaps]}  final quality {snaps[-1]
[3]:.2f}")
show_snapshots(results["challenge: mbstd + TTUR (seed 0)"][1], "challenge: mbstd=True + lr_d=4e-4
(seed 0)")
```

```
D · minibatch std, seed 1          modes per snapshot [1, 0, 0, 0, 3, 7, 8, 8]  final quality 0.4
7
E · z_dim=2, seed 1                modes per snapshot [2, 1, 2, 7, 8, 8, 8, 8]  final quality 0.6
7
challenge: mbstd + TTUR (seed 0)   modes per snapshot [0, 2, 4, 4, 5, 6, 6, 6]  final quality 0.3
4
challenge: mbstd + TTUR (seed 1)   modes per snapshot [0, 3, 4, 4, 5, 5, 7, 7]  final quality 0.3
3
```



Results (this run).

Run	modes per snapshot (every 500 steps)	final quality
D · minibatch std, **seed 1**	1, 0, 0, 0, 3, 7, 8, **8**	0.47
E · z_dim = 2, **seed 1**	2, 1, 2, 7, 8, 8, 8, **8**	0.67
challenge: mbstd + TTUR, seed 0	0, 2, 4, 4, 5, 6, 6, **6**	0.34
challenge: mbstd + TTUR, seed 1	0, 3, 4, 4, 5, 5, 7, **7**	0.33

- Both successful fixes **also work with seed 1** (8/8), so their success was not luck.
- **Combining two fixes is not automatically better:** minibatch std + TTUR climbed towards full coverage *steadily* (no long collapsed phase), but ended at 6/8 and 7/8 with lower quality than minibatch std alone, within the same 4,000 steps. A D that learns 4× faster may push G around too hard for it to settle on every cluster. There is no single right answer here (the guide says the same); the point is to test, not assume.

Part 5 · Diagnosing a real GAN on MNIST

Back to the **Class 1 MNIST GAN**: the same Generator (100 → 256 → 784, Tanh) and Discriminator (784 → 256 → 1, Sigmoid), same `BCELoss`, same Adam lr 0.0002. We add three diagnostic tools.

5.1 Loading MNIST (same as Class 1)

```
transform = transforms.Compose([transforms.ToTensor(), transforms.Normalize((0.5,), (0.5,))])
train_data = datasets.MNIST(root='./data', train=True, transform=transform, download=True)
train_loader = DataLoader(train_data, batch_size=64, shuffle=True)
print(len(train_data), "training digits,", len(train_loader), "batches of 64 per epoch")


class Generator(nn.Module):                # Class 1, unchanged
```

```
    def __init__(self):
        super().__init__()
        self.fc = nn.Sequential(nn.Linear(100, 256), nn.ReLU(), nn.Linear(256, 784), nn.Tanh())
    def forward(self, x):
        return self.fc(x).view(-1, 1, 28, 28)


class Discriminator(nn.Module):          # Class 1, unchanged
    def __init__(self):
        super().__init__()
        self.fc = nn.Sequential(nn.Linear(784, 256), nn.LeakyReLU(0.2), nn.Linear(256, 1),
nn.Sigmoid())
    def forward(self, x):
        return self.fc(x.view(-1, 784))
```

```
60000 training digits, 938 batches of 64 per epoch
```

5.2 Tool 1: a digit counter

An ordinary classifier trained for **one epoch**, used only as a measuring tool. `digit_histogram(G)` makes 1,000 fake digits, asks the classifier which digit each one is, and counts them. A healthy generator gives roughly 100 of each; a few very tall bars mean mode collapse. **Careful:** the classifier always answers *some* digit, even for noise, so read the counts together with the pictures.

```
torch.manual_seed(0)
classifier = nn.Sequential(nn.Flatten(), nn.Linear(784, 256), nn.ReLU(), nn.Linear(256, 10))
opt_c = optim.Adam(classifier.parameters(), lr=1e-3)
t0 = time.time()
for x, y in train_loader:
    opt_c.zero_grad(); nn.functional.cross_entropy(classifier(x), y).backward(); opt_c.step()
test_data = datasets.MNIST(root='./data', train=False, transform=transform, download=True)
with torch.no_grad():
    xt = (test_data.data.float() / 255 - 0.5) / 0.5            # same transform, applied to all
test images at once
    acc = (classifier(xt).argmax(1) == test_data.targets).float().mean().item()
print(f"digit counter: test accuracy {acc:.1%} after 1 epoch ({time.time() - t0:.0f} s)")


def digit_histogram(G, n=1000, seed=123):
    torch.manual_seed(seed)
    with torch.no_grad():
        digits = classifier(G(torch.randn(n, 100))).argmax(1)
    return torch.bincount(digits, minlength=10).numpy()
```

```
digit counter: test accuracy 93.8% after 1 epoch (10 s)
```

5.3 Tools 2 and 3: training with diagnostics

`train_mnist_gan(epochs=3, betas=..., real_label=...)` is the Class 1 loop plus logging. Every 100 steps it records the two losses, **mean D(x)** (D's average belief that real images are real, `out_real.mean()`) and **mean D(G(z))** (D's average belief that fakes are real, `out_fake.mean()`). After each epoch it draws the **same 16 fixed noise vectors**, so the rows of the grid show pure learning, not new random inputs.

```
def train_mnist_gan(epochs=3, betas=(0.9, 0.999), real_label=1.0, seed=0, log_every=100):
    torch.manual_seed(seed)
    G, D = Generator(), Discriminator()
    criterion = nn.BCELoss()
    opt_g = optim.Adam(G.parameters(), lr=0.0002, betas=betas)
    opt_d = optim.Adam(D.parameters(), lr=0.0002, betas=betas)
    fixed_noise = torch.randn(16, 100)
```

```
    log = {"step": [], "loss_d": [], "loss_g": [], "d_real": [], "d_fake": []}
    grids, step, t0 = [], 0, time.time()
    for epoch in range(epochs):
        for real, _ in train_loader:
            b = real.size(0)
            # Step 1: D  (real -> real_label, fake -> 0)
            opt_d.zero_grad()
            out_real = D(real)
            fake = G(torch.randn(b, 100))
            out_fake = D(fake.detach())
            loss_d = criterion(out_real, torch.full((b, 1), real_label)) + criterion(out_fake,
torch.zeros(b, 1))
            loss_d.backward(); opt_d.step()
            # Step 2: G  (non-saturating: fakes -> 1)
            opt_g.zero_grad()
            loss_g = criterion(D(fake), torch.ones(b, 1))
            loss_g.backward(); opt_g.step()
            if step % log_every == 0:
                log["step"].append(step); log["loss_d"].append(loss_d.item());
log["loss_g"].append(loss_g.item())
                log["d_real"].append(out_real.mean().item());
log["d_fake"].append(out_fake.mean().item())
            step += 1
        with torch.no_grad():
            grids.append(G(fixed_noise))
        print(f"epoch {epoch + 1}: loss D {loss_d.item():.3f}, loss G {loss_g.item():.3f}, "
              f"mean D(x) {out_real.mean().item():.3f}, mean D(G(z)) {out_fake.mean().item():.3f}
({time.time() - t0:.0f} s)")
    return G, log, grids


def show_diagnostics(G, log, grids, title):
    fig, ax = plt.subplots(1, 3, figsize=(16, 3.6))
    ax[0].plot(log["step"], log["loss_d"], label="loss D"); ax[0].plot(log["step"],
log["loss_g"], label="loss G")
    ax[0].axhline(1.386, ls="--", c="k", lw=1, label="1.386"); ax[0].axhline(0.693, ls=":",
c="k", lw=1, label="0.693")
    ax[0].set_title("losses"); ax[0].set_xlabel("step"); ax[0].legend(fontsize=8)
    ax[1].plot(log["step"], log["d_real"], label="mean D(x)"); ax[1].plot(log["step"],
log["d_fake"], label="mean D(G(z))")
    ax[1].axhline(0.5, ls=":", c="k", lw=1); ax[1].set_ylim(0, 1); ax[1].set_title("who is
winning?"); ax[1].set_xlabel("step"); ax[1].legend(fontsize=8)
    counts = digit_histogram(G)
    ax[2].bar(range(10), counts); ax[2].axhline(100, ls="--", c="k", lw=1);
ax[2].set_xticks(range(10))
    ax[2].set_title("which digits? (1,000 samples)"); ax[2].set_xlabel("digit")
    plt.suptitle(title, y=1.03); plt.tight_layout(); plt.show()
    fig, axes = plt.subplots(len(grids), 1, figsize=(16, 1.25 * len(grids)))
    for e, (a, g) in enumerate(zip(np.atleast_1d(axes), grids)):
        a.imshow(make_grid(g, nrow=16, normalize=True, value_range=(-1, 1),
pad_value=1).permute(1, 2, 0)); a.axis("off")
        a.set_title(f"{title}: 16 fixed noise vectors after epoch {e + 1}", fontsize=9,
loc="left")
    plt.tight_layout(); plt.show()
    print(f"{title}: digit counts 0..9 = {counts.tolist()}")
    return counts
```

How to read the three plots

Plot	What to look for
Losses	compare with the dashed line (1.386) and dotted line (0.693), the values when D outputs 0.5. Wobbling is normal; drifting far away and staying there can mean one player is winning completely
Who is winning?	D(x) near 1 and D(G(z)) near 0: D wins easily and G gets little signal. Both near 0.5: D is confused, which is good *only if* the images also look good

Plot	What to look for
Which digits?	even bars mean variety; a few tall bars mean mode collapse

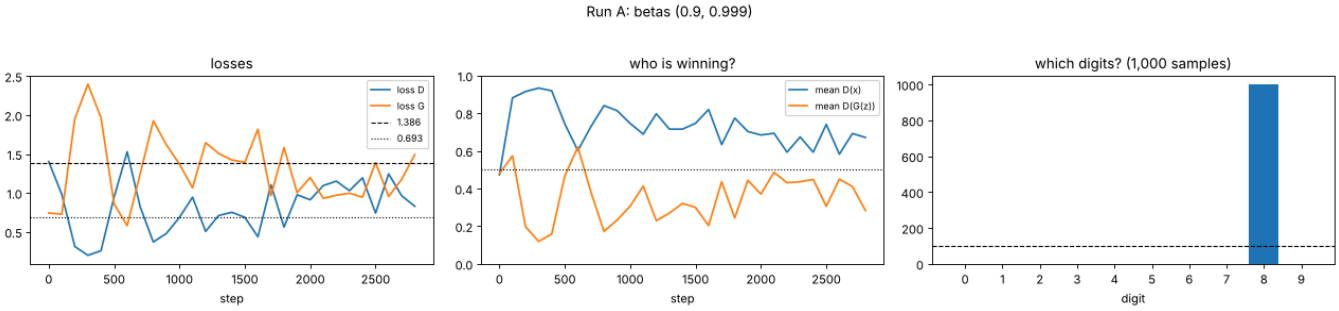
Run A vs Run B

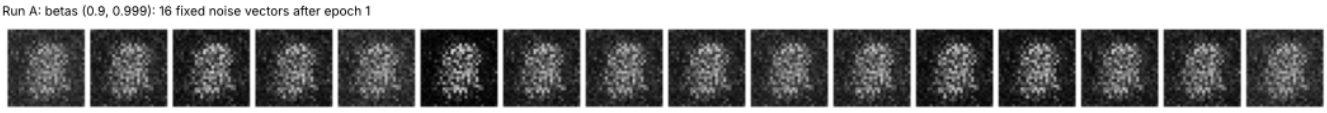
- **Run A:** the Class 1 settings: Adam `betas=(0.9, 0.999)` (the PyTorch default).
- **Run B:** `betas=(0.5, 0.999)` (DCGAN paper) **plus** one-sided label smoothing `real_label=0.9`. Lower β1 = less momentum, so the optimiser reacts faster to its changing opponent instead of overshooting.

> ✏️ **Predict:** which run will look healthier after 3 epochs? Could you tell from the losses alone?

```
G_runA, log_A, grids_A = train_mnist_gan(epochs=3, betas=(0.9, 0.999), real_label=1.0)
counts_A = show_diagnostics(G_runA, log_A, grids_A, "Run A: betas (0.9, 0.999)")
```

```
epoch 1: loss D 0.730, loss G 1.249, mean D(x) 0.758, mean D(G(z)) 0.345  (12 s)
epoch 2: loss D 1.040, loss G 1.017, mean D(x) 0.673, mean D(G(z)) 0.447  (24 s)
epoch 3: loss D 0.713, loss G 1.405, mean D(x) 0.756, mean D(G(z)) 0.315  (36 s)
```





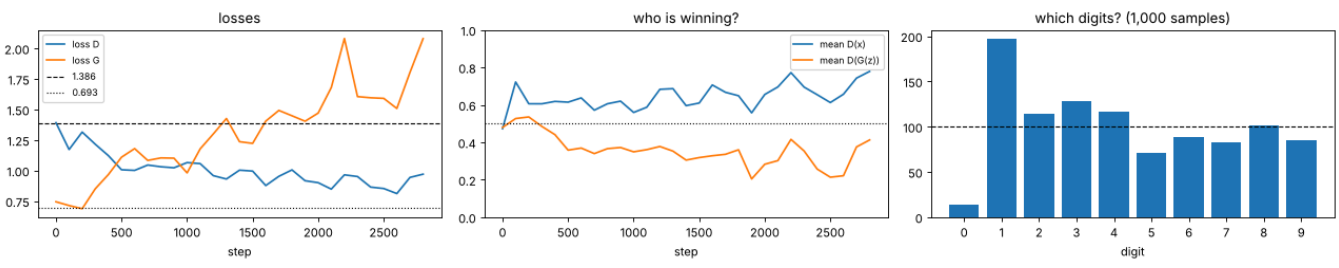


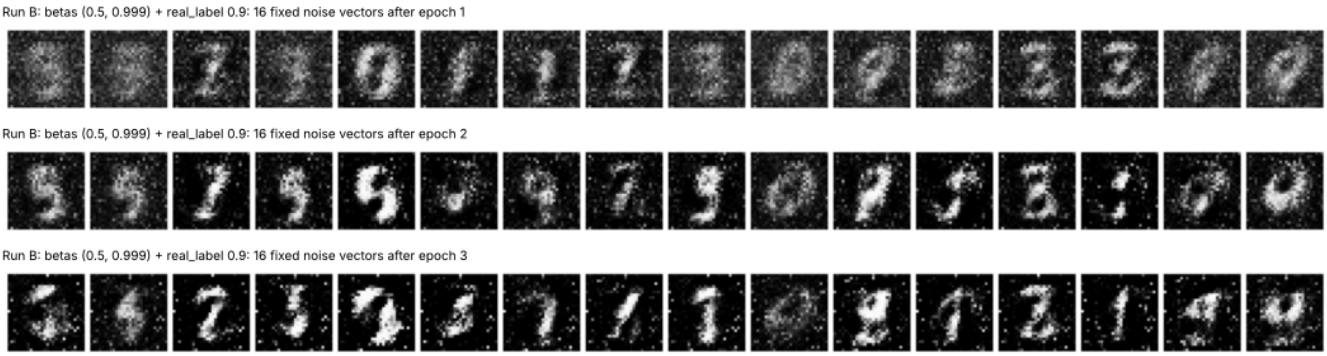



```
Run A: betas (0.9, 0.999): digit counts 0..9 = [0, 0, 0, 0, 0, 0, 0, 0, 1000, 0]
```

```
G_runB, log_B, grids_B = train_mnist_gan(epochs=3, betas=(0.5, 0.999), real_label=0.9)
counts_B = show_diagnostics(G_runB, log_B, grids_B, "Run B: betas (0.5, 0.999) + real_label 0.9")
```

```
epoch 1: loss D 1.051, loss G 1.154, mean D(x) 0.626, mean D(G(z)) 0.387  (12 s)
epoch 2: loss D 0.950, loss G 1.619, mean D(x) 0.660, mean D(G(z)) 0.327  (28 s)
epoch 3: loss D 0.975, loss G 1.302, mean D(x) 0.506, mean D(G(z)) 0.134  (44 s)
```






```
Run B: betas (0.5, 0.999) + real_label 0.9: digit counts 0..9 = [14, 197, 114, 128, 117, 71, 89,
83, 102, 85]
```

```python
for name, log, counts in [("Run A", log_A, counts_A), ("Run B", log_B, counts_B)]:
    last = slice(-10, None)        # the last 10 logged points (last ~1,000 steps)
    print(f"{name}: last 1,000 steps: mean loss D {np.mean(log['loss_d'][last]):.3f}, mean loss G
{np.mean(log['loss_g'][last]):.3f}, "
          f"mean D(x) {np.mean(log['d_real'][last]):.3f}, mean D(G(z)) {np.mean(log['d_fake']
[last]):.3f} | "
          f"digit counts min {counts.min()}, max {counts.max()}")
```

```
Run A: last 1,000 steps: mean loss D 1.013, mean loss G 1.105, mean D(x) 0.663, mean D(G(z)) 0.40
7 | digit counts min 0, max 1000
Run B: last 1,000 steps: mean loss D 0.903, mean loss G 1.682, mean D(x) 0.682, mean D(G(z)) 0.30
4 | digit counts min 14, max 197
```

Answer revealed (this run).		Run A: β1 = 0.9 (Class 1 settings)	Run B: β1 = 0.5 + real_label 0.9
	grids	**collapsed**: after every epoch all 16 fixed-noise images are the **same** speckled blob	recognisable, **varied** digits (2, 3, 1, 7, 9 ...), improving from epoch 1 to 3
	histogram (1,000 samples)	**all 1,000 labelled "8"**	all ten digits, from **14** (digit 0) to **197** (digit 1)
	who is winning?	D dominates early (D(x) ≈ 0.93, D(G(z)) ≈ 0.12 around step 300), a big swing near step 600, then zigzags; last 1,000 steps D(x) 0.66, D(G(z)) 0.41	steadier: D(x) about 0.6–0.75, D(G(z)) slowly falling from 0.5 to about 0.2–0.4; last 1,000 steps D(x) 0.68, D(G(z)) 0.30
	losses (last 1,000 steps)	D 1.01, G 1.11	D 0.90, G 1.68

This reproduces the guide's test run almost exactly: there too, Run A collapsed completely and "the digit counter labelled all 1,000 samples as 8", while Run B produced varied digits with counts spread over all ten digits (lowest about 30, highest about 160; ours: 14 and 197). The slides' description of Run A's D plot ("D dominates early, a big swing near step 600, then zigzags") matches ours too.

Exercise 4 · model answers.

1. *Which run is healthier, and what evidence?* **Run B**, clearly. Evidence: the grids (real, different digits vs one repeated blob), the histogram (ten digits vs one) and the D plot (steadier contest vs early domination and big swings).
2. *Could you have told the better run from the losses alone?* **No.** In the last 1,000 steps Run A's numbers actually look *more* "balanced" (losses 1.01 / 1.11, closer to 1.386 / 0.693; D(G(z))

0.41, closer to 0.5) than Run B's (0.90 / 1.68; D(G(z)) 0.30). As the slides say, after step 1,000 A's numbers look "healthy" too. Yet A is the completely collapsed run. **The losses alone would have picked the wrong run.** That is the main lesson of the day. And the trap: Run A's histogram says "1,000 eights", but the grid shows no eights at all, only junk; the counter always answers *some* digit.

3. *A trap to notice:* the digit counter **always** answers some digit, even for noise. Read the histogram **together with** the grid.

Part 6 · Wrap-up and reflection questions (model answers)

Notebook part	What it demonstrated (this run)
1 · x·y game	gradient play circles a Nash equilibrium; a smaller lr only slows it
3 · ring baseline	mode collapse and hopping, invisible in the loss curve
4 · five fixes	fixes work only on the problem they target; check with another seed
5 · MNIST diagnostics	D scores, fixed-noise grids and digit counts show what losses hide

1. **Explain Nash equilibrium using the x·y game.** At (0, 0), if y stays at 0, nothing x does changes V, so x can't gain by moving; the same holds for y when x is 0. Neither player can improve by changing alone, so (0, 0) is a Nash equilibrium. (For a GAN: p_g = p_data and D = ½ everywhere.)
2. **Why did simultaneous updates spiral out while alternating updates circled?** Simultaneous updates use the old values, so every step pushes the pair slightly further out, by a factor $\sqrt{1 + lr^2}$ (we measured 1.41 → 3.83 in 200 steps). With alternating updates the second player reacts to the first player's *new* position, which exactly cancels that growth, so the path stays on a loop. Neither aims at the equilibrium, because each gradient depends on the other player.
3. **Why can't a one-sample discriminator easily detect mode collapse?** It scores each sample on its own. A single realistic sample looks real whether or not every other sample is the same, so the loss gives no penalty for missing variety. (Minibatch std fixes exactly this by showing D a batch statistic.)
4. **Which diagnostic would you check first for mode collapse?** A grid of many samples, or a class histogram if a classifier is available. Losses and D scores can look normal during collapse; only looking at many outputs together shows the missing variety.

Exit ticket (last slide) with answers

1. *Define Nash equilibrium. What is the GAN's equilibrium?* Nobody gains by changing alone; p_g = p_data and D = ½.
2. *Losses look calm, but 900 of 1,000 samples are the digit 1.* Mode collapse; minibatch std / minibatch discrimination.
3. *Mean D(x) ≈ 1.0 and D(G(z)) ≈ 0.0 for a long time.* D is too strong; G's gradients vanish and G stops learning.

Common questions

- **If the equilibrium exists, why doesn't training find it?** The existence proof assumes unlimited capacity and a perfectly trained D at every step. Real training takes small alternating gradient steps with finite networks, and as Part 1 shows, those steps tend to circle the equilibrium.
- **Does a lower loss mean a better GAN?** No. Each loss depends on both players; G's loss can rise because D improved, even if G didn't change.
- **Label smoothing didn't fix collapse. Is it a bad trick?** No, it targets a different problem (an over-confident D). Choose the fix by the problem you see.
- **What is BCEWithLogitsLoss?** Sigmoid + BCE in one step: same result, no numerical problems with very confident scores. Part 5 keeps Class 1's Sigmoid + `BCELoss` so the code matches.
- **What is WGAN (mentioned in the slides)?** A redesigned GAN whose "critic" estimates a distance between the real and fake distributions instead of a probability, so it keeps giving useful gradients. Not part of this notebook, and not a guaranteed fix.

Next: `03_dcgan_and_conditional_gan.ipynb` (Class 4: convolutional G and D, and telling G which digit to draw).

Source: reconstructed from the course guide "GAN Class 2 & 3 Notebook Explained" (GAN-Class-2-3-Training-Dynamics.ipynb) and the Class 2 & 3 slides. Papers: Goodfellow et al. 2014; Radford et al. 2015 (DCGAN); Salimans et al. 2016 (label smoothing, minibatch discrimination); Arjovsky and Bottou 2017 (instance noise); Heusel et al. 2017 (TTUR); Karras et al. 2018 (minibatch std).

03 · DCGAN and conditional GAN

The Class 4 notebook (GAN-Class-4-DCGAN-cGAN.ipynb), rebuilt and run

Covers (GAN Class 4 slides): revision of Classes 2 & 3 · the three diagnostic tools · why convolutions · Conv2d / ConvTranspose2d size formulas · the DCGAN recipe · conditional GAN (cGAN): give the label to **both** players · conditional accuracy · content vs style · the assignment.

Where the code comes from. The course guide *"GAN Class 4 Notebook Explained"* describes this notebook part by part, with its function names (`classify` , `train_gan` , `show_grid` , `digit_histogram` , `show_diagnostics` , `conv_out` , `convT_out` , `init_weights` , `CondDCGenerator` , `CondDCDiscriminator` , `conditional_accuracy` , `style_vs_content` , `interpolate`), architectures and defaults. This notebook follows it exactly. It also finishes the two "later" items from the Class 1 notebook: the **DCGAN generator skeleton** (cells 6–7, the CelebA mini-project) and the **conditional-GAN assignment** (cell 8).

The guide quotes numbers from the course's own test run (seed 0, a 2-core CPU). **The numbers in this notebook come from this run**; where they differ from the guide's, both are shown. Differences are expected: random seeds, library versions and hardware all change GAN results. Compare patterns, not exact numbers.

> **The training game does not change today.** Same losses, same alternating D step and G step, same Adam settings (lr 0.0002, β1 = 0.5) that Classes 2 & 3 showed to be more stable. We change **how G and D see images** (convolutions) and **what G is told** (the label).

Part	What you do	Course CPU time (guide)
0	imports, device, load MNIST (batch 128)	seconds
1	revision quiz; CNN digit counter; `train_gan()` ; Class 1 baseline	29 s + 18 s
2	size formulas; the 2×2 → 4×4 stamp demo; Exercise 2	instant
3	build, initialise and train the DCGAN; compare with the baseline; Exercise 3	153 s
4	conditional DCGAN; conditional accuracy; style vs content; interpolation; Exercise 4	199 s
5	wrap-up, reflection questions, assignment (option C done for real)	—

Cells marked ✏️ **Predict** ask for a guess before the next cell runs.

Words you need

Term	Plain meaning
DCGAN	Deep Convolutional GAN: G and D made of convolutions (Radford, Metz and Chintala 2015)
Convolution (Conv2d)	slides one small filter over the image; the same filter is reused everywhere
Stride / padding	how many pixels the filter moves each step (stride 2 halves the size) / extra border pixels added first
Transposed convolution (ConvTranspose2d)	the reverse direction: each input pixel *stamps* a copy of the filter onto a bigger output; stride 2 doubles the size
Channel	one "layer" of an image or feature map (MNIST has 1; a feature map can have 64 or 128)
BatchNorm	rescales each channel using the batch's mean and spread, to keep training stable
Locality / weight sharing	each output depends on a small neighbourhood / the same filter weights are used at every position
Conditional GAN (cGAN)	G and D both receive extra information y, here the digit label (Mirza and Osindero 2014)
One-hot vector	a label as ten numbers, all 0 except a 1 at the label: 7 → [0,0,0,0,0,0,0,1,0,0]
Conditional accuracy	the share of generated images that a classifier reads as the digit we asked for
Content vs style	in a cGAN the label decides *which* digit (content); the noise decides *how it is written* (style)
Evenness	a 0–1 score for the digit histogram; 1.0 = all ten digits equally often

Part 0 · Setup

```
%matplotlib inline
import time
import math
import numpy as np
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torchvision import datasets, transforms
from torchvision.utils import make_grid
from torch.utils.data import DataLoader

device = "cuda" if torch.cuda.is_available() else "cpu"
torch.manual_seed(0)
np.random.seed(0)
plt.rcParams["figure.dpi"] = 80
Z_DIM = 100
print("torch", torch.__version__, "| device:", device)

transform = transforms.Compose([transforms.ToTensor(), transforms.Normalize((0.5,), (0.5,))])
train_data = datasets.MNIST(root='./data', train=True, transform=transform, download=True)
test_data = datasets.MNIST(root='./data', train=False, transform=transform, download=True)
train_loader = DataLoader(train_data, batch_size=128, shuffle=True, drop_last=True)
print(len(train_data), "training digits ->", len(train_loader), "batches of 128 per epoch
(drop_last drops the last 96)")
```

```
torch 2.14.1+cu130 | device: cpu
60000 training digits -> 468 batches of 128 per epoch (drop_last drops the last 96)
```

Two changes from Class 1's loader: **batch 128** (fewer, bigger steps; faster on a CPU) and `drop_last=True`: 60,000 = 468 × 128 + 96, and the last small batch of 96 is skipped. Every batch then has exactly 128 images, which keeps BatchNorm's statistics (Part 3) comparable from batch to batch and the shape checks simple.

Part 1 · Revision and the diagnostic toolkit

Revision quiz: answer from memory first, then open the answers

1. What is a GAN's Nash equilibrium? What does the best D output there?
2. Why did gradient steps circle the x·y equilibrium?
3. Losses calm, but 900 of 1,000 samples are "1". What is it?
4. Mean D(x) ≈ 1.0, D(G(z)) ≈ 0.0 for a long time. What happens to G?
5. Name the three diagnostics. Which one alone can fool you?

▶ **Model answers (click)**

Tool 1 · The CNN digit counter

Classes 2 & 3 used a quick fully connected classifier. Today we need a **more accurate** one, because Part 4 uses it to check whether G drew the digit we *asked* for. A small CNN: two 3×3 conv layers with max-pooling, then two linear layers, trained for one epoch.

```
torch.manual_seed(0)
counter = nn.Sequential(
    nn.Conv2d(1, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),     # 28 -> 14
    nn.Conv2d(32, 64, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),    # 14 -> 7
    nn.Flatten(), nn.Linear(64 * 7 * 7, 128), nn.ReLU(), nn.Linear(128, 10)).to(device)
opt_c = optim.Adam(counter.parameters(), lr=1e-3)
t0 = time.time()
for x, y in train_loader:                                       # one epoch
    x, y = x.to(device), y.to(device)
    opt_c.zero_grad(); F.cross_entropy(counter(x), y).backward(); opt_c.step()
counter.eval()
with torch.no_grad():
    xt = ((test_data.data.float() / 255 - 0.5) / 0.5).unsqueeze(1).to(device)
    test_acc = (counter(xt).argmax(1).cpu() == test_data.targets).float().mean().item()
print(f"CNN digit counter: {test_acc:.1%} on the 10,000 test digits (guide: 98.1%), trained in
{time.time() - t0:.0f} s")


def classify(images):
    """Predicted digit (0-9) for a batch of images in the -1..1 range."""
    with torch.no_grad():
        return counter(images.to(device)).argmax(1).cpu()
```

```
CNN digit counter: 98.1% on the 10,000 test digits (guide: 98.1%), trained in 55 s
```

Tools 2 and 3 · One training function for every model: `train_gan()`

It is the Class 1 / Class 2 & 3 loop with these settings and additions:

Code	What it does
`nn.BCEWithLogitsLoss()`	Sigmoid + BCE in one stable step, so the discriminators end with a **raw score** (no Sigmoid layer)
`fixed_z = torch.randn(100, Z_DIM)`	100 noise vectors reused after every epoch, so grids are comparable
`fixed_y = torch.arange(10).repeat_interleave(10)`	labels 0,0,...,9,9 for the grid; used only by a cGAN, so **row r asks for digit r**
`fake = G(z, y) if conditional else G(z)`	the `# cGAN` lines: pass the real batch's labels to G and D when `conditional=True`
`loss_d = bce(d_real, ones) + bce(d_fake, zeros)`	D's step: real → 1, fake → 0; `fake.detach()` keeps G frozen
`loss_g = bce(D(fake), ones)`	G's step: the non-saturating loss from Class 1
`torch.sigmoid(d_real).mean()`	Tool 2: D's average opinion, logged every 50 steps

```
def train_gan(G, D, epochs=2, conditional=False, lr=2e-4, betas=(0.5, 0.999), log_every=50):
    G.to(device); D.to(device)
    bce = nn.BCEWithLogitsLoss()
    opt_g = optim.Adam(G.parameters(), lr=lr, betas=betas)
    opt_d = optim.Adam(D.parameters(), lr=lr, betas=betas)
    fixed_z = torch.randn(100, Z_DIM, generator=torch.Generator().manual_seed(0)).to(device)   #
same 100 vectors for every model
    fixed_y = torch.arange(10).repeat_interleave(10).to(device)                                #
0,0,...,0,1,...,9
    log = {"step": [], "loss_d": [], "loss_g": [], "d_x": [], "d_gz": [], "grids": []}
    step, t0 = 0, time.time()
    for epoch in range(epochs):
        for real, labels in train_loader:
            real, labels = real.to(device), labels.to(device)
            b = real.size(0)
            ones, zeros = torch.ones(b, 1, device=device), torch.zeros(b, 1, device=device)
            z = torch.randn(b, Z_DIM, device=device)
            fake = G(z, labels) if conditional else G(z)                                   # cGAN: G
is told the label
            # Step 1: train D (real -> 1, fake -> 0)
            d_real = D(real, labels) if conditional else D(real)                           # cGAN: D
sees the label
            d_fake = D(fake.detach(), labels) if conditional else D(fake.detach())         # cGAN
            loss_d = bce(d_real, ones) + bce(d_fake, zeros)
            opt_d.zero_grad(); loss_d.backward(); opt_d.step()
            # Step 2: train G (non-saturating: fake -> 1)
            d_out = D(fake, labels) if conditional else D(fake)                            # cGAN
            loss_g = bce(d_out, ones)
            opt_g.zero_grad(); loss_g.backward(); opt_g.step()
            if step % log_every == 0:
                log["step"].append(step); log["loss_d"].append(loss_d.item());
log["loss_g"].append(loss_g.item())
                log["d_x"].append(torch.sigmoid(d_real).mean().item());
log["d_gz"].append(torch.sigmoid(d_fake).mean().item())
            step += 1
        log["grids"].append(generate(G, fixed_z, fixed_y if conditional else None))
        print(f"epoch {epoch + 1}/{epochs}: loss D {loss_d.item():.3f}, loss G
{loss_g.item():.3f}  ({time.time() - t0:.0f} s)")
    log["seconds"] = time.time() - t0
    return log


def generate(G, z, y=None):
    """Make images without training: eval mode (BatchNorm uses its running averages), no
```

```
gradients."""
    G.eval()
    with torch.no_grad():
        out = (G(z, y) if y is not None else G(z)).cpu()
    G.train()
    return out


def show_grid(images, nrow=10, title="", ax=None, size=6):
    if ax is None:
        _, ax = plt.subplots(figsize=(size, size * math.ceil(len(images) / nrow) / nrow))
    ax.imshow(make_grid(images, nrow=nrow, normalize=True, value_range=(-1, 1),
pad_value=1).permute(1, 2, 0))
    ax.axis("off"); ax.set_title(title, fontsize=10)


def evenness(counts):
    """Normalised entropy of the digit counts: 1.0 = all ten digits equally often, 0 = only one
digit."""
    p = np.asarray(counts, float) / np.sum(counts)
    p = p[p > 0]
    return max(0.0, float(-(p * np.log(p)).sum() / np.log(10)))


def digit_histogram(G, n=1000, seed=1):
    z = torch.randn(n, Z_DIM, generator=torch.Generator().manual_seed(seed)).to(device)
    return np.bincount(classify(generate(G, z)).numpy(), minlength=10)


def show_diagnostics(G, log, title):
    counts = digit_histogram(G)
    fig, ax = plt.subplots(1, 3, figsize=(16, 3.5))
    ax[0].plot(log["step"], log["loss_d"], label="loss D"); ax[0].plot(log["step"],
log["loss_g"], label="loss G")
    ax[0].axhline(1.386, ls="--", c="k", lw=1); ax[0].axhline(0.693, ls=":", c="k", lw=1)
    ax[0].set_title("losses (dashed 1.386, dotted 0.693)"); ax[0].set_xlabel("step");
ax[0].legend(fontsize=8)
    ax[1].plot(log["step"], log["d_x"], label="mean D(x)"); ax[1].plot(log["step"], log["d_gz"],
label="mean D(G(z))")
    ax[1].axhline(0.5, ls=":", c="k", lw=1); ax[1].set_ylim(0, 1); ax[1].set_title("who is
winning?"); ax[1].set_xlabel("step"); ax[1].legend(fontsize=8)
    ax[2].bar(range(10), counts); ax[2].axhline(100, ls="--", c="k", lw=1);
ax[2].set_xticks(range(10))
    ax[2].set_title(f"which digits? evenness {evenness(counts):.3f}"); ax[2].set_xlabel("digit
(CNN counter)")
    plt.suptitle(title, y=1.03); plt.tight_layout(); plt.show()
    half = len(log["d_x"]) // 2
    print(f"{title}: counts {dict(enumerate(counts.tolist()))}")
    print(f"   evenness {evenness(counts):.3f} | rarest digit {counts.argmin()} ({counts.min()}
of 1,000) | "
          f"second-epoch mean D(x) {np.mean(log['d_x'][half:]):.2f}, mean D(G(z))
{np.mean(log['d_gz'][half:]):.2f}")
    return counts
```

Look for the `# cGAN` lines in `train_gan()`. They do nothing yet (`conditional=False`); in Part 4 they pass the label to G and D.

`evenness` is the **normalised entropy** of the ten digit counts: $-\Sigma\, p \cdot \ln p$ divided by $\ln 10$. It is 1.0 when every digit has 100 of the 1,000 samples and 0 when all 1,000 are the same digit. (The guide gives only the 0–1 meaning; this formula reproduces the guide's quoted values exactly: its MLP counts give 0.786 and its DCGAN counts 0.929.)

1.3 · The baseline: the Class 1 networks, β1 = 0.5, 2 epochs

The Class 1 Generator (100 → 256 → 784, Tanh) and Discriminator (784 → 256 → 1). The only change to D: **no Sigmoid** at the end, because `BCEWithLogitsLoss` applies it.

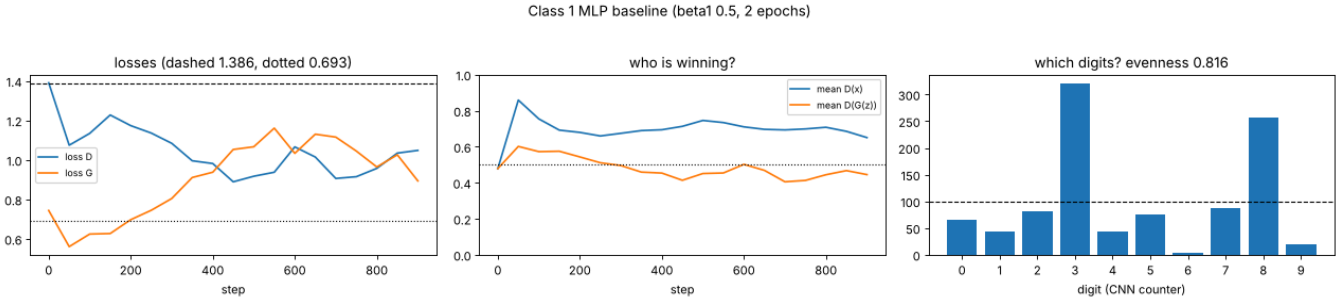
```
class MLPGenerator(nn.Module):                    # Class 1 Generator
    def __init__(self):
        super().__init__()
        self.fc = nn.Sequential(nn.Linear(100, 256), nn.ReLU(), nn.Linear(256, 784), nn.Tanh())
    def forward(self, z):
        return self.fc(z).view(-1, 1, 28, 28)


class MLPDiscriminator(nn.Module):                # Class 1 Discriminator, raw score out
    def __init__(self):
        super().__init__()
        self.fc = nn.Sequential(nn.Linear(784, 256), nn.LeakyReLU(0.2), nn.Linear(256, 1))
    def forward(self, x):
        return self.fc(x.view(-1, 784))

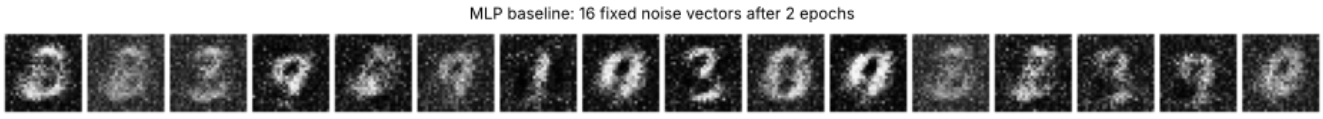

def n_params(m):
    return sum(p.numel() for p in m.parameters())

torch.manual_seed(0)
G_mlp, D_mlp = MLPGenerator(), MLPDiscriminator()
print(f"parameters: G {n_params(G_mlp):,}  D {n_params(D_mlp):,}")
log_mlp = train_gan(G_mlp, D_mlp, epochs=2)
counts_mlp = show_diagnostics(G_mlp, log_mlp, "Class 1 MLP baseline (beta1 0.5, 2 epochs)")
show_grid(log_mlp["grids"][-1][:16], nrow=16, title="MLP baseline: 16 fixed noise vectors after 2
epochs", size=16)
```

```
parameters: G 227,344  D 201,217
epoch 1/2: loss D 0.868, loss G 1.081  (10 s)
epoch 2/2: loss D 1.025, loss G 0.906  (19 s)
```




```
Class 1 MLP baseline (beta1 0.5, 2 epochs): counts {0: 66, 1: 44, 2: 82, 3: 321, 4: 43, 5: 76, 6:
4, 7: 87, 8: 257, 9: 20}
   evenness 0.816 | rarest digit 6 (4 of 1,000) | second-epoch mean D(x) 0.70, mean D(G(z)) 0.45
```



Exercise 1 · Diagnose the baseline (quality, variety, balance, edges). Guide's run: *grey, speckled, blurry digits; counts {0: 77, 1: 39, 2: 87, 3: 225, 4: 17, 5: 67, 6: 0, 7: 104, 8: 362, 9: 22}; mean D(x) about 0.7, D(G(z)) about 0.4.*

This run: counts {0: 66, 1: 44, 2: 82, 3: 321, 4: 43, 5: 76, 6: 4, 7: 87, 8: 257, 9: 20}, **evenness 0.816** (guide 0.786), second-epoch mean D(x) 0.70 and D(G(z)) 0.45. The same picture as the guide's run:

- **Quality:** blurry strokes on a **speckled grey background**; several samples are only half-formed.

- **Variety: partial collapse**: 3s (321) and 8s (257) dominate and there are almost no 6s (4 of 1,000; the guide's run had none) or 9s (20). All digits exist, but very unevenly.
- **Balance:** a live contest, nobody winning easily (D(x) ≈ 0.7, D(G(z)) ≈ 0.45, close to the guide's 0.7 / 0.4).
- **Edges:** grey noise right to the image border, where real digits are pure black, because each of the 784 outputs is learned separately: no pixel knows its neighbours are black.

Part 2 · Convolution shapes

Two formulas give every layer's output size, for a square input of size n, kernel k, stride s and padding p:

Layer	Output size	With k = 4, s = 2, p = 1
`Conv2d`	**(n + 2p − k) // s + 1**	halves: 28 → 14 → 7
`ConvTranspose2d`	**(n − 1)·s − 2p + k**	doubles: 7 → 14 → 28

> ✏️ **Predict:** `ConvTranspose2d(100, 128, kernel 7, stride 1, padding 0)` on a 1×1 input gives what size?

```
def conv_out(n, k, s, p):
    return (n + 2 * p - k) // s + 1

def convT_out(n, k, s, p):
    return (n - 1) * s - 2 * p + k

print("D (Conv2d k4 s2 p1):          28 ->", conv_out(28, 4, 2, 1), "->", conv_out(conv_out(28,
4, 2, 1), 4, 2, 1))
print("D last layer Conv2d(k7 s1 p0): 7 ->", conv_out(7, 7, 1, 0))
print("G first layer ConvT(k7 s1 p0): 1 ->", convT_out(1, 7, 1, 0))
print("G (ConvT k4 s2 p1):           7 ->", convT_out(7, 4, 2, 1), "->", convT_out(convT_out(7,
4, 2, 1), 4, 2, 1))

# check two of them with real PyTorch layers
z = torch.randn(1, 100, 1, 1)
h = nn.ConvTranspose2d(100, 128, 7, 1, 0)(z);  print("real layer:", tuple(h.shape))
h = nn.ConvTranspose2d(128, 64, 4, 2, 1)(h);   print("real layer:", tuple(h.shape))
```

```
D (Conv2d k4 s2 p1):          28 -> 14 -> 7
D last layer Conv2d(k7 s1 p0): 7 -> 1
G first layer ConvT(k7 s1 p0): 1 -> 7
G (ConvT k4 s2 p1):           7 -> 14 -> 28
real layer: (1, 128, 7, 7)
real layer: (1, 64, 14, 14)
```

Answer: (1 − 1)·1 − 0 + 7 = **7**. That is how G turns a noise vector into a 7×7 feature map.

The stamp demo: what a transposed convolution does

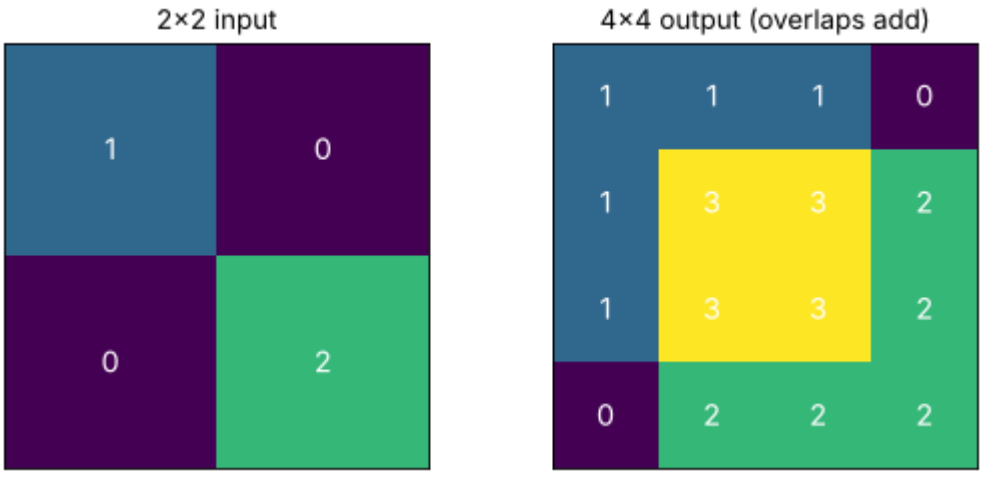
A `ConvTranspose2d(1, 1, 4, 2, 1)` with **every weight set to 1** is applied to the 2×2 input [[1, 0], [0, 2]].

> ✏️ **Predict** the 4×4 output before running. Hint: each input pixel stamps a 4×4 block of its own value, two output pixels apart; padding trims a 1-pixel border; overlaps **add**.

```
stamp = nn.ConvTranspose2d(1, 1, 4, 2, 1, bias=False)
with torch.no_grad():
    stamp.weight.fill_(1.0)
x = torch.tensor([[[[1., 0.], [0., 2.]]]])
out = stamp(x).detach()[0, 0]
print("input:\n", x[0, 0].numpy())
print("output:\n", out.numpy())

fig, ax = plt.subplots(1, 2, figsize=(7, 3.2))
for a, m, t in [(ax[0], x[0, 0], "2x2 input"), (ax[1], out, "4x4 output (overlaps add)")]:
    a.imshow(m, cmap="viridis", vmin=0, vmax=3)
    for (i, j), v in np.ndenumerate(m.numpy()):
        a.text(j, i, f"{v:.0f}", ha="center", va="center", color="w", fontsize=13)
    a.set_title(t); a.set_xticks([]); a.set_yticks([])
plt.tight_layout(); plt.show()
```

```
input:
 [[1. 0.]
 [0. 2.]]
output:
 [[1. 1. 1. 0.]
 [1. 3. 3. 2.]
 [1. 3. 3. 2.]
 [0. 2. 2. 2.]]
```



Output **[[1, 1, 1, 0], [1, 3, 3, 2], [1, 3, 3, 2], [0, 2, 2, 2]]**, as in the guide. The top-left pixel (1) stamps 1s, the bottom-right pixel (2) stamps 2s two pixels further along, and **the centre is 1 + 2 = 3** where they overlap. In a real generator the kernels are **learned**, so each feature stamps a learned patch of strokes.

Exercise 2

(a) Trace the Class 1 CelebA skeleton. The Class 1 notebook (cells 6–7) gave only this generator for the DCGAN face mini-project:

```
class DCGANGenerator(nn.Module):          # Class 1 notebook, cell 7 (as quoted in the Class 1 guide)
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.ConvTranspose2d(100, 512, 4, 1, 0, bias=False),
            nn.BatchNorm2d(512),
            nn.ReLU(True),
            nn.ConvTranspose2d(512, 256, 4, 2, 1, bias=False),
            nn.BatchNorm2d(256),
            nn.ReLU(True),
            nn.ConvTranspose2d(256, 128, 4, 2, 1, bias=False),
            nn.BatchNorm2d(128),
            nn.ReLU(True),
            nn.ConvTranspose2d(128, 3, 4, 2, 1, bias=False),
            nn.Tanh()
        )
    def forward(self, x):
        return self.net(x)

h = torch.randn(2, 100, 1, 1)           # the noise must be shaped (batch, 100, 1, 1)
print("input", tuple(h.shape))
for layer in DCGANGenerator().net:
    h = layer(h)
    if isinstance(layer, nn.ConvTranspose2d):
        print(f"{str(layer):62s} -> {tuple(h.shape)}")
```

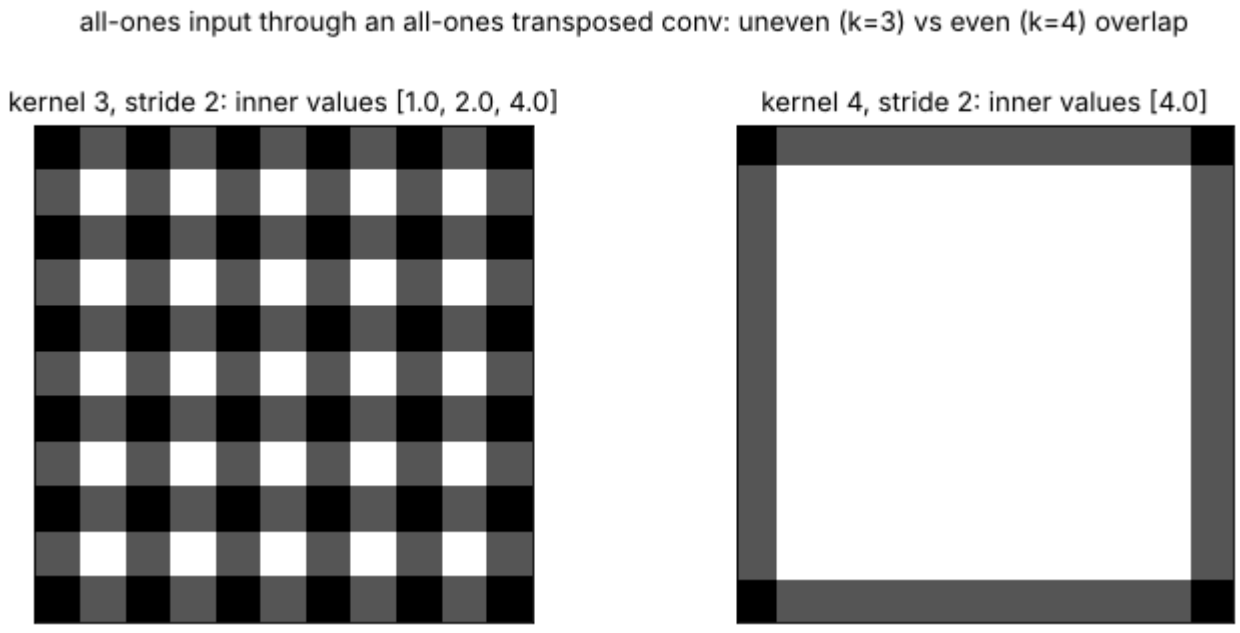
```
input (2, 100, 1, 1)
ConvTranspose2d(100, 512, kernel_size=(4, 4), stride=(1, 1), bias=False) -> (2, 512, 4, 4)
ConvTranspose2d(512, 256, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False) -> (2, 256, 8, 8)
ConvTranspose2d(256, 128, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False) -> (2, 128, 16, 16)
ConvTranspose2d(128, 3, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False) -> (2, 3, 32, 32)
```

1×1 → **4** (k4 s1 p0: (1 − 1)·1 + 4) → **8** → **16** → **32**, with 3 colour channels at the end: the final image is **[B, 3, 32, 32]**.

(b) How many layers for 64×64? One more `ConvTranspose2d(…, 4, 2, 1)` (32 → 64), the standard DCGAN face size; e.g. widen the first layer to 100 → 512, then 512 → 256 → 128 → 64 → 3. That is assignment option B (CelebA). CelebA cannot be downloaded on this machine and needs a GPU to train in reasonable time. On your own machine or Colab (GPU runtime): download the aligned CelebA images (the `torchvision.datasets.CelebA(root, download=True)` helper often fails because of Google Drive download limits; the Kaggle copy "celeba-dataset" is a common alternative), load the folder with `datasets.ImageFolder`, use `transforms.Resize(64)`, `transforms.CenterCrop(64)`, `ToTensor()` and `Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))`, and write a mirror D with **3 input channels** (64 → 32 → 16 → 8 → 4 → 1 score). The training loop is `train_gan()` unchanged.

(c) Why are the stamp centres larger? And what are checkerboard artifacts? The four centre values are 3 because stamps from both input pixels overlap there (1 + 2). When the **kernel size is not divisible by the stride**, some output pixels receive more overlapping stamps than others, and a faint grid appears in generated images: **checkerboard artifacts** (Odena, Dumoulin and Olah, Distill 2016). Kernel 4 with stride 2 overlaps evenly. We can see it by feeding an all-ones input through all-ones kernels:

```
fig, ax = plt.subplots(1, 2, figsize=(9, 4))
for a, k in zip(ax, [3, 4]):
    layer = nn.ConvTranspose2d(1, 1, k, 2, 1, bias=False)
    with torch.no_grad():
        layer.weight.fill_(1.0)
        o = layer(torch.ones(1, 1, 6, 6))[0, 0]
    a.imshow(o, cmap="gray"); a.set_xticks([]); a.set_yticks([])
    inner = o[2:-2, 2:-2]
    a.set_title(f"kernel {k}, stride 2: inner values {sorted(set(inner.flatten().tolist()))}")
plt.suptitle("all-ones input through an all-ones transposed conv: uneven (k=3) vs even (k=4)
overlap", y=1.0)
plt.tight_layout(); plt.show()
```



With kernel 3 and stride 2 the inner pixels receive 1, 2 or 4 overlapping stamps: a **checkerboard**. With kernel 4 and stride 2 every inner pixel receives exactly 4: flat. That is why DCGANs use k4

s2 p1.

Part 3 · The DCGAN

Why convolutions?

The Class 1 generator treats an image as 784 unrelated numbers: **each output pixel is learned on its own**, so no pixel knows its neighbours are black (grey speckle) and a "stroke" means nothing to a Linear layer. A convolution adds two assumptions that fit images:

- **Locality:** each output pixel is computed from a small neighbourhood (a 4×4 window), so neighbours decide together and backgrounds come out smooth.
- **Weight sharing:** the same filter slides over the whole image, so a stroke is learned once, not once per position.

The two networks (exactly the guide's table)

Network	Layer	Output shape
G	input z, reshaped with `z.view(B, -1, 1, 1)`	[B, 100, 1, 1]
G	`ConvTranspose2d(100, 128, 7, 1, 0)` + BatchNorm + ReLU	[B, 128, 7, 7]
G	`ConvTranspose2d(128, 64, 4, 2, 1)` + BatchNorm + ReLU	[B, 64, 14, 14]
G	`ConvTranspose2d(64, 1, 4, 2, 1)` + Tanh	[B, 1, 28, 28]
D	`Conv2d(1, 64, 4, 2, 1)` + LeakyReLU(0.2)	[B, 64, 14, 14]
D	`Conv2d(64, 128, 4, 2, 1)` + BatchNorm + LeakyReLU(0.2)	[B, 128, 7, 7]
D	`Conv2d(128, 1, 7, 1, 0)`, then `.view(-1, 1)`	[B, 1]: one raw score

G grows the image while the channel count shrinks; D is the mirror. `bias=False` is used before BatchNorm because BatchNorm has its own shift. `in_dim` and `in_channels` are arguments so Part 4 can reuse these classes with extra label inputs.

```
class DCGenerator(nn.Module):
    def __init__(self, in_dim=Z_DIM):
        super().__init__()
        self.net = nn.Sequential(
            nn.ConvTranspose2d(in_dim, 128, 7, 1, 0, bias=False), nn.BatchNorm2d(128),
nn.ReLU(True),   # 1 -> 7
            nn.ConvTranspose2d(128, 64, 4, 2, 1, bias=False), nn.BatchNorm2d(64), nn.ReLU(True),
# 7 -> 14
            nn.ConvTranspose2d(64, 1, 4, 2, 1, bias=False), nn.Tanh())
# 14 -> 28
    def forward(self, z):
        return self.net(z.view(z.size(0), -1, 1, 1))


class DCDiscriminator(nn.Module):
    def __init__(self, in_channels=1):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(in_channels, 64, 4, 2, 1, bias=False), nn.LeakyReLU(0.2, True),
# 28 -> 14
            nn.Conv2d(64, 128, 4, 2, 1, bias=False), nn.BatchNorm2d(128), nn.LeakyReLU(0.2,
```

```
True),      # 14 -> 7
            nn.Conv2d(128, 1, 7, 1, 0, bias=False))
# 7 -> 1
    def forward(self, x):
        return self.net(x).view(-1, 1)


def init_weights(m):
    if isinstance(m, (nn.Conv2d, nn.ConvTranspose2d)):
        nn.init.normal_(m.weight, 0.0, 0.02)
    elif isinstance(m, nn.BatchNorm2d):
        nn.init.normal_(m.weight, 1.0, 0.02)
        nn.init.zeros_(m.bias)
```

`init_weights` sets every conv weight from **N(0, 0.02)** and BatchNorm scales from N(1, 0.02), as in the DCGAN paper.

Shape check: one batch through G and D, printing every layer's shape, and the parameter counts.

```
torch.manual_seed(0)
G_dc, D_dc = DCGenerator(), DCDiscriminator()
G_dc.apply(init_weights); D_dc.apply(init_weights)

h = torch.randn(4, Z_DIM).view(4, -1, 1, 1)
print("G: z", tuple(h.shape))
for layer in G_dc.net:
    h = layer(h)
    if isinstance(layer, (nn.ConvTranspose2d, nn.Tanh)): print(f"
{layer.__class__.__name__:16s} -> {tuple(h.shape)}")
print("D: image", tuple(h.shape))
for layer in D_dc.net:
    h = layer(h)
    if isinstance(layer, nn.Conv2d): print(f"   {layer.__class__.__name__:16s} ->
{tuple(h.shape)}")
print(f"\nparameters   MLP G {n_params(G_mlp):>8,}   DCGAN G {n_params(G_dc):>8,}
({n_params(G_dc) / n_params(G_mlp):.1f}x)")
print(f"             MLP D {n_params(D_mlp):>8,}   DCGAN D {n_params(D_dc):>8,}  ({1 -
n_params(D_dc) / n_params(D_mlp):.0%} smaller)")
```

```
G: z (4, 100, 1, 1)
   ConvTranspose2d  -> (4, 128, 7, 7)
   ConvTranspose2d  -> (4, 64, 14, 14)
   ConvTranspose2d  -> (4, 1, 28, 28)
   Tanh             -> (4, 1, 28, 28)
D: image (4, 1, 28, 28)
   Conv2d           -> (4, 64, 14, 14)
   Conv2d           -> (4, 128, 7, 7)
   Conv2d           -> (4, 1, 1, 1)

parameters   MLP G  227,344   DCGAN G  759,680  (3.3x)
             MLP D  201,217   DCGAN D  138,624  (31% smaller)
```

The convolutional D is **smaller** than the MLP's, yet a much better critic: each small filter is reused across the whole image. A better critic is a better teacher, because G only learns through D's gradient.

The DCGAN rules and why

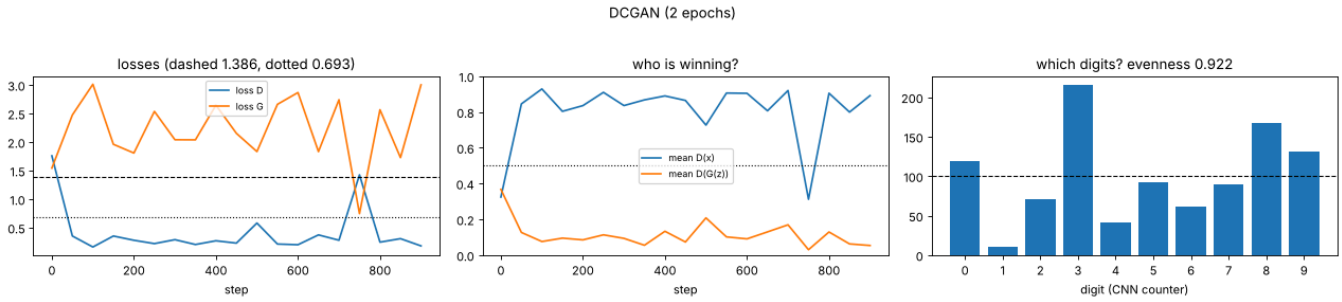
Rule	Why it helps
strided convolutions, no pooling	the network learns how to shrink and grow

Rule	Why it helps
BatchNorm in G and D	stable activations; the paper credits it with preventing G collapsing to a single point
no BatchNorm on G's output or D's input	BN on every layer caused sample oscillation and instability
ReLU in G, Tanh output	output in [−1, 1], matching the normalised images
LeakyReLU(0.2) in D	gradients still flow for negative inputs
Adam lr 0.0002, β1 = 0.5	β1 = 0.9 caused oscillation (Class 2 & 3 Run A)
weights from N(0, 0.02)	a small, standard start

> ✏️ **Predict:** after the same 2 epochs, will the DCGAN's digits look cleaner than the MLP's? Will the histogram be more even?

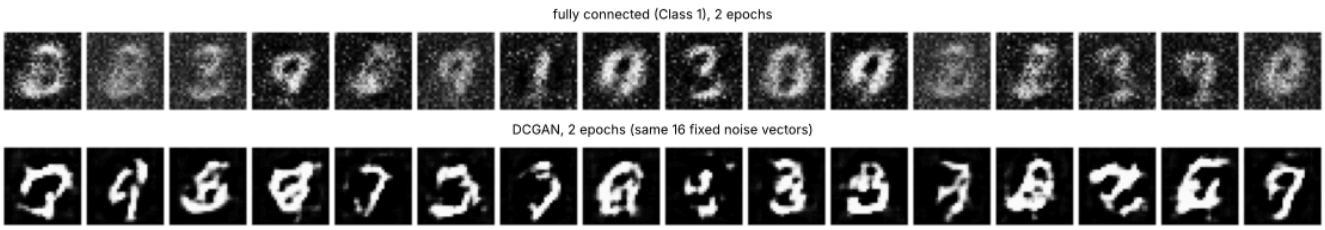
```
log_dc = train_gan(G_dc, D_dc, epochs=2)
counts_dc = show_diagnostics(G_dc, log_dc, "DCGAN (2 epochs)")
```

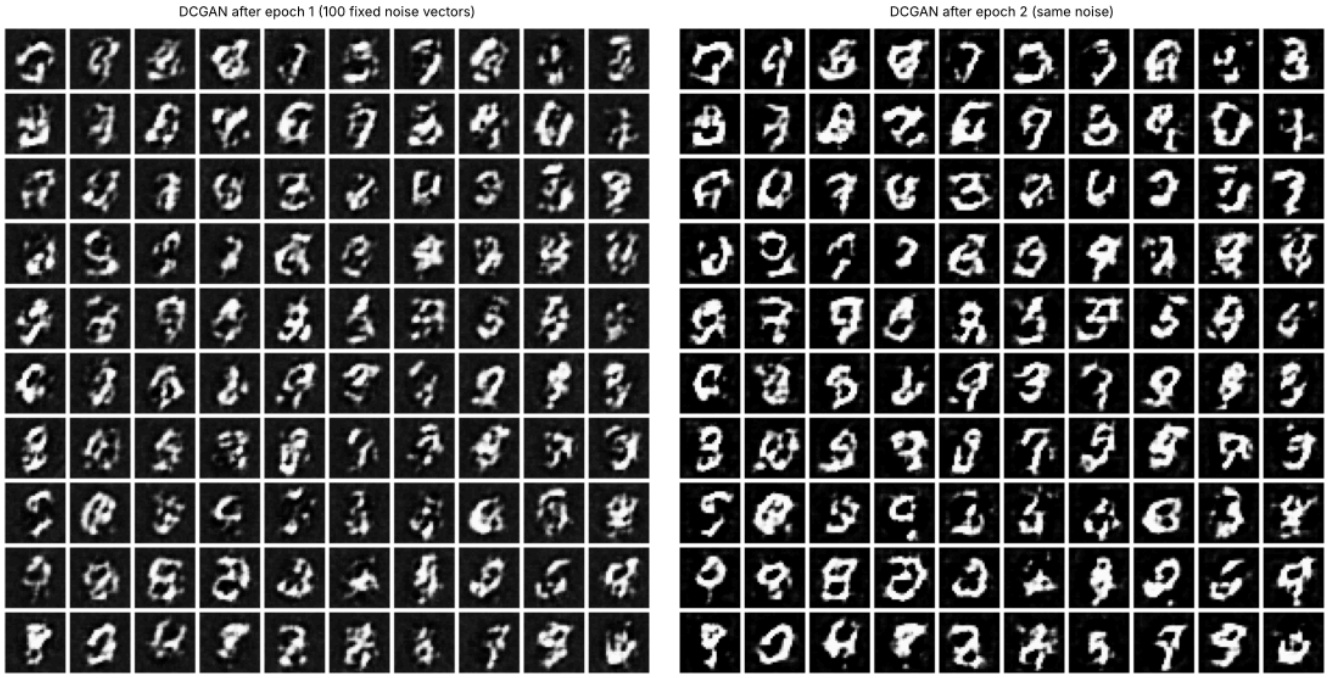
```
epoch 1/2: loss D 0.217, loss G 2.262  (124 s)
epoch 2/2: loss D 0.773, loss G 1.834  (245 s)
```




```
DCGAN (2 epochs): counts {0: 119, 1: 10, 2: 71, 3: 216, 4: 41, 5: 92, 6: 62, 7: 90, 8: 168, 9: 131}
   evenness 0.922 | rarest digit 1 (10 of 1,000) | second-epoch mean D(x) 0.80, mean D(G(z)) 0.11
```

```
fig, ax = plt.subplots(2, 1, figsize=(16, 2.6))
show_grid(log_mlp["grids"][-1][:16], nrow=16, title="fully connected (Class 1), 2 epochs",
ax=ax[0])
show_grid(log_dc["grids"][-1][:16], nrow=16, title="DCGAN, 2 epochs (same 16 fixed noise
vectors)", ax=ax[1])
plt.tight_layout(); plt.show()
fig, ax = plt.subplots(1, 2, figsize=(14, 7))
show_grid(log_dc["grids"][0], title="DCGAN after epoch 1 (100 fixed noise vectors)", ax=ax[0])
show_grid(log_dc["grids"][1], title="DCGAN after epoch 2 (same noise)", ax=ax[1])
plt.tight_layout(); plt.show()
```





What happened (this run). - **Quality: clean black backgrounds and solid, connected strokes**, no speckle. Several samples are still malformed, because 2 epochs is short (real DCGAN runs train for 25 or more).

- **Variety:** all ten digits appear: {0: 119, 1: 10, 2: 71, 3: 216, 4: 41, 5: 92, 6: 62, 7: 90, 8: 168, 9: 131}. **Evenness 0.922** (guide 0.929) vs the MLP's 0.816. Rarest digit: **1, with 10 of 1,000** (guide: 1, with 12), vs the MLP's 6 (4 of 1,000).
- **Balance:** mean D(x) mostly 0.8–0.93, mean D(G(z)) mostly 0.05–0.2 (one brief swing near step 750 where they crossed). D is clearly ahead, but D(G(z)) is not stuck at 0 and the samples improve, so G still gets a useful gradient: **case d (healthy)** from the revision checkpoint. Epoch-end losses: D 0.217 / G 2.262, then D 0.773 / G 1.834 (guide: 0.213 / 2.739, then 0.258 / 2.147; last-batch losses are noisy snapshots).

Exercise 3 · MLP vs DCGAN

	Fully connected (Class 1)	DCGAN	Guide's run (MLP → DCGAN)
parameters G / D	227,344 / 201,217	759,680 (3.3×) / 138,624 (31% smaller)	same counts
look of samples	grey speckle, blurry, broken strokes	black background, solid connected strokes, some malformed shapes	same
evenness (1,000 samples)	0.816	**0.922**	0.786 → 0.929
rarest digit	6 (4 of 1,000)	1 (10 of 1,000)	6 (0) → 1 (12)
mean D(x) / D(G(z)), epoch 2	≈ 0.70 / 0.45	≈ 0.80 / 0.11	≈ 0.7 / 0.4 → ≈ 0.9 / 0.15

A smaller D gives a better critic: each filter is reused across the whole image, so it gets more useful features from fewer weights, and a better critic is a better teacher for G.

Epoch 1 vs epoch 2: after epoch 1 the strokes are thinner, greyer and often broken; after epoch 2 (same noise vectors) they are brighter, thicker and more connected, and more cells are readable digits. The model is clearly still improving.

Optional (not run here to save time): retrain with `betas=(0.9, 0.999)`, or remove the BatchNorm layers from G. The guide warns results vary by machine and seed: report what you see and connect it to Class 2 & 3's Run A; do not expect a guaranteed collapse.

Part 4 · The conditional DCGAN: choose the digit

Classes 1, 2 and 3 looped `for real, _ in train_loader`: the `_` **is the digit label**, thrown away. An unconditional G(z) gives *some* digit, chosen by chance. To get a 7 you would have to sample, classify and throw away nine out of ten.

A **conditional GAN** plays the same minimax game with the label y added everywhere:

min over G, max over D of E[log D(x, y)] + E[log(1 − D(G(z, y), y))]

- **G(z, y)** = "draw digit y in style z".
- **D(x, y)** asks "is x a real image **of digit y**?", so it rejects two kinds of mistake: images that look fake, and real-looking images of the **wrong digit**.
- If only G had the label, D would accept a perfect 3 labelled "7", G would get no signal to obey y, and it would learn to **ignore** it. So **the label must go to both players.**

How the label goes in

Class	Change from the DCGAN	Code
`CondDCGenerator`	**110** input channels: 100 noise + 10 one-hot	`torch.cat([z, one_hot(y, 10)], dim=1)`
`CondDCDiscriminator`	**11** input channels: the image + 10 label maps of 28×28, the true label's map all ones	`torch.cat([x, y_maps], dim=1)`

Everything else is the DCGAN unchanged.

```
class CondDCGenerator(DCGenerator):
    def __init__(self):
        super().__init__(in_dim=Z_DIM + 10)                          # 110 input channels
    def forward(self, z, y):
        zy = torch.cat([z, F.one_hot(y, 10).float()], dim=1)         # [B, 100] + [B, 10] -> [B,
110]
        return super().forward(zy)


class CondDCDiscriminator(DCDiscriminator):
    def __init__(self):
        super().__init__(in_channels=1 + 10)                         # image + 10 label maps
    def forward(self, x, y):
        y_maps = F.one_hot(y, 10).float()[:, :, None, None].expand(-1, 10, 28, 28)   # [B, 10,
28, 28]
        return super().forward(torch.cat([x, y_maps], dim=1))        # [B, 11, 28, 28]


# what D receives for a real "5"
```

```
idx5 = int((train_data.targets == 5).nonzero()[0])
x5, y5 = train_data[idx5]
y_maps = F.one_hot(torch.tensor([y5]), 10).float()[:, :, None, None].expand(-1, 10, 28, 28)
d_input = torch.cat([x5[None], y_maps], dim=1)[0]
print("D's input for one real 5:", tuple(d_input.shape))
fig, axes = plt.subplots(1, 11, figsize=(16, 1.9))
for c in range(11):
    axes[c].imshow(d_input[c], cmap="gray", vmin=-1 if c == 0 else 0, vmax=1)
    axes[c].set_title("image" if c == 0 else f"label map {c - 1}", fontsize=8);
axes[c].set_xticks([]); axes[c].set_yticks([])
    for s in axes[c].spines.values(): s.set_edgecolor("orange" if c == 6 else "grey")
plt.suptitle("the 11 channels D sees for a real '5': the digit, then nine black maps and one white map (channel for 5)", y=1.08)
plt.show()
```

```
D's input for one real 5: (11, 28, 28)
```

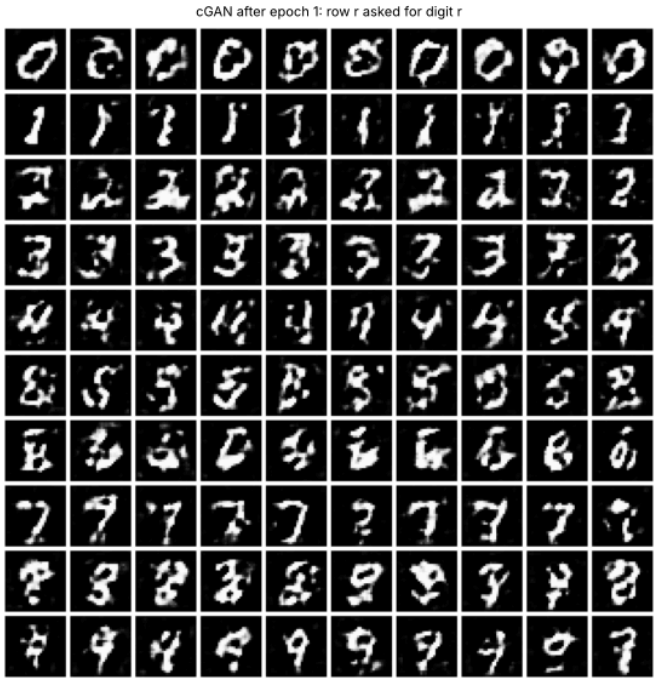


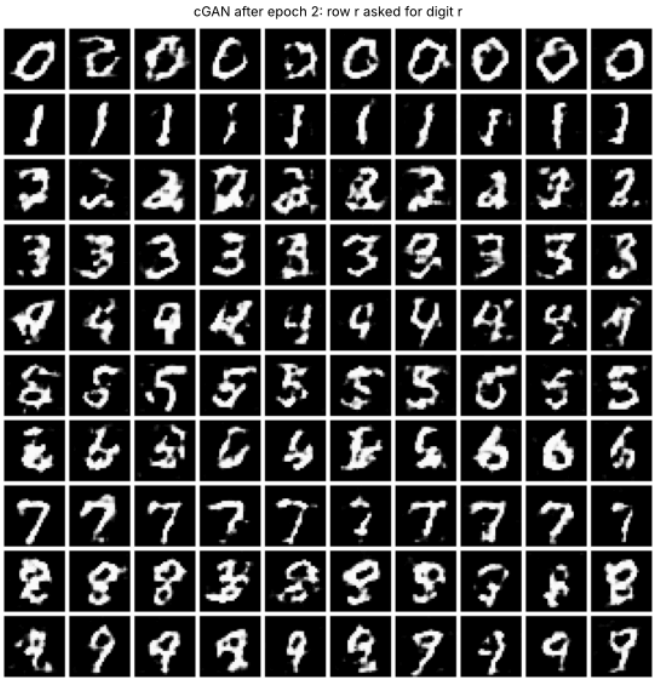
> ✏️ **Predict:** in `train_gan(..., conditional=True)`, G draws the **real batch's labels**, and D scores each fake **with the same label**. What would go wrong if D scored each fake with a *different*, random label?

Answer (from the guide). D would mostly see mismatched pairs and could learn "mismatch = fake" as a shortcut; worse, G's gradient would push it towards the label D used, not the one G was given, so G could never learn to obey y.

```
torch.manual_seed(0)
G_c, D_c = CondDCGenerator(), CondDCDiscriminator()
G_c.apply(init_weights); D_c.apply(init_weights)
print(f"parameters: G {n_params(G_c):,}  D {n_params(D_c):,}")
log_c = train_gan(G_c, D_c, epochs=2, conditional=True)
fig, ax = plt.subplots(1, 2, figsize=(14, 7.3))
show_grid(log_c["grids"][0], title="cGAN after epoch 1: row r asked for digit r", ax=ax[0])
show_grid(log_c["grids"][1], title="cGAN after epoch 2: row r asked for digit r", ax=ax[1])
plt.tight_layout(); plt.show()
```

```
parameters: G 822,400  D 148,864
epoch 1/2: loss D 1.193, loss G 2.262  (148 s)
epoch 2/2: loss D 0.940, loss G 2.061  (295 s)
```





What happened (this run). Already after epoch 1, row r shows digit r in almost every cell; after epoch 2 the shapes are cleaner. **Within a row the samples differ** (thick, thin, slanted, open, closed): that variety comes from z. The weakest rows are 2, 8 and 9 (and a few malformed 6s). As the guide says, the shapes look cleaner than the unconditional DCGAN's after the same training: knowing the digit makes G's job easier. Epoch-end losses: D 1.193 / G 2.262, then D 0.940 / G 2.061 (guide: 1.413 / 2.623, then 0.856 / 0.844).

4.1 · Conditional accuracy: did G obey the label?

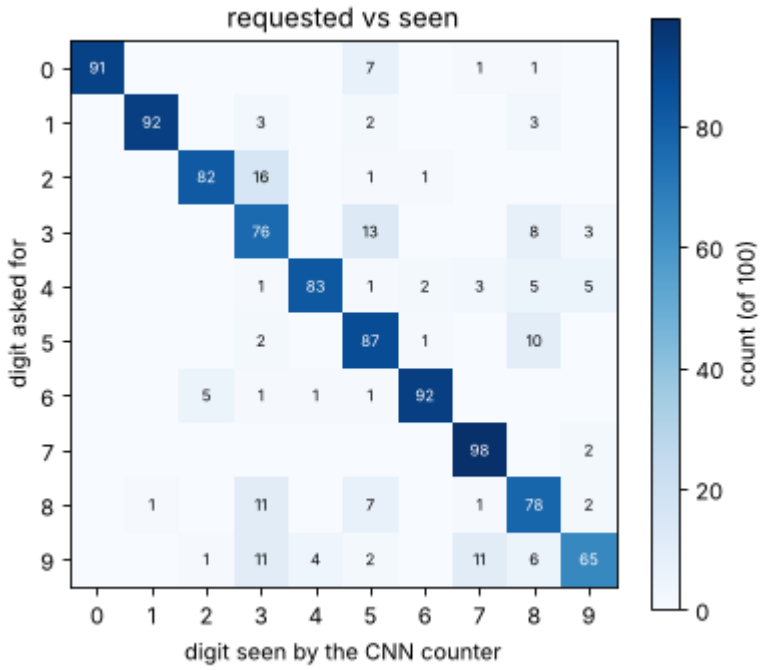
`conditional_accuracy(G_c)` generates **100 samples per digit** and counts how many the CNN counter reads as the requested digit.

```
def conditional_accuracy(G, n_per_digit=100, seed=2):
    gen = torch.Generator().manual_seed(seed)
    asked = torch.arange(10).repeat_interleave(n_per_digit)
    z = torch.randn(len(asked), Z_DIM, generator=gen)
    seen = classify(generate(G, z.to(device), asked.to(device)))
    confusion = np.zeros((10, 10), int)                    # rows = asked, columns = seen
    for a, s in zip(asked.tolist(), seen.tolist()):
        confusion[a, s] += 1
    return (seen == asked).float().mean().item(), confusion


acc_c, conf_c = conditional_accuracy(G_c)
print(f"conditional accuracy: {acc_c:.1%}  (guide's run: 84.4%)")
print("digit          " + " ".join(f"{d:>4d}" for d in range(10)))
print("accuracy (%) " + " ".join(f"{conf_c[d, d]:>4d}" for d in range(10)))
print("guide (%)    " + " ".join(f"{v:>4d}" for v in [97, 92, 84, 82, 73, 88, 82, 93, 84, 69]))
off = [(conf_c[a, s], a, s) for a in range(10) for s in range(10) if a != s]
print("largest confusions (asked -> seen, out of 100):", ", ".join(f"{a} -> {s} ({n})" for n, a,
s in sorted(off, reverse=True)[:5]))

plt.figure(figsize=(5.5, 4.8))
plt.imshow(conf_c, cmap="Blues"); plt.colorbar(label="count (of 100)")
for (a, s), v in np.ndenumerate(conf_c):
    if v: plt.text(s, a, v, ha="center", va="center", fontsize=7, color="w" if v > 50 else "k")
plt.xticks(range(10)); plt.yticks(range(10)); plt.xlabel("digit seen by the CNN counter");
plt.ylabel("digit asked for")
plt.title("requested vs seen"); plt.show()
```

```
conditional accuracy: 84.4%  (guide's run: 84.4%)
digit           0    1    2    3    4    5    6    7    8    9
accuracy (%)   91   92   82   76   83   87   92   98   78   65
guide (%)      97   92   84   82   73   88   82   93   84   69
largest confusions (asked -> seen, out of 100): 2 -> 3 (16), 3 -> 5 (13), 9 -> 7 (11), 9 -> 3 (1
1), 8 -> 3 (11)
```



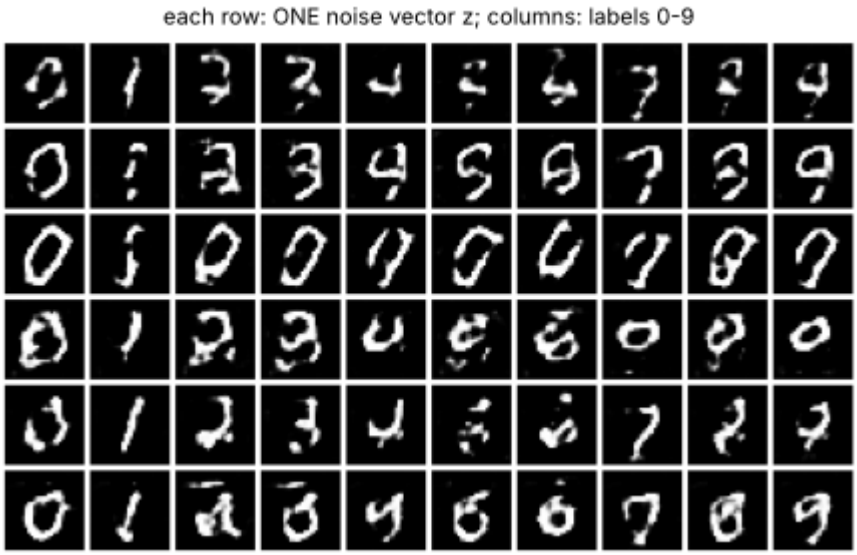
This run: 84.4%, the same headline number as the guide's run (84.4%). That is partly coincidence: the per-digit split differs. Ours is best for 7 (98%), 1 and 6 (92%), and worst for **9 (65%)**, 3 (76%) and 8 (78%); the guide's was best for 0 (97%) and 7 (93%), worst for 9 (69%) and 4 (73%). Largest confusions here: 2 → 3 (16), 3 → 5 (13), 9 → 7 (11), 9 → 3 (11), 8 → 3 (11); three of the guide's four pairs (9 → 7, 2 → 3, 3 → 5) appear again. These pairs share strokes. Part of the 15.6% is the counter's own error on unusual handwriting (it is 98.1% accurate on real digits).

4.2 · Content vs style

`style_vs_content` uses **one noise vector per row** with labels 0–9 across the columns. If the label controls content and the noise controls style, each row should show ten different digits **in the same handwriting**.

```python
def style_vs_content(G, n_rows=6, seed=3):
    z = torch.randn(n_rows, Z_DIM, generator=torch.Generator().manual_seed(seed))
    z = z.repeat_interleave(10, dim=0)                    # each z used 10 times...
    y = torch.arange(10).repeat(n_rows)                   # ...with labels 0..9
    show_grid(generate(G, z.to(device), y.to(device)), nrow=10, title="each row: ONE noise vector
z; columns: labels 0-9", size=7)
    plt.show()

style_vs_content(G_c)
```



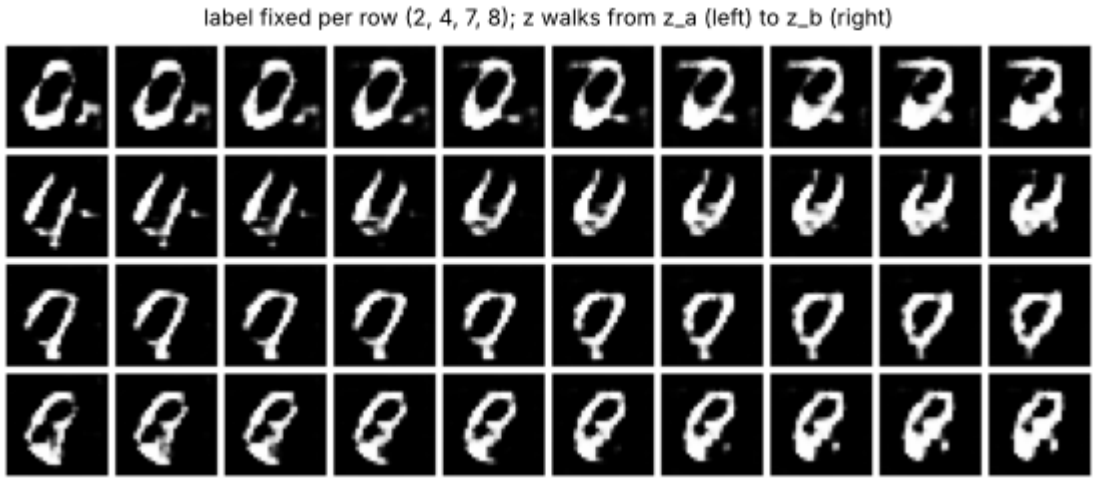
Content vs style (this run). Along each row the **digit** changes with the label while the **handwriting** stays similar: row 2 has thin strokes, row 3 thick, bold loops, row 6 neat medium strokes. The label controls content; the noise controls style. Some noise vectors give messy styles after only 2 epochs: in row 3 the labels 2 to 4, and in row 4 the labels 7 to 9, come out as round, 0-like loops, so those z's encode a "round" style that G cannot yet combine with every digit.

4.3 · Interpolation

`interpolate` keeps the **label fixed** and walks z in 10 steps in a straight line from z_a to z_b.

```
def interpolate(G, label, steps=10, seed=4):
    gen = torch.Generator().manual_seed(seed)
    z_a, z_b = torch.randn(1, Z_DIM, generator=gen), torch.randn(1, Z_DIM, generator=gen)
    t = torch.linspace(0, 1, steps).view(-1, 1)
    z = (1 - t) * z_a + t * z_b
    return generate(G, z.to(device), torch.full((steps,), label, device=device))

rows = torch.cat([interpolate(G_c, d) for d in [2, 4, 7, 8]])
show_grid(rows, nrow=10, title="label fixed per row (2, 4, 7, 8); z walks from z_a (left) to z_b
(right)", size=9)
plt.show()
```



Interpolation (this run). With the label fixed, the image changes **gradually** from left to right: slant, thickness and loop shape morph step by step, with no sudden jumps. G has learned a smooth

style space (a theme of the VAE module next class). Not every intermediate is clean: the "2" row starts from a 0-like loop that only becomes a clear 2 towards the right, another sign that 2 epochs is short training.

Exercise 4

1. **Your conditional accuracy, hardest digits and their confusions: 84.4%** in this run (80–90% after 2 epochs is typical, says the guide). Hardest digits: 9 (65%, often seen as 7 or 3), 3 (76%, seen as 5) and 8 (78%, seen as 3).
2. **What does one z fix across all ten digits?** The handwriting **style**: thickness, slant, size, loop shape (Part 4.2).
3. **Mismatch test:** does D check the **match** between image and label, or only realism? Score 200 generated 7s with label 7 and with label 1.

```
D_c.eval()
z = torch.randn(200, Z_DIM, generator=torch.Generator().manual_seed(5)).to(device)
sevens = generate(G_c, z, torch.full((200,), 7, device=device)).to(device)
with torch.no_grad():
    s7 = torch.sigmoid(D_c(sevens, torch.full((200,), 7, device=device))).mean().item()
    s1 = torch.sigmoid(D_c(sevens, torch.full((200,), 1, device=device))).mean().item()
D_c.train()
print(f"D's mean P(real) for 200 generated 7s:  labelled 7 -> {s7:.3f}   labelled 1 -> {s1:.3f}
(guide: 0.315 vs 0.141)")
```

```
D's mean P(real) for 200 generated 7s:  labelled 7 -> 0.248   labelled 1 -> 0.063   (guide: 0.315
vs 0.141)
```

Answer (this run): D's mean P(real) for the same 200 generated 7s is **0.248 with label 7** and **0.063 with label 1** (guide: 0.315 vs 0.141). The image is identical; only the label changed, and D's score dropped about fourfold. **D has learned the image–label match**, not only realism. That is exactly the signal that teaches G to obey y. (Both numbers are well below 0.5 because D is ahead of G, as in Part 3.)

4. **Can a cGAN still collapse?** Yes: it can collapse **within a class** (every 7 identical). A row of the 10×10 grid or the style-vs-content grid shows it; **conditional accuracy does not**, because it measures control, not variety. In our grids there is no sign of within-class collapse yet: every row of the 10×10 grid contains clearly different handwriting.

Part 5 · Wrap-up

What changed from the unconditional GAN (summary for the viva)

	Unconditional (Classes 1–3, Part 3)	Conditional (Part 4)
data loop	`for real, _ in train_loader` (label thrown away)	`for real, labels in train_loader` (label used)
G's input	z: 100 numbers	z + one-hot(y): 110 numbers
D's input	the image (1 channel)	the image + 10 label maps (11 channels)
D's question	"is x real?"	"is x a real image **of digit y**?"
D step	real → 1, G(z) → 0	(real, its label) → 1, (G(z, y), y) → 0

	Unconditional (Classes 1–3, Part 3)	Conditional (Part 4)
G step	make D(G(z)) → 1	make D(G(z, y), y) → 1
what you control	nothing: the digit is chosen by chance	**content** by y, **style** by z
new measurement	digit histogram (variety)	conditional accuracy (control)

The loss function, optimiser, alternating D/G steps and architecture body are **unchanged**.

Reflection questions (model answers)

1. **Why does the fully connected G leave speckle?** It learns each of the 784 outputs separately, so neighbouring background pixels never agree. Convolutions compute each output from a small neighbourhood with shared filters, so backgrounds come out smooth and strokes connected.
2. **`ConvTranspose2d(k=4, s=2, p=1)` on 16×16?** (16 − 1)·2 − 2 + 4 = **32×32**. This choice exactly doubles any size and mirrors `Conv2d(k=4, s=2, p=1)`, which exactly halves, so G and D can be mirror images.
3. **Why must D see the label?** To reject realistic digits of the wrong class. If only G had the label, G would get no signal to obey it and would learn to ignore y.
4. **Can a cGAN collapse?** Yes, within a class (all 7s identical). The rows of the 10×10 grid or the style-vs-content grid show it.

Exit ticket (from the slides) with answers

1. *D(x) ≈ 0.99 and D(G(z)) ≈ 0.01 for 2,000 steps; the grid is frozen.* D too strong; vanishing gradient for G.
2. *`ConvTranspose2d(k4, s2, p1)` layers after a 7×7 start: size after two of them?* **28×28** (7 → 14 → 28).
3. *In a cGAN, what does D receive, and why does it need the label?* The image + label maps; otherwise G ignores y.
4. *Fixed z, labels 0 to 9: what stays the same across the row?* The style (thickness, slant).

Assignment (choose one)

Option	Task	Submit
A · Fashion-MNIST	train today's DCGAN on `datasets.FashionMNIST` (same 28×28 size)	fixed-noise grid per epoch and a short note on what improved
B · DCGAN faces	extend the Class 1 `DCGANGenerator` to 64×64, write a mirror D, train on CelebA (GPU recommended; see Exercise 2b for how)	samples and a short discussion of failures seen
C · Embedding	replace the one-hot in `CondDCGenerator` with `nn.Embedding(10, 50)`	conditional accuracy compared with today's version

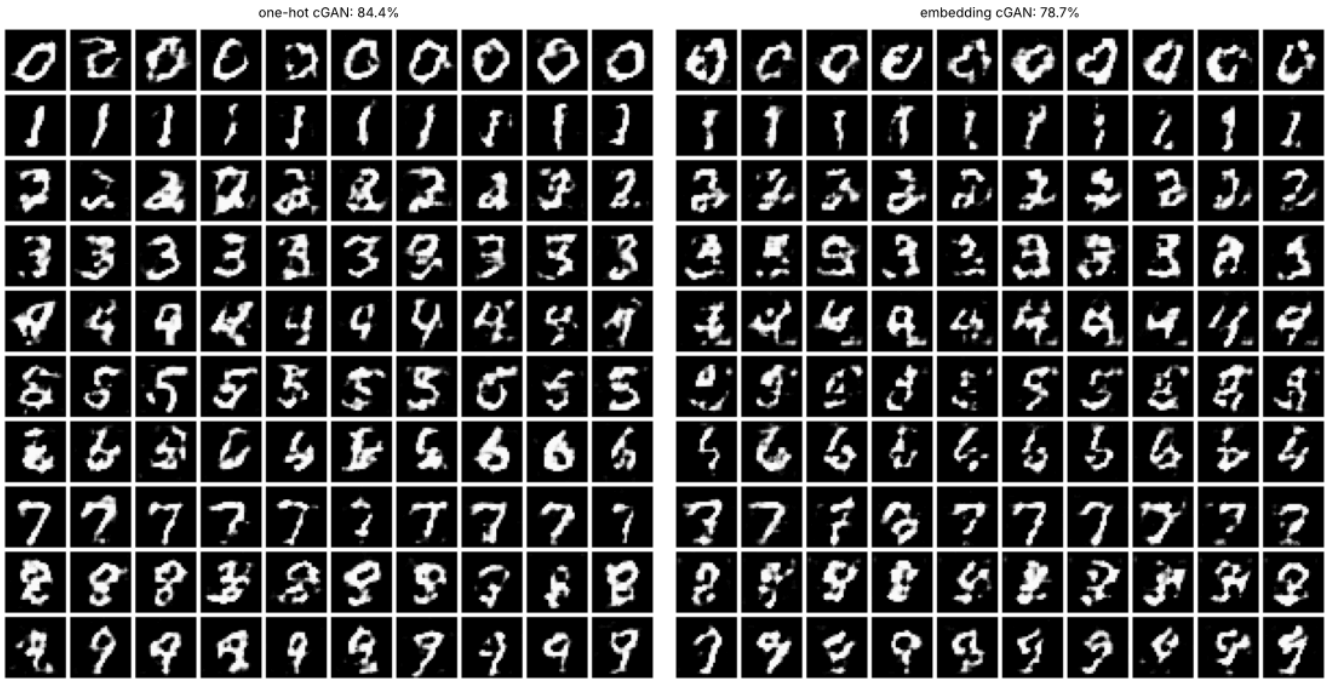
Assignment option C, done: `nn.Embedding(10, 50)` instead of one-hot

A one-hot label is a fixed code. An **embedding** is a small trainable table: each digit gets **50 learned numbers**, trained along with G, so G can learn its own description of each digit. G's input becomes 100 + 50 = **150** channels. D is left exactly as in Part 4 (image + 10 one-hot label maps), so only G's way of reading the label changes. Same seed, same 2 epochs.

```
class EmbedDCGenerator(DCGenerator):
    def __init__(self, embed_dim=50):
        super().__init__(in_dim=Z_DIM + embed_dim)              # 150 input channels
        self.embed = nn.Embedding(10, embed_dim)                # a learned 50-number code per
digit
    def forward(self, z, y):
        return super().forward(torch.cat([z, self.embed(y)], dim=1))

torch.manual_seed(0)
G_e, D_e = EmbedDCGenerator(), CondDCDiscriminator()
G_e.apply(init_weights); D_e.apply(init_weights)
log_e = train_gan(G_e, D_e, epochs=2, conditional=True)
acc_e, conf_e = conditional_accuracy(G_e)
print(f"\nconditional accuracy   one-hot (Part 4): {acc_c:.1%}   nn.Embedding(10, 50):
{acc_e:.1%}")
print("per digit, one-hot  :", [int(conf_c[d, d]) for d in range(10)])
print("per digit, embedding:", [int(conf_e[d, d]) for d in range(10)])
fig, ax = plt.subplots(1, 2, figsize=(14, 7.3))
show_grid(log_c["grids"][-1], title=f"one-hot cGAN: {acc_c:.1%}", ax=ax[0])
show_grid(log_e["grids"][-1], title=f"embedding cGAN: {acc_e:.1%}", ax=ax[1])
plt.tight_layout(); plt.show()
```

```
epoch 1/2: loss D 0.700, loss G 2.297  (152 s)
epoch 2/2: loss D 1.227, loss G 1.656  (305 s)
conditional accuracy   one-hot (Part 4): 84.4%   nn.Embedding(10, 50): 78.7%
per digit, one-hot  : [91, 92, 82, 76, 83, 87, 92, 98, 78, 65]
per digit, embedding: [95, 67, 74, 79, 87, 89, 82, 83, 74, 57]
```



Result (this run, one seed, 2 epochs): the embedding version reached **78.7%** conditional accuracy vs **84.4%** for one-hot. Per digit it was better for 0, 3, 4 and 5, but clearly worse for 1 (67% vs 92%), 2, 7, 8 and 9, and its grid looks messier.

How to read this honestly: the embedding is not "worse" in general. A one-hot code is a fixed, perfectly distinct signal from the first step, while the 50-number embedding starts random and must be **learned at the same time** as everything else; with only 2 epochs that is one more thing to learn. With longer training, the two usually end up similar; embeddings matter more when there are many classes (one-hot vectors grow with the number of classes, an embedding stays the size you choose). One run with one seed cannot settle it: for a proper submission, train both longer and on two or three seeds and compare the averages.

Training time in this run

```
for name, log in [("MLP baseline", log_mlp), ("DCGAN", log_dc), ("conditional DCGAN", log_c),
("embedding cGAN (assignment C)", log_e)]:
    print(f"{name:30s} {log['seconds']:6.0f} s for 2 epochs")
print("guide (2-core CPU): MLP 18 s, DCGAN 153 s, cGAN 199 s")
```

```
MLP baseline                    19 s for 2 epochs
DCGAN                          245 s for 2 epochs
conditional DCGAN              295 s for 2 epochs
embedding cGAN (assignment C)  305 s for 2 epochs
guide (2-core CPU): MLP 18 s, DCGAN 153 s, cGAN 199 s
```

These times depend heavily on the machine and on what else it is running. This notebook was executed on a shared 4-core CPU server that was busy with other jobs, so the times above are slower than the guide's 2-core test run. On a GPU (e.g. Colab with a GPU runtime) every cell runs in seconds. If training is too slow on your laptop, the guide suggests training Part 3 with `epochs=1` and keeping 2 epochs for Part 4.

Next class: autoencoders and VAEs (encoder, bottleneck, decoder, reconstruction loss).

Source: reconstructed from the course guide "GAN Class 4 Notebook Explained" (GAN-Class-4-DCGAN-cGAN.ipynb) and the Class 4 slides; the DCGAN skeleton from the Class 1 guide (CV-Week4-Lecture1.ipynb, cells 6–8). Papers: Radford, Metz and Chintala 2015 (DCGAN); Mirza and Osindero 2014 (cGAN); Odena, Dumoulin and Olah 2016 (checkerboard artifacts).

CV Week 4 · GANs Cheat Sheet

Class 1 · The framework

- **GAN = Generative Adversarial Network(s)** (Goodfellow et al., 2014). **Generative** = creates data · **Adversarial** = two opponents with opposite goals · **Networks** = both are neural nets.
- **Generator G** (forger/counterfeiter): noise **z** (100 numbers, `torch.randn`) → image. **Never sees real data**; learns only through D's gradient. **Tanh** output (matches Normalize(0.5, 0.5) → −1…1). Kept after training.
- **Discriminator D** (inspector/police): image → **P(real)** via **Sigmoid**. LeakyReLU(0.2). Labels for free: real = 1, fake = 0. Usually discarded.
- Notebook nets: G 100 → 256 → 784 (Tanh), D 784 → 256 → 1 (Sigmoid), `BCELoss`, Adam lr 2e-4, batch 64, 2 epochs.
- **Step 1 (D):** real → 1, `fake.detach()` → 0, `loss_d = loss_real + loss_fake`, only `optimizer_d.step()`.
- **Step 2 (G):** `D(fake)` (no detach) vs **labels_real** = the **non-saturating** loss −log D(G(z)), strong gradients early. Only `optimizer_g.step()`.
- **BCE(p, y) = −[y log p + (1 − y) log(1 − p)]** · examples: (0.9, 1) → 0.105 · (0.1, 1) → 2.303 · (0.2, 0) → 0.223 · G on the same fake → 1.609.
- **Minimax:** min_G max_D E[log D(x)] + E[log(1 − D(G(z)))] · zero-sum.
- **Optimal D*** = p_data/(p_data + p_g) · ideal: **p_g = p_data, D = ½**, loss_d = 2 ln 2 ≈ **1.386**, loss_g = ln 2 ≈ **0.693**.
- **Losses don't measure quality: look at the images.** Fixed-noise grids show learning; interpolating z shows a smooth latent space.

Classes 2 & 3 · Training dynamics

- A GAN chases a **moving target**: a **saddle point / equilibrium**, not a minimum.
- **Nash equilibrium:** no player gains by changing alone. GAN's: **p_g = p_data, D = ½**. It exists, but training needn't reach it.
- **x·y game:** x ← x − lr·y, y ← y + lr·x → simultaneous **spirals out** (1.41 → 3.83), alternating **orbits**. A smaller lr only slows the circling.
- **Instability:** oscillating losses · quality going backwards · one player wins (D too strong → G's gradient vanishes) · fragile settings. Momentum overshoots → **Adam β1 = 0.5** (DCGAN).
- **Who is winning:** D(x) ≈ 1 and D(G(z)) ≈ 0 = D too strong · 0.6–0.8 / 0.2–0.4 = healthy · ≈ 0.5 / 0.5 = can't tell (check the samples) · swinging = oscillation.
- **Mode collapse:** few modes (e.g. one digit); **hopping** = the collapsed output jumps around. Cause: D judges one sample at a time. Detect with 64+ sample grids, fixed-noise grids, class histograms (read with the grid), coverage.
- **Toolbox:** non-saturating loss (early gradients) · β1 0.5 (oscillation) · label smoothing 0.9 and instance noise (over-confident D) · TTUR (D lagging) · **minibatch std/discrimination (collapse)** · spectral norm · WGAN-GP (distance-based critic).
- **Ring results:** only **minibatch std** and **z_dim = 2** fixed the collapse (8/8). **Match the fix to the cause.**

Class 4 · DCGAN & conditional GAN

- **Why convolutions:** locality + weight sharing → smooth backgrounds, connected strokes. Conv D smaller (139k vs 201k) but a better critic.
- **Sizes:** Conv2d **(n + 2p − k)//s + 1** (k4 s2 p1 halves) · ConvTranspose2d **(n − 1)s − 2p + k** (k4 s2 p1 doubles) · ConvT(100, 128, 7, 1, 0): 1×1 → **7×7**.
- **Stamp demo:** each input pixel stamps the kernel, overlaps add ([[1, 0], [0, 2]] → centre 3) · uneven overlap → **checkerboard artifacts** (k4 s2 is even).
- **DCGAN 28×28:** G 100×1×1 → 128×7×7 → 64×14×14 → 1×28×28 (Tanh) · D the mirror → 1 raw score (`BCEWithLogitsLoss`).
- **Rules:** strided convs, no pooling · BatchNorm in G and D, but **not** on G's output or D's input · ReLU in G / Tanh output · LeakyReLU(0.2) in D · Adam 2e-4, β1 0.5 · weights from N(0, 0.02).
- **cGAN** (Mirza & Osindero 2014): min_G max_D E[log D(x, y)] + E[log(1 − D(G(z, y), y))]. **The label goes to BOTH players**, otherwise G ignores it. G: z + one-hot (110 channels). D: image + 10 label maps (11 channels).
- **Conditional accuracy** = share of fakes classified as the requested digit (84.4%). It measures **control**, not variety (a cGAN can still collapse within a class).
- **Label = content (which digit), noise = style (handwriting).** Conditions can be labels, images (pix2pix), domains (CycleGAN) or text (text-to-image).